

UNIVERSITÀ DEGLI STUDI DI MILANO
FACOLTÀ DI SCIENZE E TECNOLOGIE

DIPARTIMENTO DI INFORMATICA
GIOVANNI DEGLI ANTONI

Corso di Laurea triennale in
Informatica Musicale

EMULAZIONE COME STRUMENTO VIRTUALE VST
DEL SINTETIZZATORE ANALOGICO MINIMOOG

Relatore: Prof. Luca Andrea Ludovico
Correlatore: Prof. Giorgio Presti

Tesi di Laurea di:
Davide Bianchi
Matr. Nr. 938447

ANNO ACCADEMICO 2020-2021

“Senza fretta, ma senza tregua”

- Lev Tolstoj

Ringraziamenti

Prima di procedere con la trattazione, vorrei ringraziare il Professor Presti per avermi dato l'opportunità di lavorare negli scorsi mesi a questo progetto per me molto importante dal punto di vista formativo, e per avermi fornito consigli essenziali durante il suo sviluppo. Un sentito grazie va ai miei genitori Enrico e Nadia per il loro supporto morale ed economico, non solo in questi ultimi anni universitari, ma sin dalle scuole elementari quando mi ascoltavano mentre ripetevo le lezioni di scienze. Loro più di tutti, e prima di tutti, hanno cercato di spiegarmi quanto studiare fosse importante, e dopo 16 anni da studente penso di averlo capito. Infine, vorrei dedicare un grazie a tutti i miei compagni di viaggio, universitari e non, con i quali in questi ultimi anni ho condiviso esperienze che mi hanno messo alla prova, ispirato, consentito di crescere e soprattutto divertirmi tra una sessione di esame e la successiva.

Indice

Ringraziamenti	ii
Indice	iii
1 Introduzione	1
1.1 Cenni storici	1
1.2 Il significato di <i>Virtual Analog</i>	1
1.3 Organizzazione della tesi	2
2 Stato dell'arte	3
2.1 Bandlimited Impulse Train (BLIT)	3
2.2 Filtro ladder del Minimoog	5
2.3 Applicazioni commerciali	6
3 Tecnologie utilizzate	7
3.1 C++	7
3.2 JUCE	7
3.3 Projucer	8
3.4 Visual Studio	8
3.5 Digital Audio Workstation (DAW)	9
3.6 MATLAB	9
4 Emulazione virtuale del Minimoog	10
4.1 Schema a blocchi del circuito	10
4.2 Caratteristiche generali del sintetizzatore	10
4.2.1 Oscillator Bank	11
4.2.2 Mixer	13
4.2.3 Modifiers	13
4.2.4 Output	15
4.2.5 Controllers	16
4.3 Implementazione software	17

4.3.1	Elenco e breve descrizione delle classi	18
4.3.2	Flusso generale del programma	19
5	Bandlimited Impulse Trains	22
5.1	Introduzione	22
5.2	Rappresentazione dell'impulso nel dominio tempo-discreto	22
5.3	Bandlimited Impulse Train	25
5.3.1	Sum of Windowed Sincs (BLIT-SWS)	25
5.4	Generazione delle forme d'onda da BLIT	29
5.4.1	Dente di sega da BLIT	29
5.4.2	Onda rettangolare da BP-BLIT (BLIT bipolare)	31
5.4.3	Onda triangolare da rettangolare	33
5.4.4	Onda Triangolare/Dente di sega	34
5.5	Gestione dei DC offset	34
5.6	Forme d'onda generate dall'algoritmo	35
6	Il filtro ladder del Minimoog	40
6.1	Introduzione ai filtri	40
6.2	Filtri tempo-continui	41
6.2.1	I filtri RC	41
6.2.2	Funzione di trasferimento	42
6.2.3	Filtro passa-basso ad 1 polo	44
6.2.4	Filtro ladder lineare del Minimoog	45
6.2.5	Filtro ladder non lineare del Minimoog	48
6.3	Filtri tempo-discreti	51
6.3.1	Integrazione discreta naive	51
6.3.2	Filtro passa-basso naive	53
6.4	Diagramma a blocchi	53
6.5	Funzione di trasferimento	54
6.6	Integrazione trapezoidale	55
6.7	Trasformata Bilineare	60
6.8	Cutoff Prewarping	62
6.9	Zero-Delay Feedback	64
6.10	Implementazione del filtro	67
6.10.1	Filtro passa-basso ad un polo lineare	67
6.10.2	Filtro passa-basso a 4 poli lineare	68
6.10.3	Equazione zero-delay feedback non lineare	70
6.10.4	Filtro passa-basso a 4 poli non lineare	71

7	Testing	78
7.1	Oscillatori	78
7.1.1	Onda dente di sega	79
7.1.2	Onda quadra	79
7.1.3	Grafici dell'onda dente di sega	80
7.1.4	Grafici dell'onda quadra	85
7.2	Filtro	89
8	Conclusioni	94
8.1	Conclusioni	94
8.2	Sviluppi futuri	94
A	Informazioni aggiuntive sul testing degli oscillatori	97
A.0.1	Onda dente di sega	97
A.0.2	Onda quadra	99
	Bibliografia	103

Capitolo 1

Introduzione

1.1 Cenni storici

Il Minimoog Model D è un sintetizzatore analogico creato nel 1970 dalla compagnia americana Moog Music. Esso fu il primo synth ad essere portato fuori dai laboratori universitari e gli studi di registrazione e suonato direttamente sui palchi di tutto il mondo grazie alla sua dimensione ridotta, al costo più accessibile e alla semplicità d'uso del pannello di controllo.

Prima degli anni '70 venivano principalmente usati i sintetizzatori modulari nei quali i moduli erano interconnessi tra di loro per generare suoni di varia natura. Essi erano ingombranti, con costi proibitivi e non di semplice utilizzo, mentre nel Minimoog il pannello frontale raggruppa tutti i componenti principali e rende la sintesi dell'audio intuibile anche ai meno esperti.

1.2 Il significato di *Virtual Analog*

L'obiettivo di questo elaborato è quello di creare un plugin conforme allo standard VST3 che emula il comportamento del Minimoog.

Con il termine *Virtual Analog* si intende ricreare il suono dei sintetizzatori analogici attraverso degli algoritmi implementati digitalmente.

In particolare si vuole:

- Emulare il banco oscillatori usando tecniche efficienti e permettendo la modulazione in ampiezza e in frequenza delle forme d'onda generate cercando di ridurre il più possibile il fenomeno dell'aliasing.
- Emulare il comportamento del filtro ladder e le sue non linearità permettendo la modulazione della frequenza di taglio a frequenze audio. Non è stata fatta

un'emulazione precisa del circuito ma è stato implementato un modello che rispecchiasse i comportamenti principali del filtro.

1.3 Organizzazione della tesi

La tesi è organizzata nel seguente modo:

Nel **Capitolo 2** è stata fatta una raccolta e un riassunto degli articoli scientifici riguardanti il BLIT e la modellazione del filtro ladder del Minimoog. Inoltre, sono state menzionate anche due applicazioni software che emulano il sintetizzatore.

Nel **Capitolo 3** è stata fatta una lista delle tecnologie utilizzate per lo sviluppo del progetto.

Nel **Capitolo 4** viene fatta una descrizione dei blocchi del circuito del Minimoog e delle caratteristiche principali della versione implementata. In aggiunta, viene descritta l'implementazione software ad alto livello spiegando il flusso del programma e la funzione di ogni classe che si occupa di elaborare il segnale.

Il **Capitolo 5** è interamente dedicato alla descrizione della tecnica di sintesi *Band-limited Impulse Train* e uno dei metodi per poterla generare ovvero la *Sum of Windowed Sincs*. Viene poi spiegato come generare la forma d'onda desiderata partendo dal treno di impulsi.

Il **Capitolo 6** è dedicato interamente al filtro ladder del Minimoog. In particolare, vengono prima descritti il filtro passa-basso ad un polo, il filtro ladder lineare e il filtro ladder non lineare nel dominio tempo-continuo e, successivamente, ognuno di questi filtri viene rivisitato nel dominio tempo-discreto.

Il **Capitolo 7** contiene una descrizione della fase di testing degli oscillatori e del filtro del Minimoog. Per quanto riguarda gli oscillatori è stato testata la loro accordatura con l'analizzatore di spettro ed è poi stato fatto un confronto (sempre a livello spettrale) tra le onde generate con la tecnica BLIT e con la tecnica naive.

Il filtro è stato testato analizzando la sua risposta in frequenza e la distorsione armonica per valori di frequenze di taglio e risonanze diverse.

Il **Capitolo 8** è dedicato alle conclusioni e agli sviluppi futuri del progetto.

Capitolo 2

Stato dell'arte

2.1 Bandlimited Impulse Train (BLIT)

Bandlimited Impulse Trains è una tecnica di sintesi sonora che permette di generare artificialmente un suono limitato in banda. Di seguito ci sono alcuni articoli che la descrivono e propongono diverse varianti ed ottimizzazioni.

Il quarto capitolo della tesi [1] e il relativo articolo [2] di Timothy Scott Stilson revisionati dal dott. Julius Smith sono dedicati interamente alla sintesi digitale del suono senza aliasing.

Inizialmente viene spiegato perchè le tecniche naive generano aliasing e successivamente sono introdotte le tecniche di sintesi limitate in banda.

Viene poi spiegato la tecnica del BLIT partendo dai vari metodi per la generazione dei sinc come la DSF (Discrete Summation Formula), il metodo Sinc_M e il metodo BLIT-SWS (Sum of Windowed Sinc) fino alla generazione delle varie forme d'onda degli oscillatori analogici (dente di sega, quadra, triangolare) tramite l'integrazione e la trasformazione dei sinc generati.

L'articolo di Vesa Välimäki and Antti Huovilainen [3] esamina gli algoritmi di sintesi sottrattiva per la generazione del suono.

Gli algoritmi sono divisi in 3 categorie: limitati in banda, quasi-limitati in banda e metodi per la riduzione dell'aliasing.

Nella prima categoria fanno parte i metodi che usano le wavetable. Nella seconda categoria sono usati metodi che filtrano il segnale tempo continuo prima di campionarlo. In questa sezione ci sono versioni ottimizzate del BLIT, BLEP (Bandlimited Impulse Step) ed è introdotta la tecnica del PolyBLEP.

Nella terza categoria è presentato l'algoritmo DPW (Differential Parabolic Wave) il quale genera un'onda dente di sega approssimata con una piccola quantità di aliasing.

L'articolo di Nam, Välimäki, Abel e Smith [4] presenta algoritmi efficienti per ridurre l'aliasing usando i filtri a delay frazionario nella sintesi BLIT. Sono esaminati gli interpolatori di Lagrange e B-spline e i filtri allpass a delay frazionari. Sono anche discussi metodi ottimizzati per creare le forme d'onda classiche (dente di sega, quadra e triangolare) e anche forme d'onda armonicamente più complesse come PWM, hard-sync e supersaw.

Il risultato dell'elaborato mostra che la sintesi BLIT usando l'interpolazione B-spline del terzo ordine genera delle forme d'onda senza aliasing.

L'articolo di Pekonen, Nam, Smith e Välimäki [5] presenta una tecnica efficiente di anti-aliasing la quale si basa sulla rappresentazione di una forma d'onda attraverso la somma di funzioni approssimate da un polinomio e sfasate nel tempo. Viene dimostrato che ottimizzando i coefficienti del polinomio che approssima la funzione, l'aliasing è minimizzato e la banda delle forme d'onda classiche può essere più estesa. Viene inoltre mostrato che, nel caso migliore, un'onda dente di sega non genera aliasing fino ad una frequenza fondamentale di 10kHz.

L'articolo di Kraft e Zölzer [6] mostra la tecnica di sintesi BLIT usando un impulso Hammerich al posto del classico sinc.

Nel metodo tradizionale gli impulsi sono rappresentati da un seno cardinale il quale limita la banda del segnale nel BLIT e attraverso il quale posso impostare la frequenza di taglio. Esso mi permette di generare forme d'onda con un determinato numero di armoniche di egual ampiezza. L'impulso Hammerich, invece, può essere usato per impostare sia la frequenza di taglio che il roll-off tra banda passante e banda attenuata. Se nella sintesi BLIT il sinc viene sostituito con l'impulso Hammerich, è possibile sintetizzare una serie di segnali con uno spettro passa-basso regolabile.

L'articolo di Tassart [7] descrive come la generazione di treni di impulsi limitati in banda sia il cuore della maggior parte degli algoritmi degli oscillatori limitati in banda (BLO: Band Limited Oscillator). La maggior parte degli algoritmi BLIT sono basati su filtri con risposta all'impulso finita (FIR).

Questo articolo presenta un metodo innovativo per generare BLIT basandosi su filtri con risposta all'impulso infinita (IIR) i quali presentano un comportamento migliore in banda attenuata. Il BLIT è ottenuto emulando, ad una determinata frequenza di campionamento, il campionamento di un treno di impulsi filtrati da un filtro analogico IIR. Questo metodo si basa infatti sulla progettazione di un filtro analogico passa-basso. Infine, come per la maggior parte dei metodi basati su BLIT, l'aliasing è ridotto anche in casi di modulazione in frequenza a frequenze audio.

2.2 Filtro ladder del Minimoog

Il filtro ladder del Minimoog è stato creato nel 1965 da Robert Moog e brevettato nel 1969. La sua struttura consiste in 4 filtri passa-basso ad un polo e una retroazione negativa tra l'ultimo filtro ed il primo.

Nel corso degli anni sono stati pubblicati articoli accademici che propongono diversi modelli digitali del filtro. Alcuni di essi cercano di emulare il circuito analogico in dettaglio (tecnica chiamata *white box modelling* o *modellazione fisica*), altri invece emulano solo il comportamento generale e le caratteristiche principali senza considerare la sua struttura interna (tecnica chiamata *black box modelling*)¹.

Di seguito ci sono alcuni articoli scientifici che mostrano vari approcci per la modellazione del filtro.

"The Art of VA filter design" [8] di Vadim Zavalishin è un libro che introduce la progettazione di vari tipi di filtri musicali sia dal punto di vista teorico che pratico. L'obiettivo del libro è quello di progettare modelli digitali delle strutture dei filtri analogici usati nei sintetizzatori usando l'approccio *topology-preserving transform* senza la necessità di studiare il circuito elettronico. Grazie a questo metodo è possibile progettare filtri digitali tempo-varianti non lineari con una risposta in ampiezza e fase il più simile possibile al modello analogico.

L'articolo di Tim Stilson e Julius Smith [9] analizza il filtro ladder del Moog nel dominio tempo-continuo e propone 2 tipi di discretizzazione usando la trasformata bilineare e la backward difference transform.

Inoltre, una sezione è dedicata all'analisi del VCF (Voltage Controlled Filter) usando la tecnica del luogo delle radici (spesso utilizzata per l'analisi dei sistemi di controllo). Maggiori informazioni possono essere trovate nel capitolo 3 della tesi di Timothy Stilson [1].

L'articolo di Antti Huovilainen [10] presenta un'implementazione digitale e non lineare del filtro del Minimoog. Inizialmente il circuito analogico è stato analizzato ed è stata ricavata l'equazione differenziale che lo descrive, successivamente è stata effettuata la discretizzazione ed è stata ricavata l'equazione alle differenze da implementare digitalmente.

L'articolo di Antti Huovilainen e Vesa Välimäki [11] introduce una versione modificata del filtro sopra citato. La nuova struttura riduce il costo computazionale nell'emulazione delle non linearità rispetto la versione precedente.

L'articolo di Stefano D'Angelo e Vesa Valimäki [12] introduce un'altra discretizzazione del circuito analogico del filtro usando la trasformata bilineare e cercando di emulare ogni suo componente. Rispetto l'articolo precedente, questo modello rispecchia meglio le caratteristiche del circuito analogico.

¹Esiste anche un approccio chiamato *grey box modelling* il quale è una via di mezzo tra i due

2.3 Applicazioni commerciali

Tra le varie applicazioni commerciali che emulano il comportamento del Minimoog ne esistono 2 degne di nota:

- Minimoog model D di Moog Music Inc²: è un'applicazione che emula il comportamento del Minimoog sviluppata interamente per dispositivi mobili IOS. Oltre alle caratteristiche del suo antecedente analogico, questa nuova versione è polifonica (permettendo di suonare fino a 4 note contemporaneamente), permette di effettuare dei loop, è dotata di un arpeggiatore e di una serie di effetti di modulazione.
- Monark di Virtual Instrument³: è un sintetizzatore monofonico ispirato al Minimoog sia per l'interfaccia grafica sia per i suoni emulati che cercano di rispecchiare la potenza, la ricchezza e il tono di quelli analogici.

²<https://www.moogmusic.com/products/minimoog-model-d-synthesizer-app>

³<https://www.native-instruments.com/en/products/komplete/synths/monark/>

Capitolo 3

Tecnologie utilizzate

Per lo sviluppo di questo progetto sono state utilizzate le seguenti tecnologie.

3.1 C++

C++¹ è un linguaggio di programmazione general purpose sviluppato in origine da Bjarne Stroustrup nei Bell Labs nel 1983 come evoluzione del linguaggio C inserendo la programmazione orientata agli oggetti.

Esso è un linguaggio compilato, flessibile e soprattutto efficiente, caratteristica fondamentale nell'elaborazione digitale dei segnali e quindi nella programmazione audio.

3.2 JUCE

JUCE² è un framework open source e multi piattaforma utilizzato per lo sviluppo di plugin audio, di GUI, ma anche di applicazioni desktop e mobile in C++. Rispetto ad altri framework, JUCE ha numerose funzionalità audio integrate al suo interno:

- Supporto per dispositivi audio: DirectSound, ASIO, ...
- Lettori per i formati audio più comuni: WAV, AIFF, FLAC...
- Numerose librerie per l'elaborazione digitale dei segnali
- Dei wrapper per poter compilare lo stesso codice in formati plugin audio diversi tra cui: VST&VST3, RTAS, AAX, Audio Units.

¹<https://www.cplusplus.com/>

²<https://juce.com/>

Tra le classi di Juce utilizzate le principali sono:

- `AudioProcessor` ³: è la classe base per i plugin che svolgono elaborazione audio. Essa si occupa dell'interfacciamento con la DAW gestendo il numero di bus di input e di output, lo stato dei parametri del plugin e di altri aspetti.
- `Synthesiser` ⁴: è la classe base che definisce la struttura del sintetizzatore. Essa permette di gestire la generazione del suono e la gestione degli eventi midi che avvengono durante il rendering dell'audio.
Per creare un sintetizzatore bisogna prima istanziare una sottoclasse di `SynthesiserSound` e una di `SynthesiserVoice`. La prima classe è passiva e definisce quale evento midi può attivare un determinato suono e su quale canale. La seconda classe invece definisce le voci che un sintetizzatore può usare per generare un determinato suono. Il sintetizzatore può essere monofonico (con una singola voce) o polifonico (con molteplici voci).
- `ADSR` ⁵: questa classe permette di gestire l'involuppo di ampiezza delle onde generate dagli oscillatori e della frequenza di taglio del filtro.

3.3 Projucer

Projucer è un ambiente di sviluppo usato per creare e gestire i progetti JUCE. Una volta specificati i file e le impostazioni di un progetto, esso si occupa di generare un insieme di file per permettere al codice di essere compilato nativamente su ogni piattaforma.

Al momento JUCE supporta i seguenti ambienti di sviluppo: Visual Studio, Xcode e CodeBlocks. Oltre alla gestione dei file e delle impostazioni, Projucer ha anche un editor per il codice e l'interfaccia grafica.

3.4 Visual Studio

Visual Studio ⁶ è un'ambiente di sviluppo integrato creato da Microsoft. Esso è utilizzato per lo sviluppo di programmi desktop, mobile e applicativi web. Visual Studio ha un editor che supporta Intellisense (auto completamento) e il refactoring del codice. Esso dispone di un debugger, di un profiler, di un ambiente per progettare interfacce grafiche, pagine web, diagrammi di classi e di database.

³<https://docs.juce.com/master/classAudioProcessor.html>

⁴<https://docs.juce.com/master/classSynthesiser.html>

⁵<https://docs.juce.com/master/classADSR.html>

⁶<https://visualstudio.microsoft.com/it/>

VS permette l'uso di plug-ins che espandono le sue funzionalità e l'uso di sistemi come Git o Azure. Esso supporta 36 tipi di linguaggi di programmazione diversi.

3.5 Digital Audio Workstation (DAW)

La DAW è un software usato per la registrazione, l'editing e la produzione di file audio. Essa è stata usata per caricare ed eseguire il plugin e testarlo tramite l'utilizzo di strumenti come l'oscilloscopio e l'analizzatore di spettro.

3.6 MATLAB

MATLAB ⁷ è un linguaggio di programmazione proprietario multiparadigma e un ambiente di calcolo numerico sviluppato da MathWorks. Esso consente manipolazioni di matrici, tracciatura di funzioni e dati, implementazione di algoritmi, creazione di interfacce utente e interfacciamento con programmi scritti in altri linguaggi. MATLAB è stato utilizzato per analizzare lo spettro degli oscillatori e la risposta in frequenza del filtro in modo più preciso e scientifico rispetto quanto fatto con la DAW.

⁷<https://it.mathworks.com/products/matlab.html>

Capitolo 4

Emulazione virtuale del Minimoog

4.1 Schema a blocchi del circuito

Nei sintetizzatori analogici ogni circuito è un modulo auto-contenuto che effettua determinate operazioni.

La figura 1 mostra uno schema a blocchi della struttura circuitale del Minimoog. I blocchi più importanti sono gli oscillatori, il filtro e i generatori di involuppo, i quali sono connessi tra di loro attraverso segnali audio o di controllo.

I segnali audio sono quelli che vengono uditi mentre i segnali di controllo sono creati per modificare il comportamento dei circuiti che generano i segnali audio. Dal momento che i segnali di controllo sono delle tensioni variabili, il sintetizzatore si dice controllato in tensione.

La figura 2 mostra invece uno schema del pannello frontale del Minimoog e la figura 3 mostra il pannello frontale del plugin sviluppato.

Si consideri che le descrizioni future si adattano alla versione del sintetizzatore progettato e non a quella ufficiale, pertanto, l'interfaccia grafica e il design del plugin saranno leggermente diverse rispetto l'originale. Inoltre, alcuni blocchi dello schema circuitale non sono stati implementati (per maggiori informazioni si veda il capitolo 8.2).

4.2 Caratteristiche generali del sintetizzatore

Il Minimoog è un sintetizzatore monofonico, diretto discendente del sintetizzatore modulare Moog. Le sezioni fondamentali sono raggruppate nel pannello frontale e sono le seguenti: Oscillator Bank, Mixer, Modifier, Output, Controllers.

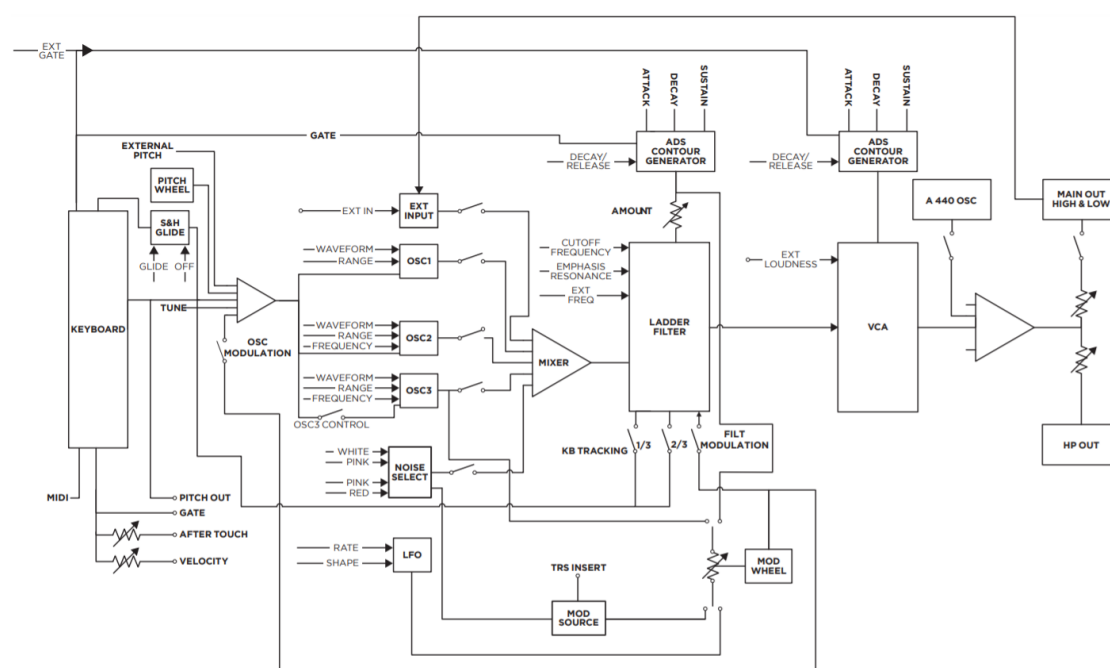


Figura 1: Schema a blocchi del circuito del Minimoog (immagine adattata da [13]).

Ognuno di questi moduli è formato da circuiti elettronici indipendenti controllati da switch e manopole. Il colore di uno switch indica la sua funzione, in particolare:

- Switch arancioni: connettono le sorgenti di modulazione alle loro destinazioni;
- Switch blu: attivano/disattivano le sorgenti audio;
- Switch bianchi: attivano/disattivano glide e release;
- Switch neri: selezionano le sorgenti di modulazione;

4.2.1 Oscillator Bank

Gli oscillatori sono la sorgente del suono principale in un sintetizzatore. Il Minimoog contiene tre oscillatori identici il cui volume viene controllato dal mixer. Questa sezione contiene tre controlli fondamentali mostrati nella figura 4:

- Manopola del *range*: serve per controllare l'ottava fondamentale. La posizione standard è 8', spostando la manopola verso sinistra l'ottava diminuisce mentre verso destra cresce. Nella posizione LO la frequenza raggiunge livelli molto bassi fuori dal range di udibilità dell'orecchio umano.

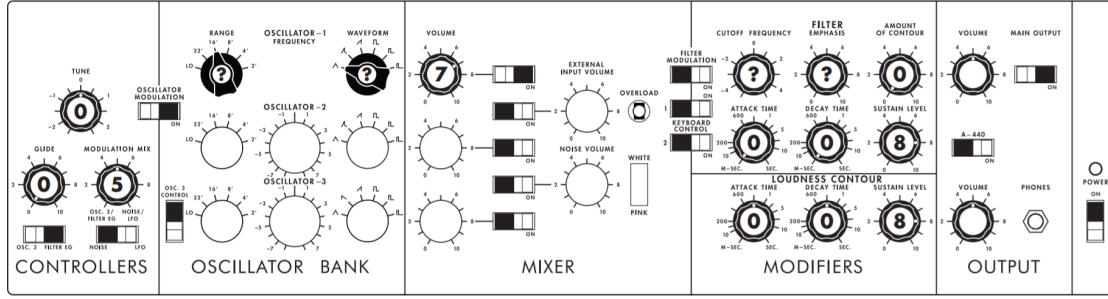


Figura 2: Pannello frontale del Minimoog (immagine adattata da [13]).



Figura 3: Pannello frontale del Minimoog sviluppato.

- Manopola *frequency*: permette di 'scordare' il Minimoog fino ad un massimo di 8 semitoni permettendo di creare un suono ricco e complesso. Questa caratteristica è implementata solo negli oscillatori 1 e 2;
- Manopola *waveform*: permette di cambiare tra 6 forme d'onda diverse, ognuna con un contenuto armonico ed un timbro particolare. Le forme d'onda selezionabili sono: triangolare, triangolare/dente di sega, dente di sega inversa, dente di sega, quadra, rettangolare larga, rettangolare stretta. L'oscillatore 1 e 2 sono identici mentre il terzo genera un'onda dente di sega inversa al posto della triangolare/dente di sega.

Sulla sinistra è presente lo switch *osc.3 control* il quale permette di scollegare l'oscillatore 3 dalla tastiera e dal pitch wheel permettendo di modificare la sua frequenza solo dalla manopola frequency e detune.

Lo switch *Oscillator Modulation* permette di modulare la frequenza degli oscillatori; la fonte di modulazione è scelta dalla sezione Controllers.



Figura 4: Banco oscillatori del Minimoog.

4.2.2 Mixer

Il mixer, mostrato in figura 5, serve per impostare i livelli delle 5 sorgenti audio del Minimoog: i 3 oscillatori, il generatore di rumore e l'input audio esterno. Ogni sorgente può quindi essere livellata dalla manopola del volume e attraverso lo switch può essere mutata.

Rumore

Nel Minimoog sono presenti due tipologie di rumore selezionabili dall'apposito switch:

- *Rumore Bianco*: le sue componenti hanno un'ampiezza costante su tutto lo spettro di frequenze;
- *Rumore Rosa*: le componenti a bassa frequenza hanno ampiezza maggiore rispetto quelle ad alta frequenza;

4.2.3 Modifiers

Questa parte è composta da 3 sezioni separate mostrate in figura 6:

- *Filter*: modifica il contenuto armonico del suono;
- *Filter contour*: fornisce un segnale di controllo che cambia nel tempo agendo sulla frequenza di taglio del filtro;

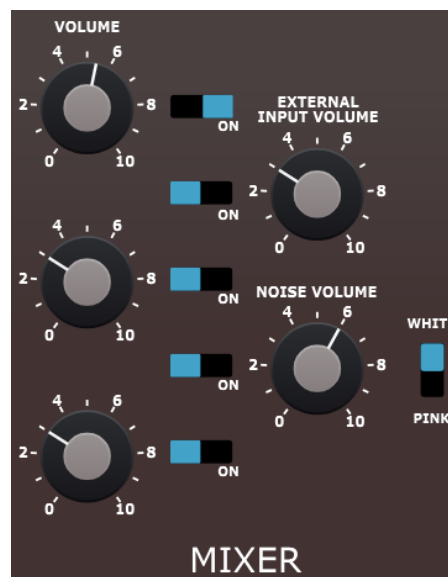


Figura 5: Mixer del Minimoog.

- *Loudness contour*: è un segnale analogo al filter contour che controlla il volume dell'output nel tempo;

Filter

Il Minimoog è equipaggiato con un ladder filter passa-basso con un roll-off di 24 dB/ottava e una risposta in frequenza compresa tra i 10Hz e i 32 kHz.

La manopola di enfasi prende una porzione dell'uscita del filtro e la riporta al suo ingresso creando un picco di *risonanza* che si presenta alla frequenza di taglio. Impostando la manopola di enfasi al massimo e la frequenza di taglio al minimo, il filtro entra in uno stato di auto oscillazione e genera una sinusoide la cui frequenza può essere controllata da tastiera grazie agli switch *keyboard control*.

Keyboard Control (1 & 2) switches

Questi switch permettono alle note suonate da tastiera di modificare la frequenza di taglio del filtro. Questo processo prende il nome di *key tracking* e permette alle note più acute di avere un suono più chiaro e viceversa.

Lo switch *keyboard control 1* abilita 1/3 della quantità totale di key tracking mentre lo switch *keyboard control 2* abilita 2/3 della quantità totale di key tracking. Se sono entrambi attivi si ottiene la quantità di key tracking totale.

Inviluppo del filtro

L'inviluppo del filtro è controllato da 3 manopole: tempo di attacco, tempo di decay, tempo di rilascio.

- Tempo di *attacco*: indica il tempo richiesto al generatore di inviluppo per aumentare la frequenza di taglio del filtro dal suo valore impostato manualmente al valore massimo impostato dalla manopola *amount of contour*. L'inviluppo è attivato ogni volta che un tasto viene premuto.
- Tempo di *decay*: indica il tempo richiesto dal generatore di inviluppo del filtro per abbassare la frequenza di taglio dal livello raggiunto durante la fase di attack al livello definito dalla manopola di sustain. Quando lo switch 'decay' è attivo, il tempo di decay indica il tempo che ci vuole per far tornare la frequenza di cutoff alle impostazioni manuali.
- Livello di *sustain*: indica il valore della frequenza di taglio una volta che la fase di attack e decay sono terminate e fin tanto che il tasto rimane premuto.

Amount of Contour: questa manopola determina quanto del segnale di controllo creato dal generatore di inviluppo influenza la frequenza di taglio del filtro nel tempo.

Loudness Contour

- Tempo di *attacco*: indica il tempo impiegato dal generatore per aumentare il volume da 0 al suo livello massimo una volta che un tasto viene premuto.
- Tempo di *decay*: indica il tempo che impiega il generatore per abbassare il volume dal livello massimo raggiunto nella fase di attack al livello definito dal sustain. Quando lo switch decay è attivo, il tempo di decay indica il tempo richiesto alla nota per dissolversi completamente.
- Livello di *sustain*: una volta che la fase di attack e decay sono terminate il generatore di inviluppo mantiene il livello di volume determinato dal sustain fin tanto che la nota è premuta.

Decay switch: permette di attivare/disattivare la release nell'inviluppo del filtro e del volume.

4.2.4 Output

Questa sezione, come mostra la figura 7, è divisa in:

- *Pitch Wheel*: aumenta o diminuisce la frequenza degli oscillatori fino ad un massimo di 1 ottava permettendo un effetto di *bending*.

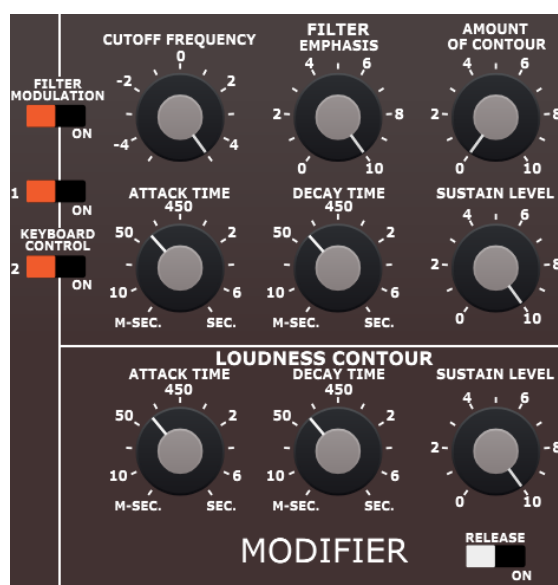


Figura 6: Modifier del Minimoog.

- *Glide*: quando è attivo la frequenza degli oscillatori cambia gradualmente tra una nota e la successiva senza saltare direttamente alla frequenza successiva. Il tempo di transizione è impostato dalla relativa manopola mentre lo switch permette la sua attivazione/disattivazione.
- *Output* principale: questa manopola determina il livello di segnale da mandare in output. È presente anche uno switch per poter mutare completamente il Minimoog.

4.2.5 Controllers

Nel Minimoog model D ci sono diverse sorgenti di modulazione che permettono di modificare la frequenza degli oscillatori e la frequenza di taglio del filtro.

Le modulazioni sono attivate tramite gli switch *Oscillator Modulation* e *Filter Modulation* presenti nei relativi pannelli. Nella sezione controllers, mostrata nella figura 8 ci sono 2 switch neri che permettono di selezionare le sorgenti di modulazione:

- *Switch OSC.3/Filter EG*: permette di selezionare come fonte di modulazione l'oscillatore 3 o il generatore di inviluppo del filtro.
- *Switch Noise/LFO*: permette di selezionare come fonte di modulazione il rumore o l'oscillatore dedicato a bassa frequenza.



Figura 7: Output del Minimoog.

Ci sono anche 2 manopole che gestiscono i vari livelli della modulazione:

- *Modulation Mix* serve per bilanciare le 2 sorgenti selezionate dai 2 switch, in particolare, se la manopola è girata completamente a sinistra la fonte di modulazione sarà quella selezionata dallo switch sinistro e viceversa.
- *Modulation Wheel*: gestisce la quantità di modulazione che influenza la frequenza di taglio del filtro e la frequenza degli oscillatori.

LFO Rate: imposta la frequenza dell'oscillatore a bassa frequenza. Le forme d'onda che genera sono triangolare e quadra selezionabili dal relativo switch.

4.3 Implementazione software

Uno degli step fondamentali in questo progetto è stato quello di definire la struttura del codice, ovvero quante e quali classi implementare e come farle interagire tra di loro in modo da rispecchiare il più possibile i blocchi principali del Minimoog descritti nella sezione precedente.

Grazie all'uso del framework Juce è stato possibile sin da subito impostare il flusso di lavoro e utilizzare delle classi già implementate per la creazione del progetto.



Figura 8: Controllers del Minimoog.

4.3.1 Elenco e breve descrizione delle classi

Le classi implementate nel progetto sono le seguenti:

- MoogOsc: è utilizzata per la gestione dei 3 oscillatori del Minimoog.
- Blit: è utilizzata all'interno della classe MoogOsc. Essa si occupa della generazione delle forme d'onda utilizzando la tecnica di sintesi *Bandlimited Impulse Train*.
- MoogLFO: è utilizzata per generare le forme d'onda degli oscillatori a bassa frequenza.
- MoogNoise: è utilizzata per generare il rumore bianco.
- PinkNoise: è utilizzata all'interno della classe MoogNoise. Essa implementa un *Pinking Filter* ovvero un filtro con un roll-off di 6dB/ottava che permette di ricavare il rumore rosa partendo dal rumore bianco.
- MoogMixer: è usata per gestire i livelli dei segnali provenienti dai 3 oscillatori, dall'input audio esterno e dal rumore.
- MoogFilter: implementa il filtro ladder passa-basso non lineare a 4 poli del Minimoog.

- **Matrix**: implementa l'algoritmo di Newton-Raphson. Esso è utilizzato nella classe `MoogFilter` per risolvere numericamente l'equazione del filtro e calcolare l'uscita.

La figura 9 mostra il diagramma UML delle classi.

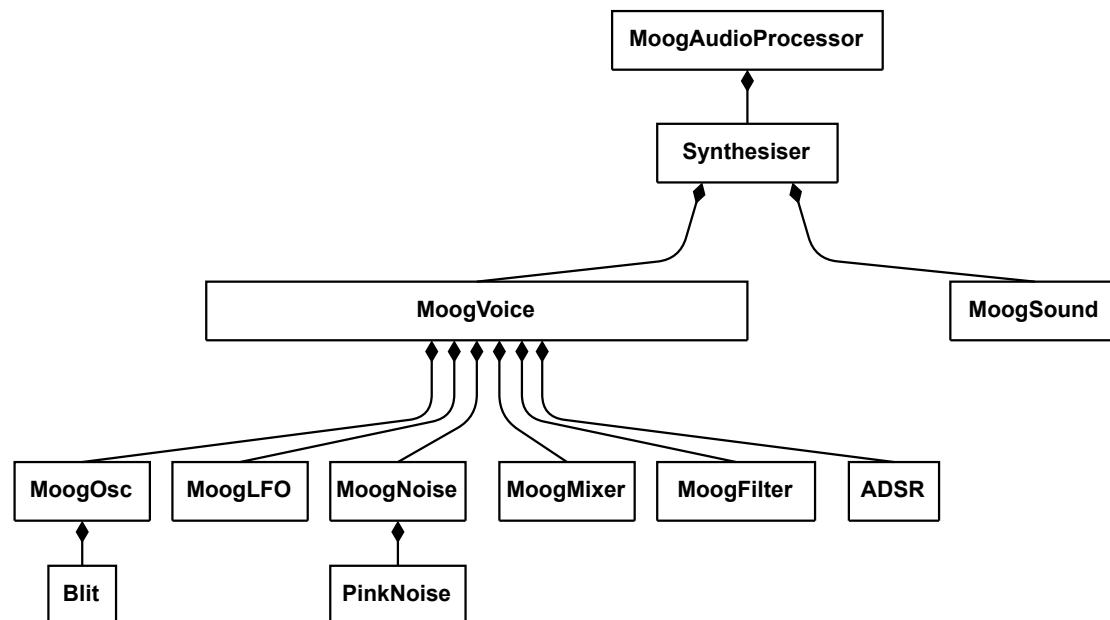


Figura 9: Diagramma UML delle classi

4.3.2 Flusso generale del programma

In questa sezione si vuole descrivere la catena di elaborazione del suono nel programma la quale rispecchia lo schema circuitale in figura 1.

MoogAudioProcessor

In questa classe attraverso il metodo *processBlock()* viene passato un riferimento al buffer in cui salvare i campioni una volta elaborati. Successivamente viene chiamato il metodo *renderNextBlock()* della classe `Synthesiser` nel quale saranno invocati specifici metodi di altre classi per effettuare determinate operazioni sui campioni. Dato che il Minimoog è un sintetizzatore monofonico, ogni classe lavora su un array monodimensionale.

Se i canali di uscita sono 2, non si vanno ad effettuare le stesse operazioni 2 volte per entrambi ma si effettua l'elaborazione per un singolo canale e successivamente si copiano i dati nell'altro prima di restituire il controllo alla DAW.

Codice

```

1 void MoogAudioProcessor::processBlock(juce::AudioBuffer<float>&
  buffer)
2 {
3     const auto numSamples = buffer.getNumSamples();
4     muteExtInput ?
5         0 : buffer.clear();
6     synth.renderNextBlock(buffer, midiMessages, 0, numSamples);
7     totalNumOutputChannels == 2 ?
8         buffer.copyFrom(1, 0, buffer, 0, 0, numSamples) : 0;
9 }

```

Synthesiser

Questa classe funziona da wrapper per tutte le altre classi che effettuano DSP¹. Analizzando il metodo *renderNextBlock()* si può notare la catena dell'elaborazione dell'audio. Per prima cosa viene chiamato il metodo *frequencyModulation()* per gestire i parametri della modulazione in frequenza degli oscillatori e del filtro.

In secondo luogo gli oscillatori *osc1* e *osc2* e il rumore *noise* attraverso il metodo *getNextAudioBlock()* generano i campioni delle loro forme d'onda e li salvano in 3 buffer separati². Successivamente l'oggetto *mixer* tramite il metodo *getNextAudioBlock()* prende i buffer elaborati dalle classi precedenti, livella il valore dei campioni e li copia in un unico buffer da restituire alla DAW.

Il filtro del Minimoog, rappresentato dall'oggetto *moogLpFilter*, elabora il buffer tramite il metodo *processSample()*.

Infine, l'oggetto *adsrLoudness* gestisce l'ADSR del volume, mentre *outputBuffer* gestisce il guadagno finale.

Codice

```

1 void MoogVoice::renderNextBlock(AudioBuffer< float >& outputBuffer
  , ...)
2 {
3     float** bufferData = outputBuffer.getArrayOfWritePointers();
4     frequencyModulation(startSample, numSamples);
5     // Audio Source
6     osc1.getNextAudioBlock(...);
7     osc2.getNextAudioBlock(...);
8     noise.getNextAudioBlock(...);
9     // Mixer
10    mixer.getNextAudioBlock(...);
11 }

```

¹Digital Signal Processing

²Dato che *osc3* è anche una fonte di modulazione, i campioni della sua forma d'onda sono gestiti nel metodo *frequencyModulation()*.

```
12 // Ladder Filter
13 for (int i = startSample; i < startSample + numSamples; i++) {
14     moogLpFilter.setCutoffFrequency(...);
15     bufferData[0][i] = moogLpFilter.processSample();
16 }
17 // VCA
18 adsrLoudness.applyEnvelopeToBuffer(...);
19 // Final Gain
20 outputBuffer.applyGain(...);
21 }
```

Capitolo 5

Bandlimited Impulse Trains

5.1 Introduzione

Uno degli obbiettivi di questo elaborato è quello di implementare digitalmente un algoritmo che cerchi di emulare il suono del banco oscillatori del Minimoog con la minor quantità di aliasing possibile.

Esistono diverse tecniche di sintesi del suono, quella utilizzata in questo caso è chiamata *Bandlimited Impulse Train* che consiste nel generare una sequenza di impulsi limitati in banda ed elaborarli in modi diversi per ottenere la forma d'onda desiderata.

5.2 Rappresentazione dell'impulso nel dominio tempo-discreto

Si consideri un treno di impulsi ideali periodico e bipolare in un dominio tempo-continuo. Integrando ogni impulso è possibile generare un'onda quadra come mostrato in figura 10.

Si consideri lo stesso treno di impulsi ma unipolare; sottraendo un determinato offset (DC) ad ogni impulso e integrando il risultato è possibile ottenere un'onda dente di sega come mostra la figura 11.

Dal momento che non si sta lavorando con sistemi analogici ma bensì digitali, il dominio è tempo-discreto e il segnale viene rappresentato da una sequenza di campioni. Considerando lo stesso treno di impulsi ideali nel dominio tempo-discreto si possono verificare 2 casi:

- L'istante in cui sono presenti i campioni è lo stesso in cui sono presenti gli impulsi. Ciò avviene quando la frequenza del treno di impulsi divide esattamente la frequenza di campionamento.

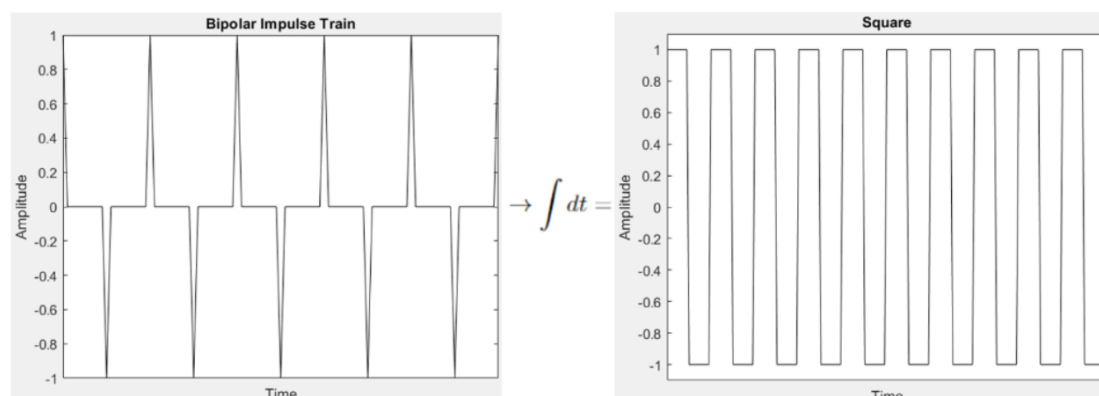


Figura 10: Generazione di un'onda quadra tramite integrazione del treno di impulsi

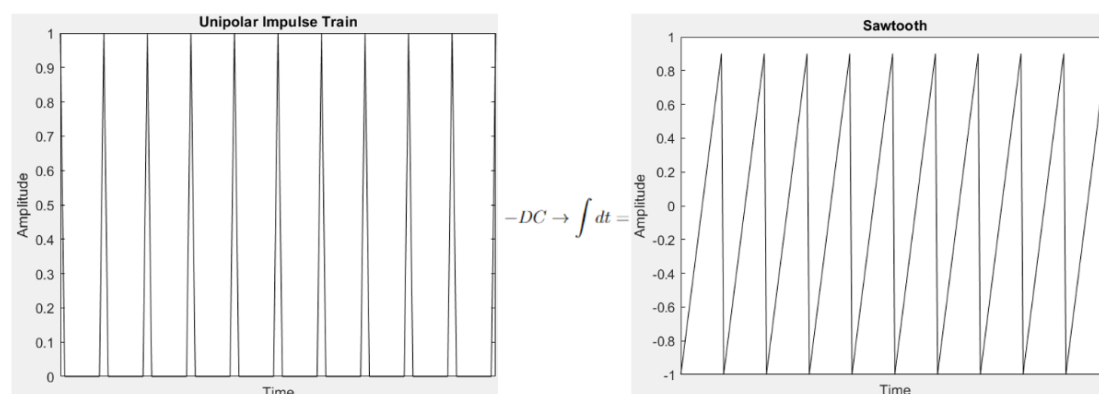


Figura 11: Generazione di un'onda dente di sega tramite integrazione del treno di impulsi

- Gli impulsi sono compresi tra 2 campioni adiacenti. Ciò avviene quando la frequenza del treno di impulsi non divide esattamente la frequenza di campionamento. In questo caso alcuni degli impulsi non verrebbero campionati e il loro valore andrebbe perso.

Entrambe le situazioni sono mostrate in figura 12. Se nel primo caso gli impulsi sono letti correttamente, nel secondo bisogna trovare un modo per poter approssimare il loro valore se non si trovano nella stessa posizione del campione.

La soluzione è quella di fornire in input ad un filtro passa-basso anti-aliasing l'impulso ideale tempo-continuo e ricavare l'uscita. In altre parole, devo sostituire l'impulso ideale con la sua versione limitata in banda.

Un filtro anti-aliasing ideale ha una *risposta in frequenza* rappresentabile da una finestra rettangolare con una frequenza di taglio pari alla metà della frequenza di campionamento (anche chiamata frequenza di Nyquist).

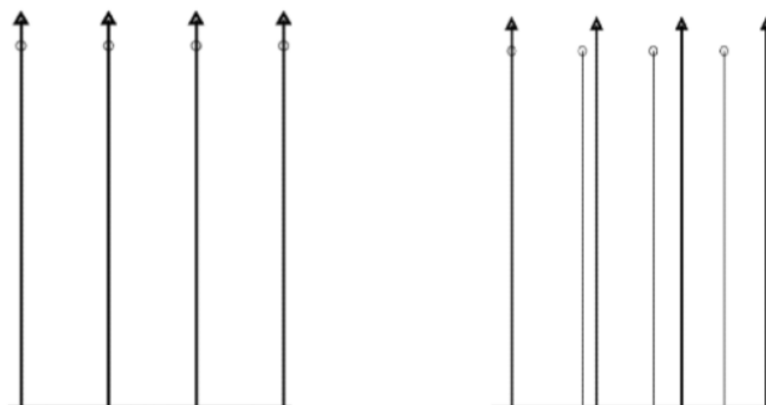


Figura 12: A sinistra: il treno di impulsi viene campionato correttamente. La posizione dei campioni è la stessa degli impulsi. A destra: 1 campione su 4 si trova nella posizione dell'impulso, gli altri 3 sono in posizioni diverse.

L'uscita del filtro può essere ottenuta calcolando la trasformata inversa di Fourier della risposta in frequenza del filtro:

$$h_s(t) = \text{sinc}(F_s t) = \frac{\sin(\pi F_s t)}{\pi F_s t}$$

dove F_s è la frequenza di campionamento e h_s è la *risposta all'impulso*.

Dal risultato ottenuto si può notare che la versione limitata in banda di un impulso in qualsiasi istante di tempo può essere rappresentata da un seno cardinale che si estende idealmente tra $-\infty$ e $+\infty$ nel tempo e che assume valore zero ogni T_s secondi dove $T_s = 1/F_s$. L'uscita del filtro viene poi campionata, quantizzata e i campioni sono salvati digitalmente all'interno di un computer. La figura 13 mostra i passaggi spiegati fino ad ora con un dominio limitato a $[+10, -10]$.

Dal momento che sia la versione continua che quella discreta della risposta all'impulso hanno un dominio infinito, i campioni da salvare dovrebbero essere infiniti. Purtroppo, la memoria di un calcolatore è limitata e quindi l'uscita del filtro viene finestrata e resa locale nel tempo in modo da poterla rappresentare con un numero finito di campioni. Esistono diversi tipi di funzioni finestra, quella utilizzata nel progetto è la finestra di Hamming¹. La figura 14 mostra la risposta all'impulso moltiplicata da tale finestra. In questo caso il segnale non ha più dominio infinito e il numero di campioni da salvare sono finiti.

¹Per maggiori informazioni sui vari tipi di funzioni finestra si veda il seguente link: https://en.wikipedia.org/wiki/Window_function

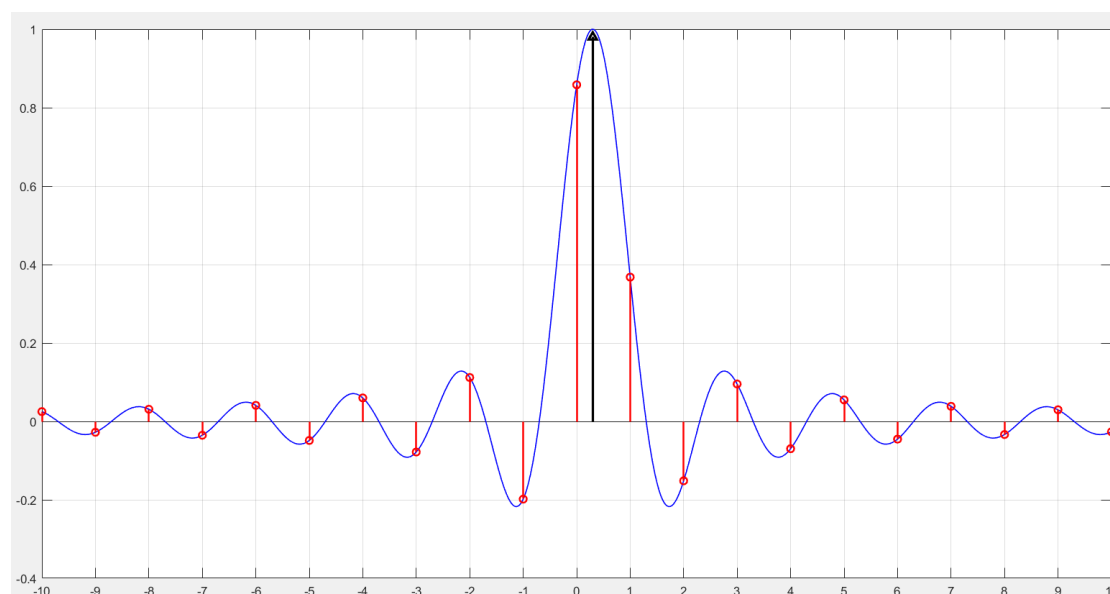


Figura 13: In nero: impulso ideale. In blu: risposta all'impulso di un filtro passa-basso ideale. In rosso: la risposta campionata (F_s normalizzata ad 1).

5.3 Bandlimited Impulse Train

Nel precedente paragrafo si è spiegato come un impulso in qualsiasi istante di tempo viene rappresentato digitalmente in maniera approssimata tramite una sequenza finita di campioni. In questo paragrafo si descrive come creare il treno di impulsi limitati in banda per poi generare le forme d'onda degli oscillatori del Minimoog.

5.3.1 Sum of Windowed Sincs (BLIT-SWS)

Esistono diverse tecniche per la generazione del BLIT, quella usata nel progetto (*Somma di sinc finestrati*) può essere riassunta in 4 passaggi.

Step 1

Calcolare una matrice contenente sinc finestrati, campionati e sfasati di un tempo minore del periodo di campionamento ². Ognuno di essi rappresenta l'approssimazione di un impulso ideale in un determinato istante compreso tra un campione ed il successivo. Maggiore è il numero di sinc pre-calcolati, maggiore sarà la precisione dato che lo sfasamento tra di essi sarà minore. Per esempio, in figura 15 sono

²Questo lavoro viene fatto solo nel momento in cui il plugin è caricato dalla DAW o quando alcune impostazioni vengono cambiate (come la frequenza di campionamento).

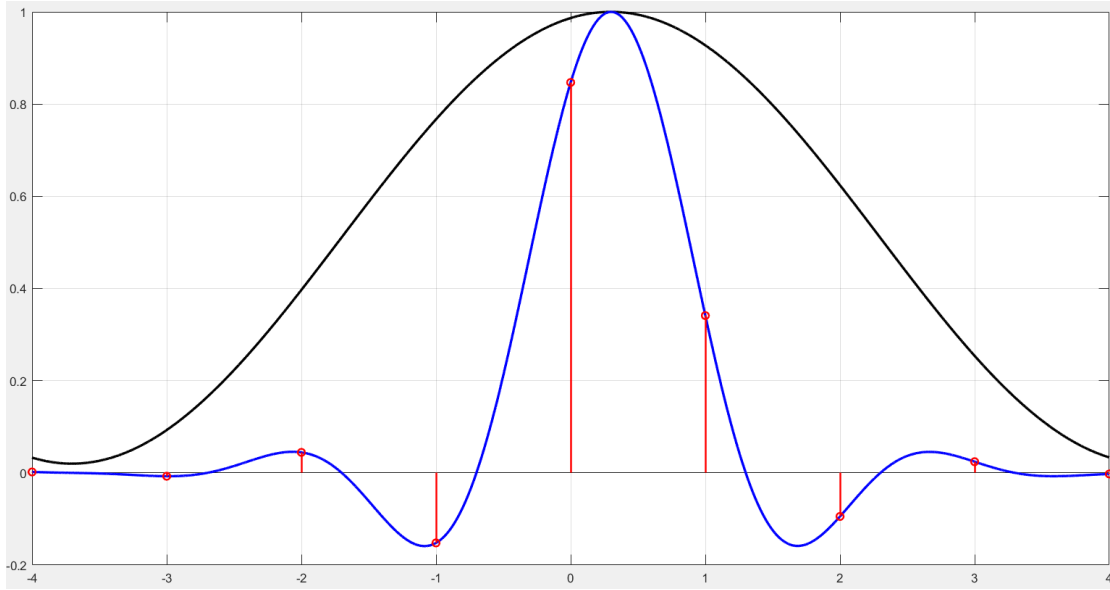


Figura 14: In nero: la finestra di Hamming. In blu: la risposta all'impulso del filtro finestrata. In rosso: la risposta campionata (F_s normalizzata ad 1).

calcolati 5 sinc sfasati di $1/5$ del periodo di campionamento, ognuno dei quali sarà rappresentato da una sequenza finita di campioni contenuta in una determinata riga della matrice. Nel progetto sono stati calcolati 1000 sinc tra un campione e il successivo per ottenere una risoluzione molto elevata; lo sfasamento tra di essi sarà quindi di 1 millesimo del periodo di campionamento. Inoltre, il numero di campioni con cui viene approssimato il sinc è 32 ottenendo quindi una matrice di 1000 righe e 32 colonne³. Questa scelta è dettata dal fatto che il plugin del Minimoog viene eseguito su un computer general purpose e non su architetture con una memoria limitata. Su queste ultime sia il numero di sinc che il numero di campioni che li approssima sono ridotti e molto spesso si ricorre all'uso dell'interpolazione.

Step 2

Calcolare il tempo in campioni che intercorre tra un impulso e il successivo. Esso varia al variare della frequenza del segnale da generare. Si supponga che la frequenza di campionamento $F_s = 48kHz$ e che si voglia generare un BLIT ad una frequenza $f_1 = 523Hz$ (Do5). La durata del periodo in campioni è ottenuta dal

³Ogni sinc è anche stato normalizzato, ovvero, la somma dei 32 campioni di ogni sinc nella matrice dà come risultato 1.

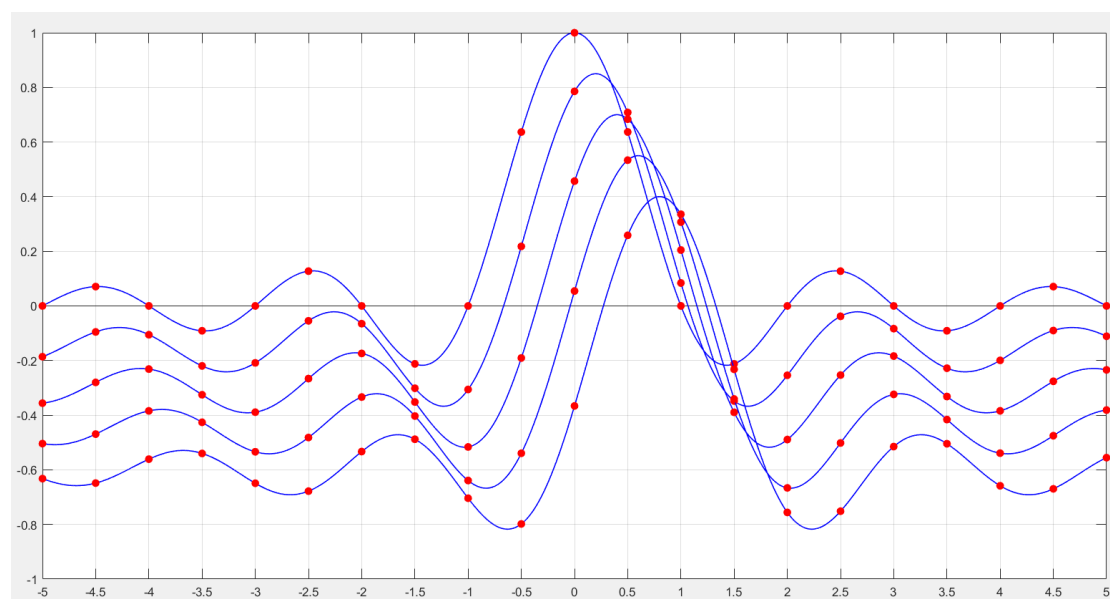


Figura 15: Sinc Visione tridimensionale di 5 sinc sfasati di un tempo corrispondente ad $1/5$ del periodo di campionamento.

rapporto tra la frequenza di campionamento e la frequenza del segnale:

$$nextEdge = \frac{F_s}{f_1} = \frac{48000}{523} = 91.\overline{778}$$

Step 3

Inizializzare un contatore intero che indichi il numero del campione che si sta elaborando all'interno del periodo. All'inizio di ogni nuovo periodo, il contatore viene azzerato e viene selezionato e copiato sul buffer in uscita il sinc nella tabella calcolata nello Step1. Riprendendo l'esempio allo Step2, quando il contatore assume il valore 91 (ovvero $int(nextEdge)$) bisogna prendere la parte decimale del numero (in questo caso 778), selezionare il sinc contenuto nella riga 778 della matrice e copiarlo nel buffer in uscita campione per campione. Ovviamente il numero della riga deve essere proporzionato correttamente in base al numero dei sinc contenuti nella matrice. Se i sinc sono 256, gli indici validi sono da 0 a 255 e il numero 778 deve essere mappato correttamente in quel range. Inoltre, la parte decimale deve essere troncata (come nell'esempio sopra) o approssimata ad un numero di cifre pari al numero di righe della matrice.

Step 4

Man mano che i sinc sono copiati nel buffer in uscita sono effettuate integrazioni e altre operazioni lineari sui campioni per poter ottenere la forma d'onda desiderata (maggiori informazioni nella sezione successiva).

Infine, l'algoritmo può essere schematizzato nel seguente modo:

- Un *produttore* che si occupa di prelevare il sinc corretto dalla matrice e di copiarlo nelle locazioni del buffer in uscita al momento giusto.
- Un *consumatore* che ha il compito di effettuare operazioni lineari sui sinc nel buffer per creare forme d'onda diverse.

Codice

Il seguente codice è un modello generale di una classe che implementa BLIT.

```

1  class Blit
2  {
3      const float blitsTab[1000][32] = {0};    //BlitsTab
4      float blit_array[256] = {0};             //Save current blit
5      double nextEdge = 0;
6      float accumulator = 0;                   //Save current sample
7      float decay = 0.999;                     //Leakyness
8      int sample_cont = 0;
9      unsigned char readIndex = 0;             //Read blit sample
10
11     float updateWave(float frequencySample) {
12         nextEdge = sampleRate / frequencySample;
13         (sampleCont >= int(positiveEdge)) ?
14             (sampleCont = 0, getNextBlit()) : 0;
15         accumulator = accumulator * decay + blit_array[readIndex];
16         sampleCont++;
17         readIndex++;
18         return accumulator;
19     }
20
21     void getNextBlit() {
22         int blitIndex = (nextEdge - int(nextEdge))*1000;
23         const float* blit = blitsTab[blitIndex];
24         unsigned char pos = readIndex;
25         for (int i = 0; i < 32; i++) {
26             blit_array[pos] += blit[i];
27             pos++;
28         }
29     }
30 }
```

Il metodo *updateWave()* si occupa di calcolare il prossimo campione della forma d'onda da generare. Quando ci si trova in prossimità di un impulso del treno, viene invocato il metodo *getNextBlit()* che ha il compito di selezionare dalla matrice dei sinc quello corretto e salvarlo temporaneamente in *blit_array*. I campioni del sinc nell'array sono poi sommati ad ogni ciclo alla variabile *accumulator* che rappresenta il campione dell'onda.

5.4 Generazione delle forme d'onda da BLIT

Una volta prelevato il sinc corretto dalla matrice e salvato temporaneamente in un array, esso deve essere elaborato in modo da poter generare le forme d'onda presenti nel banco oscillatori del Minimoog. Le operazioni che si effettuano sul segnale sono lineari, per questo motivo non sono generate frequenze che già non si trovano nel suo spettro e, dal momento che la nostra sequenza di impulsi discreti è limitata in banda, una volta modificati saranno ancora limitati in banda.

5.4.1 Dente di sega da BLIT

Una funzione dente di sega discreta può essere generata nel seguente modo:

$$Saw(n) = \sum_{k=0}^n (BLIT(k) - C)$$

Dove $BLIT(k)$ rappresenta il treno di impulsi tempo-discreto e C indica il valore medio del treno di impulsi. Come si può notare dalla formula, l'onda può essere ottenuta dalla sommatoria dei campioni del BLIT al quale viene sottratto uno ad uno l'offset C per evitare che l'integrazione vada ad infinito e faccia saturare il Minimoog. Il valore di C dipende dalla frequenza del segnale ed è calcolato nel seguente modo: $C = a/T$, dove a è l'ampiezza dell'impulso (somma dei campioni di un singolo sinc ⁴) mentre T è il periodo in campioni del treno di impulsi.

La figura 16 mostra graficamente la generazione dell'onda (il coefficiente C_2 è negativo), mentre la figura 24 mostra l'onda generata dall'algoritmo.

La funzione discreta dente di sega inversa è l'esatto opposto della precedente e può essere generata nel seguente modo:

$$InvSaw(n) = \sum_{k=0}^n (-BLIT(k) + C)$$

⁴Dato che i sinc nell'array sono normalizzati, la somma dei campioni che rappresentano un singolo sinc equivale a 1

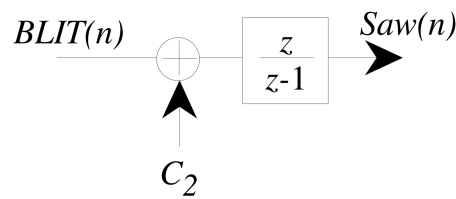


Figura 16: Schema a blocchi della generazione di un'onda dente di sega (immagine adattata da [1]).

Le considerazioni da fare sono analoghe. In questo caso, ovviamente, il BLIT avrà una polarità negativa mentre l'offset sarà sommato (e non sottratto come prima). La figura 17 mostra la trasformazione del treno di impulsi in un'onda dente di sega.

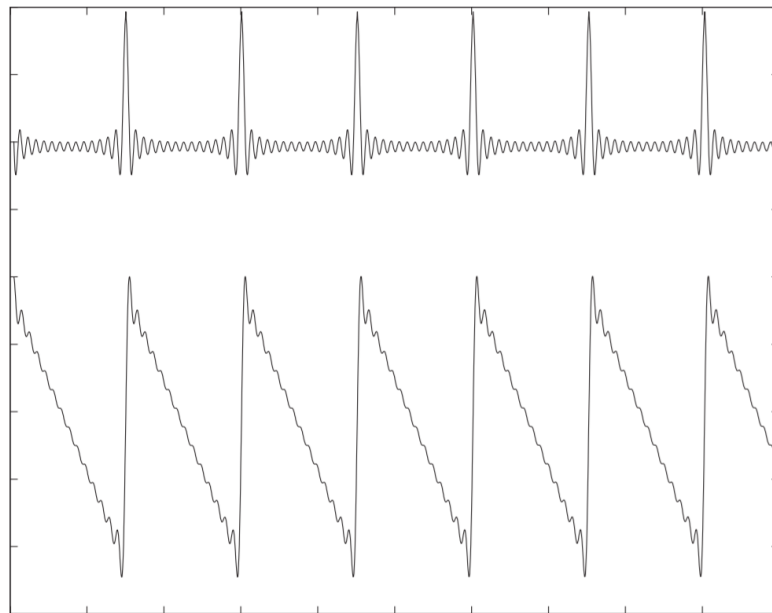


Figura 17: Generazione di un'onda dente di sega inversa partendo dal BLIT (immagine adattata da [1]).

Codice onda dente di sega

```

1 float Blit::updateSawUp(float f) {
2     nEdge = sr / f + subOff2;
3     sampleCont >= int(nEdge) ?
4         (sampleCont=0, getNegativeBlit(), decrementStep=-1.0*f/sr):0;
5     accSaw = accSaw * decay + nBlit[index] - decrementStep;
6     return accSaw;
7 }

```

Codice onda dente di sega inversa

```

1 float Blit::updateSawDown(float f) {
2     pEdge = sr / f + subOff1;
3     sampleCont >= int(pEdge) ?
4         (sampleCont = 0, getPositiveBlit(), decrementStep = f/sr):0;
5     accSaw = accSaw * decay + pBlit[index] - decrementStep;
6     return accSaw;
7 }

```

5.4.2 Onda rettangolare da BP-BLIT (BLIT bipolare)

Un'onda rettangolare tempo-discreta può essere creata nel seguente modo:

$$\begin{aligned}
 Rect(n) &= \sum_{k=0}^n \left(BLIT(k) - BLIT(k - k_0) - C \right) \\
 &= \sum_{k=0}^n \left(BP-BLIT_{k_0}(k) - C \right)
 \end{aligned}$$

dove $BLIT(k)$ rappresenta il treno di impulsi tempo-discreto, $BLIT(k - k_0)$ rappresenta un treno di impulsi sfasato di una costante k_0 , $BP-BLIT_{k_0}(k)$ indica la differenza tra i 2 treni e C indica il valore medio del treno d'impulsi.

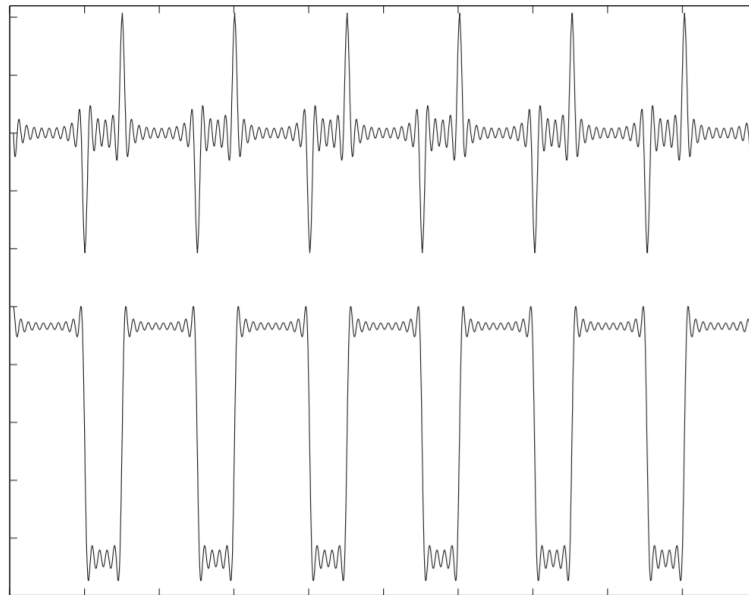


Figura 18: Generazione di un'onda quadra partendo dal BP-BLIT (immagine adattata da [1]).

L'immagine 18 mostra la generazione dell'onda partendo dal BP-BLIT. Come si può notare dall'equazione, un'onda quadra è ottenuta dall'integrazione di un treno di impulsi bipolari (BP-BLIT) opportunamente sfasati tra di loro di una costante k_0 . Essa indica lo spazio in campioni che intercorre tra un impulso positivo e negativo (il suo dominio è quindi $[0, \text{Periodo}]$).

Se k_0 è uguale alla metà del periodo verrà generata un'onda quadra mentre, per altri valori, verrà generato un segnale PWM con un duty-cycle variabile.

Dal momento che un treno di impulsi bipolare ha una componente continua pari a 0 vuol dire che anche $C=0$. Tuttavia, in base alla condizione iniziale dell'integratore, alla variazione nel tempo di k_0 e/o della frequenza del BP-BLIT potrebbero essere generati dei piccoli offset che possono essere attenuati dall'integratore.

Codice onda quadra (duty-cycle 50%)

```
1 float Blit::updateSquare(float f) {
2     pEdge = sr/f + subOff1;
3     nEdge = (pEdge + subOff1) * 0.5f;
4     sampleCont >= int(pEdge) ?
5         (sampleCont = 0, getPositiveBlit()) : 0;
6     sampleCont == int(nEdge) ? getNegativeBlit() : 0;
7     accSquare = accSquare * decay + pBlit[index] + nBlit[index];
8     return accSquare;
9 }
```

Codice onda rettangolare (duty-cycle 65%)

```
1 float Blit::updateWideSquare(float f) {
2     pEdge = sr / f + subOff1;
3     nEdge = pEdge * 0.65 + subOff1*0.35;
4     sampleCont >= int(pEdge) ?
5         (sampleCont = 0, getPositiveBlit(), offset = -0.15f) : 0;
6     sampleCont == int(nEdge) ? getNegativeBlit() : 0;
7     accSquare = accSquare * decay + pBlit[index] + nBlit[index];
8     return accSquare - offset;
9 }
```

Codice onda rettangolare (duty-cycle 80%)

```
1 float Blit::updateNarrowSquare(float f) {
2     pEdge = sr / f + subOff1;
3     nEdge = pEdge * 0.8 + subOff1 * 0.2;
4     sampleCont >= int(pEdge) ?
5         (sampleCont = 0, getPositiveBlit(), offset = -0.30f) : 0;
6     sampleCont == int(nEdge) ? getNegativeBlit() : 0;
7     accSquare = accSquare * decay + pBlit[index] + nBlit[index];
8     return accSquare - offset;
9 }
```

5.4.3 Onda triangolare da rettangolare

Un'onda triangolare tempo-discreta può essere generata direttamente dall'integrazione di un'onda quadra come mostra la figura 19:

$$Tri(n) = \sum_{k=0}^n g(f, d) Rect(k)$$

dove $g(f, d)$ è una funzione che permette di ottenere un'onda triangolare con l'ampiezza appropriata:

$$g(f, d) = \frac{2f}{d(1-d)}$$

dove f indica la frequenza in cicli per campione e d indica il valore del duty-cycle (entrambi i valori hanno dominio $[0,1]$). Dal momento che il duty-cycle dell'onda è del 50% (0.5), sostituendo i valori nell'equazione precedente:

$$g(f, 0.5) = \frac{2f}{0.5(1-0.5)} = \frac{2}{0.25}f = 8f$$

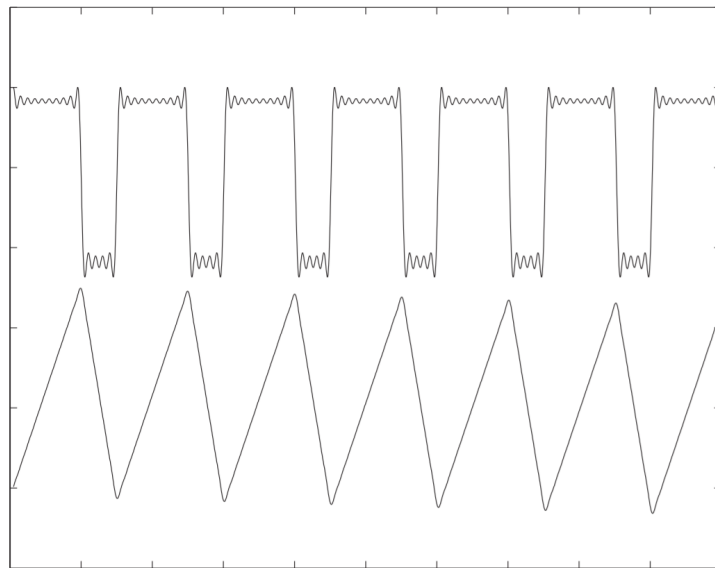


Figura 19: Generazione di un'onda triangolare tramite integrazione di un'onda quadra (immagine adattata da [1]).

Codice

```
1 float Blit::updateTriangle(float f) {
2     accTri = (accTri * decay + updateSquare(f)*8.0*f/sr);
3     return accTri;
4 }
```

Il processo di generazione di un'onda quadra (o rettangolare) e di un'onda triangolare sono ben schematizzati in figura 20.

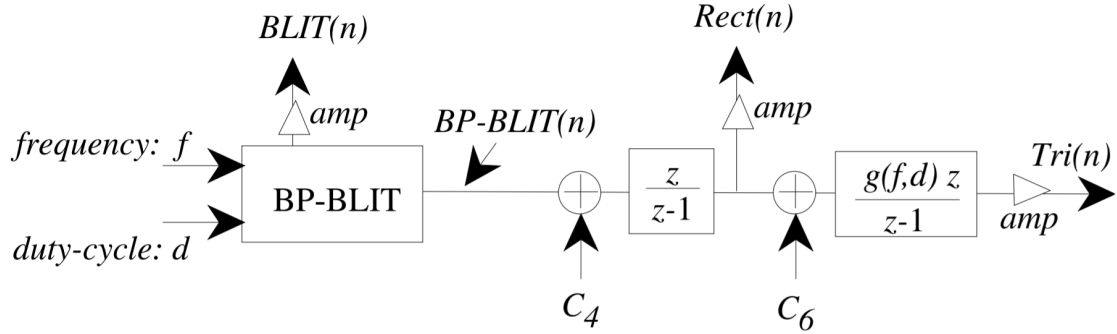


Figura 20: Schema della generazione di un'onda rettangolare (con vari duty-cycle) e triangolare (immagine adattata da [1]).

5.4.4 Onda Triangolare/Dente di sega

Facendo una somma pesata tra l'onda triangolare e l'onda dente di sega è possibile ottenere un'onda triangolare/dente di sega che si trova nel banco oscillatori del Minimoog:

$$TriSaw(n) = \sum_{k=0}^n \left(-0.75 \cdot Tri(k) + 0.25 \cdot Saw(k) \right)$$

Codice

```
1 float Blit::updateSawTri(float f) {
2     float sawSample = updateSawDown(f);
3     float triSample = -updateTriangle(f);
4     return triSample*0.75 + sawSample*0.25;
5 }
```

5.5 Gestione dei DC offset

Dal momento che anche degli offset temporanei possono essere sommati al segnale dall'integratore bisogna cercare di gestirli per evitare che l'onda vada fuori range. Una soluzione che è possibile adottare è quella di utilizzare un *leaky integrator*, ovvero un filtro passa-basso con una risposta all'impulso che decresce esponenzialmente. Grazie ad esso gli offset non continuano a sommarsi all'infinito ma sono

attenuati col tempo. La funzione di trasferimento del filtro è la seguente:

$$H(z) = \frac{1}{1 - 0.999z^{-1}}$$

e questa è la relativa equazione alle differenze:

$$y[n] = 0.999 \cdot y[n - 1] + x[n]$$

In figura 21 viene mostrata la risposta in ampiezza dell'integratore, mentre in figura 22 viene mostrato il diagramma poli-zeri. Sebbene la soluzione presentata

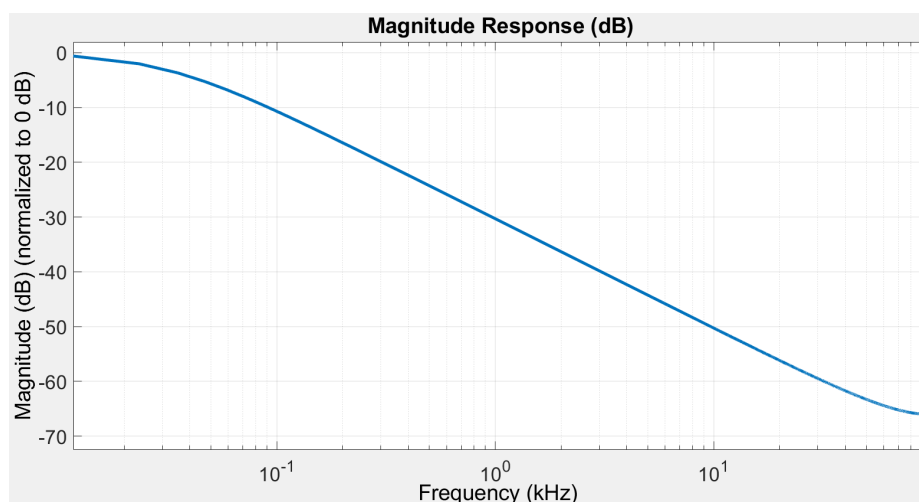


Figura 21: Risposta in ampiezza di un leaky integrator.

funziona bene se la frequenza delle onde non varia velocemente, quando gli oscillatori sono modulati a frequenze sostenute dal LFO o dall'oscillatore 3 un semplice leaky integrator non basta, infatti, l'onda modulata assume valori fuori dal range creando effetti audio sgradevoli. Per questo motivo, quando gli oscillatori sono modulati da quelle 2 sorgenti, il coefficiente del filtro (quello che moltiplica $y[n-1]$) assume valori diversi in base alla frequenza di modulazione. In particolare, il loro rapporto è inversamente proporzionale: maggiore è la frequenza minore sarà il coefficiente e viceversa. Tutto ciò equivale a modificare la posizione del polo (che di default si trova in prossimità della circonferenza unitaria) e di conseguenza anche la risposta in frequenza del filtro.

5.6 Forme d'onda generate dall'algoritmo

Di seguito sono riportate le forme d'onda generate dalla classe Blit del Minimoog implementato. La frequenza del segnale è di 130.81Hz (Do3).

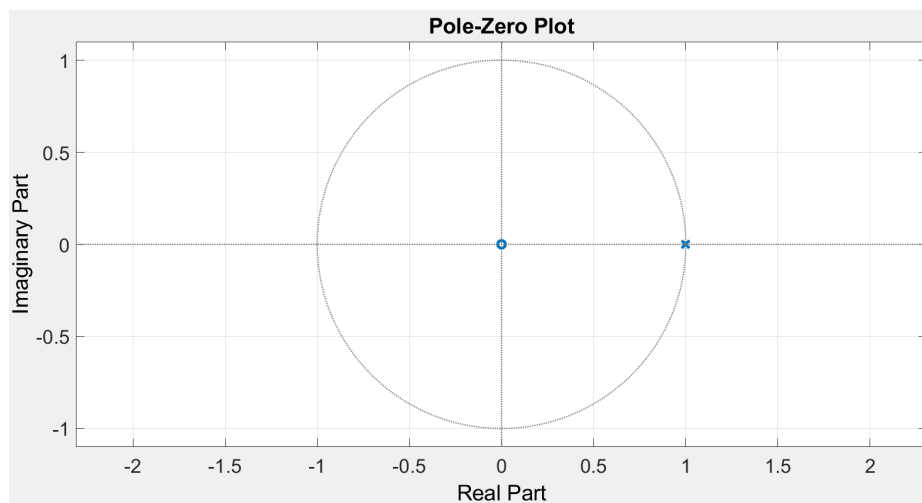


Figura 22: Diagramma poli-zero di un leaky integrator.

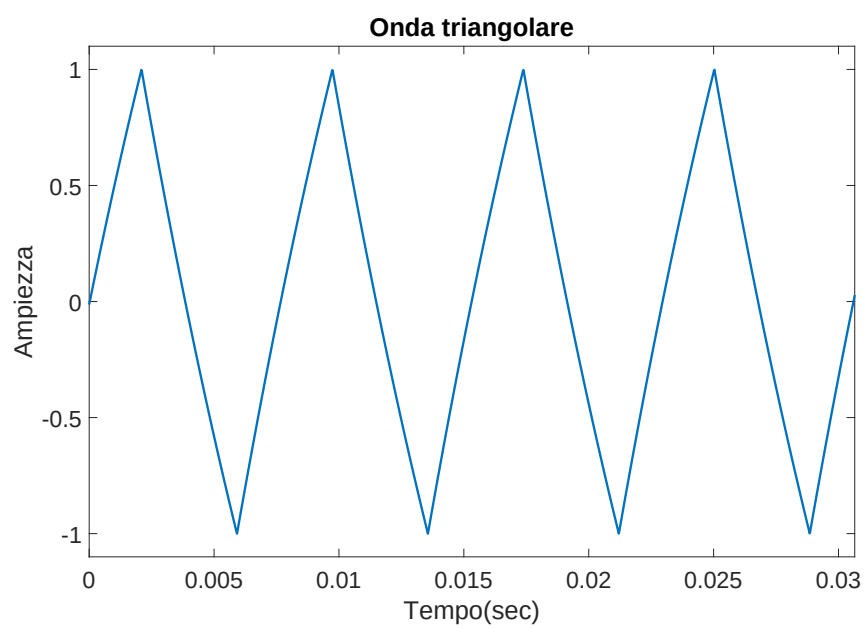


Figura 23: Onda Triangolare.

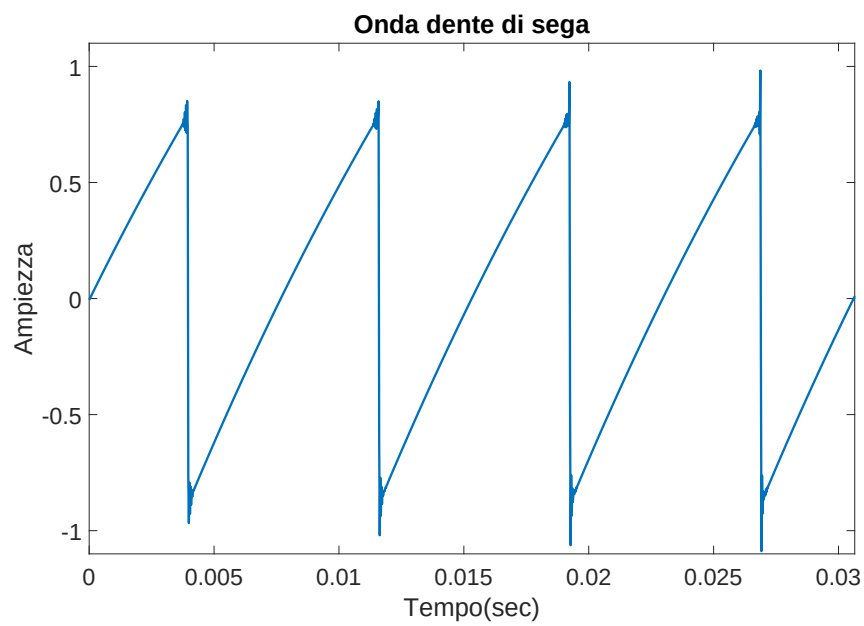


Figura 24: Onda dente di sega.

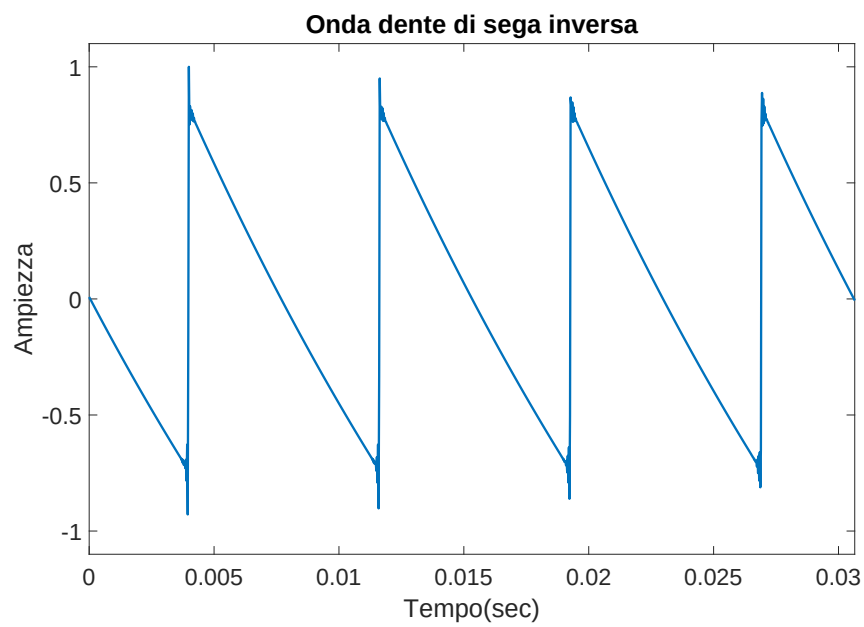


Figura 25: Onda dente di sega inversa.

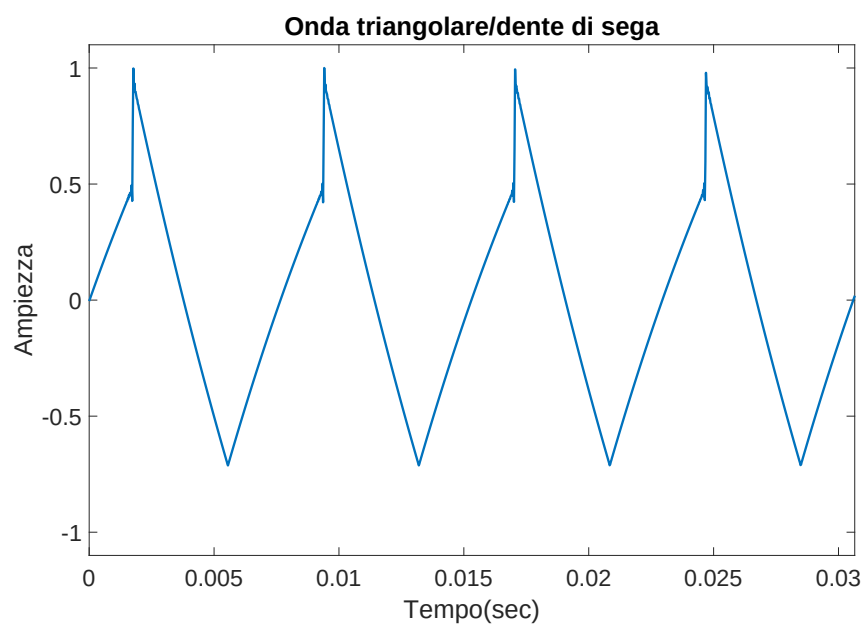


Figura 26: Onda Triangolare/Dente di sega.

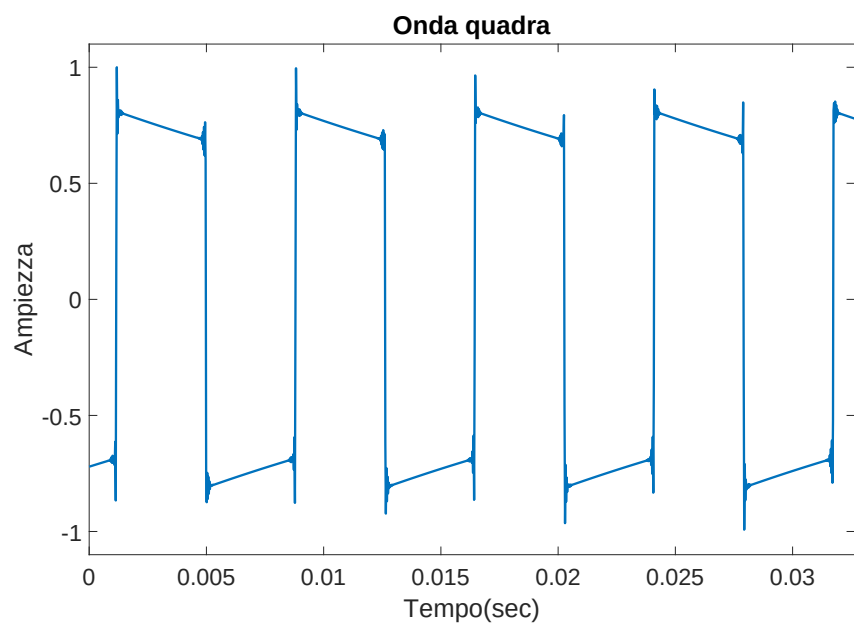


Figura 27: Onda quadra.

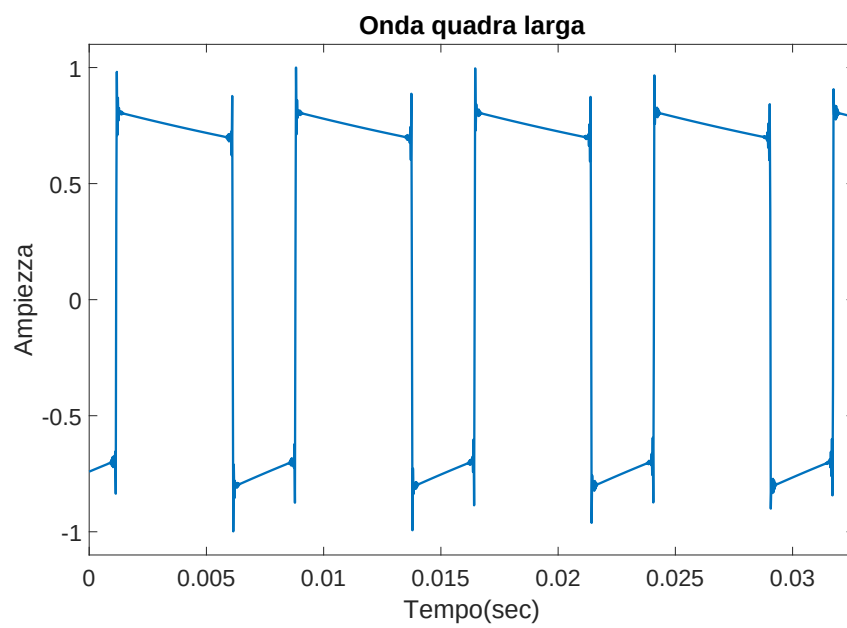


Figura 28: Onda rettangolare (duty-cycle 65%).

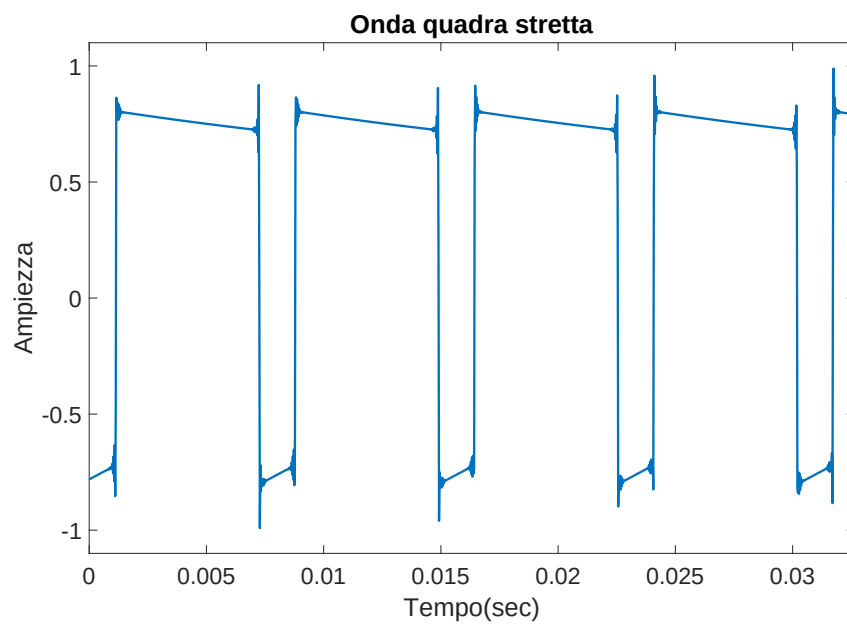


Figura 29: Onda rettangolare (duty-cycle 80%).

Capitolo 6

Il filtro ladder del Minimoog

Il filtro ladder del Minimoog è uno delle componenti principali del sintetizzatore e permette la creazione di suoni ricchi e complessi modificando il contenuto armonico del suono prodotto dagli oscillatori.

Esso è stato brevettato nel 1969 da Robert Moog, il quale, grazie alla recente invenzione dei transistor, riuscì a creare un filtro 'musicale' con una struttura semplice e controllabile in tensione. Dagli anni 60 ad oggi, il Minimoog, grazie anche alla musicalità del suo filtro, è usato in moltissimi generi diversi: rock and roll, progressive rock, R&B, musica elettronica...

Nel corso di questo capitolo verranno analizzati nel dominio tempo-continuo i filtri passa-basso ad 1 polo e il filtro ladder sia in versione lineare che non lineare. Successivamente, le stesse configurazioni saranno analizzate nel dominio tempo-discreto.

6.1 Introduzione ai filtri

Nella teoria dei segnali un filtro è un sistema lineare e tempo-invariante che riceve in ingresso un segnale espresso dalla funzione $x(t)$, lo elabora attraverso una specifica legge di trasformazione $S()$ e fornisce in uscita il segnale risultante $y(t) = S(x(t))$. La sua funzione classica è quella di attenuare una determinata banda di frequenza di un segnale e lasciare la restante invariata. In funzione dell'intervallo di frequenze che il filtro non modifica, esso può essere classificato in:

- Passa-basso: le componenti del segnale al di sotto della frequenza di taglio sono preservate mentre quelle al di sopra sono attenuate.
- Passa-alto: le componenti del segnale al di sotto della frequenza di taglio sono attenuate mentre quelle al di sopra sono preservate.

- Passa-banda: è la combinazione di un filtro passa-alto e passa-basso. Le componenti del segnale che sono comprese tra la frequenza di taglio superiore e quella inferiore saranno preservate mentre le altre saranno attenuate.
- Elimina-banda: è il complementare del filtro passa-banda. Le componenti in frequenza comprese tra quella inferiore e superiore saranno attenuate mentre le altre rimarranno invariate.

6.2 Filtri tempo-continui

I filtri tempo-continui sono dei sistemi che ricevono in ingresso e forniscono in uscita segnali continui nel tempo.

6.2.1 I filtri RC

Si consideri il circuito in figura 30. Esso rappresenta il più semplice filtro passa-basso ad 1 polo dove: $x(t)$ rappresenta il segnale in ingresso, $y(t)$ il segnale in uscita, R il valore della resistenza e C il valore della capacità del condensatore.

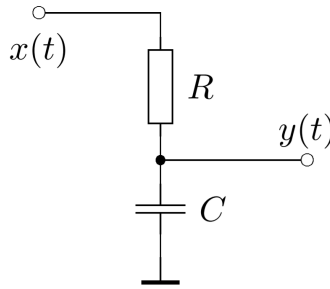


Figura 30: Filtro passa-basso RC ad un polo (immagine adattata da [8]).

L'uscita del circuito può essere espressa come:

$$y = y(t_0) + \int_{t_0}^t \frac{1}{RC} (x(\tau) - y(\tau)) d\tau$$

dove t_0 indica l'istante iniziale.

Introducendo la variabile $\omega_c = \frac{1}{RC}$, la quale rappresenta la *frequenza di taglio* del filtro, l'equazione diventa:

$$y = y(t_0) + \int_{t_0}^t \omega_c (x(\tau) - y(\tau)) d\tau$$

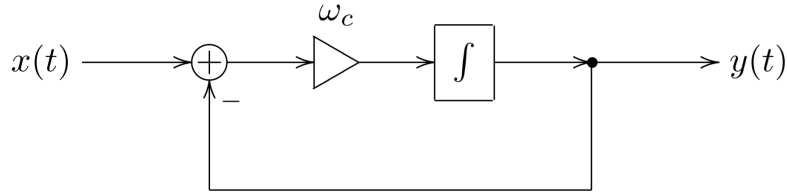


Figura 31: Diagramma a blocchi di un filtro passa-basso RC ad 1 polo (immagine adattata da [8]).

Il risultato ottenuto può essere espresso graficamente dal diagramma a blocchi in figura 31. Al segnale $x(t)$ in ingresso viene sommato il valore di $y(t)$ cambiato di segno e il blocco di gain (rappresentato dal triangolo) moltiplica il segnale in input per un fattore ω_c . Il blocco integratore ha ovviamente il compito di integrare il segnale in input:

$$out(t) = out(t_0) + \int_{t_0}^t in(\tau) d\tau$$

A livello fisico l'equazione indica il comportamento del condensatore che accumula carica man mano che la corrente lo attraversa:

$$q_C(t) = q_C(t_0) + \int_{t_0}^t I(\tau) d\tau$$

dove $q_C(t)$ indica la quantità di carica del condensatore nel tempo, $q_C(t_0)$ indica la carica del condensatore al tempo t_0 e l'integrale indica l'accumulo della corrente che fluisce nel condensatore istante per istante.

In conclusione, il circuito presentato in questo paragrafo ha il compito di scalare il segnale in ingresso di un fattore ω_c e di 'ammorbidirlo' attraverso il blocco dell'integratore.

6.2.2 Funzione di trasferimento

Si consideri la seguente funzione (la quale descrive il blocco dell'integratore in figura 31) e si supponga che $x(t) = e^{st}$ (dove $s = j\omega$):

$$y(t) = y(t_0) + \int_{t_0}^t e^{s\tau} d\tau = y(t_0) + \frac{1}{s} e^{st} \Big|_{\tau=t_0}^t = \frac{1}{s} e^{st} + \left(y(t_0) - \frac{1}{s} e^{st_0} \right)$$

Questa espressione dimostra che, se viene dato in input un esponenziale complesso ad un integratore, in uscita si ottiene lo stesso segnale scalato di un fattore complesso $1/s$ con una componente continua pari a $y(t_0) - e^{st_0}/s$. Supponendo

che l'istante t_0 sia avvenuto molto tempo fa, la componente continua può essere trascurata e l'espressione può essere riscritta nel seguente modo:

$$\int e^{s\tau} d\tau = \frac{1}{s} e^{st}$$

Questo implica che, se il segnale $x(t)$ ha la forma di un esponenziale complesso e^{st} (o di una combinazione di essi), è possibile sostituire l'integratore con un blocco di gain che scala le componenti del segnale di un fattore $1/s$ ($1/j\omega$) come mostrato in figura 32.

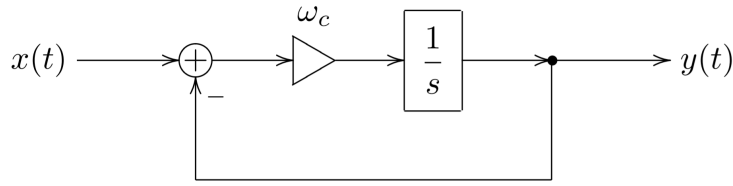


Figura 32: Filtro passa-basso RC ad un polo con l'integratore rappresentato come $1/s$ (immagine adattata da [8]).

Supponendo che $y(t) = Y(s)e^{st}$ e $x(t) = X(s)e^{st}$ l'uscita del filtro può essere espressa come:

$$\begin{aligned} y(t) &= \omega_c \left(\frac{x(t) - y(t)}{s} \right) \\ Y(s)e^{st} &= \omega_c \left(\frac{X(s) - Y(s)}{s} \right) e^{st} \\ Y(s) &= \omega_c \left(\frac{X(s) - Y(s)}{s} \right) \end{aligned}$$

Da cui:

$$\begin{aligned} sY(s) &= \omega_c X(s) - \omega_c Y(s) \\ Y(s) &= \frac{\omega_c}{s + \omega_c} X(s) \\ H(s) &= \frac{Y(s)}{X(s)} = \frac{\omega_c}{s + \omega_c} \end{aligned}$$

Quindi:

$$Y(s) = H(s)X(s)$$

$H(s)$ è definita *funzione di trasferimento*. Essa è una funzione complessa ad argomenti complessi e indica come viene modificato il segnale in uscita $Y(s)$ rispetto

il segnale in ingresso $X(s)$. Il segnale $x(t)$ nel dominio di Laplace è definito come:

$$x(t) = \int_{\rho-j\omega}^{\rho+j\omega} X(s) e^{st} \frac{ds}{2\pi j}$$

Dal momento che il circuito in figura 32 è lineare, l'uscita $y(t)$ può essere ricavata integrando il risultato del prodotto tra la funzione di trasferimento e il segnale in ingresso al filtro:

$$y(t) = \int_{\rho-j\omega}^{\rho+j\omega} H(s) X(s) e^{st} \frac{ds}{2\pi j}$$

Dopo questi passaggi si può arrivare alla conclusione che il circuito in figura 31 modifica in modo indipendente di un fattore $H(s)$ le ampiezze complesse (rappresentate da $X(s)$) dei fasori e^{st} che compongono il segnale. Si ricordi che la funzione di trasferimento è usata per rappresentare sistemi lineari e tempo-invarianti, se queste proprietà non sussistono, tale relazione non può essere usata.

6.2.3 Filtro passa-basso ad 1 polo

Si consideri la funzione di trasferimento ricavata nel paragrafo precedente:

$$H(s) = \frac{\omega_c}{s + \omega_c}$$

La *risposta in ampiezza* mostrata in figura 33 è definita come:

$$|H(j\omega)| = \left| \frac{\omega_c}{\omega_c + j\omega} \right|$$

Dall'espressione si può dedurre che per $\omega = 0$ si ottiene $H(0) = 1$. Mentre, se ω aumenta, la risposta in ampiezza decrementa fino a 0 per $\omega \rightarrow \infty$.

Il comportamento descritto dal grafico 33 è proprio quello di un filtro passa-basso: esso lascia inalterate le componenti del segnale con una frequenza $\omega \ll \omega_c$ e attenua completamente le componenti del segnale con una frequenza $\omega \gg \omega_c$ ¹.

Il range di frequenze dove $|H(j\omega)| \simeq 1$ si chiama *banda passante* del filtro.

Il range di frequenze dove $|H(j\omega)| \simeq 0$ si chiama *banda attenuata*. Il range di frequenze dove $|H(j\omega)|$ cambia approssimativamente tra 1 e 0 si chiama *banda di transizione*. Dal grafico in figura 33 si può notare che se ω raddoppia, la risposta in ampiezza viene dimezzata (diminuisce di 6dB).

La pendenza della curva viene chiamata *roll-off* ed è direttamente proporzionale al numero di poli nella funzione di trasferimento del filtro².

¹Si ricordi che ω_c è la frequenza di taglio del filtro.

²Se si considera $H(s) = N(s)/D(s)$ i poli sono i valori di s per i quali $H(s) \rightarrow \pm\infty$. In particolare, essi corrispondono alle n radici del denominatore $D(s)$.

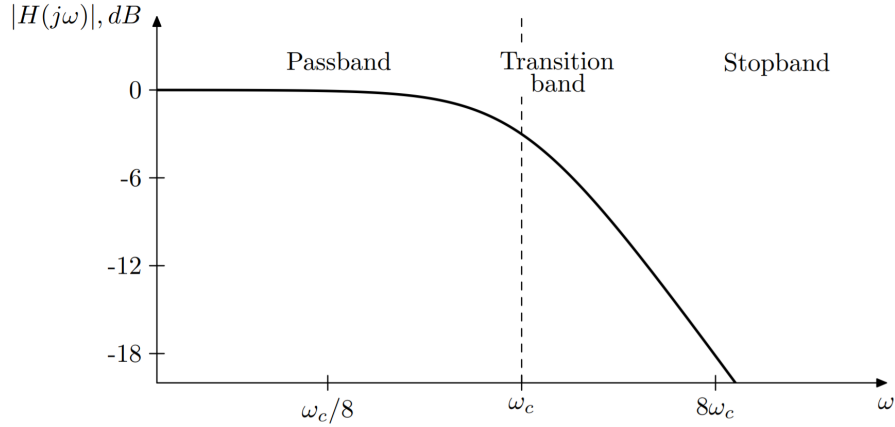


Figura 33: Risposta in ampiezza di un filtro passa-basso ad un polo (immagine adattata da [8]).

In questo caso, il filtro è del primo ordine dato che la funzione di trasferimento ha un singolo polo e il roll-off è pari a 6dB/oct (20 dB/dec)³. Maggiore è l'ordine del filtro (il numero dei suoi poli), maggiore sarà la pendenza della curva e quindi il passaggio tra banda passante e banda attenuata sarà più netto.

Un'altra proprietà di questi filtri è che la risposta in ampiezza viene ridotta di 3dB alla frequenza di taglio. Infatti:

$$|H(j\omega_c)| = \left| \frac{\omega_c}{\omega_c + j\omega_c} \right| = \left| \frac{1}{1 + j} \right| = \frac{1}{\sqrt{2}} = -3dB$$

La *risposta in fase* del filtro passa-basso mostrata in figura 34 è definita come:

$$\arg H(j\omega) = \arg \frac{\omega_c}{\omega_c + j\omega}$$

Essa equivale a 0 per $\omega = 0$, $-\pi/4$ per $\omega = \omega_c$ e $-\pi/2$ per $\omega \rightarrow +\infty$

6.2.4 Filtro ladder lineare del Minimoog

Il modello analogico del filtro è mostrata in figura 35. Esso è formato da 4 filtri passa-basso ad un polo identici tra di loro ed una retroazione negativa tra l'ultimo filtro e il primo.

L'implementazione circuitale classica del filtro ladder viene fatta usando transistor anche se ci sono implementazioni che usano gli amplificatori operazionali.

³L'ordine del filtro può essere stabilito equivalentemente dal grado dell'espressione al denominatore $D(s)$ della funzione di trasferimento $H(s)$ del filtro.

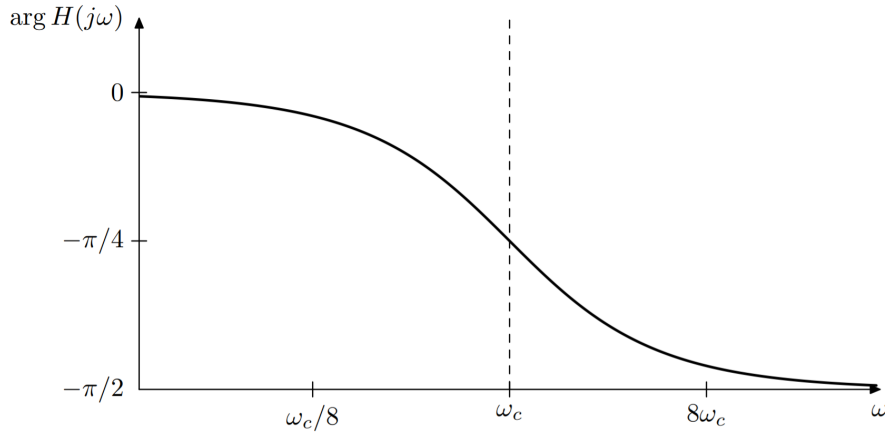


Figura 34: Risposta in fase di un filtro passa-basso RC ad un polo (immagine adattata da [8]).

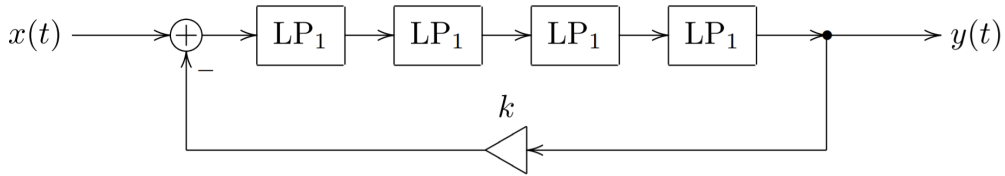


Figura 35: Filtro ladder passa-basso a 4 poli (immagine adattata da [8]).

La differenza tra i due, dal punto di vista della modellazione digitale, sta nel fatto che hanno comportamenti non lineari diversi. Il coefficiente k è un parametro che controlla il livello della retroazione del sistema, ovvero la quantità del segnale in uscita che deve essere sommata al segnale in ingresso. Normalmente il valore di $k \geq 0$ e aumentandolo viene creata una *risonanza* che si trasforma in un'auto oscillazione per $k = 4$. Assumendo $\omega_c = 1$ la funzione di trasferimento di un filtro passa-basso di 1 polo è:

$$H_1(s) = \frac{1}{1+s}$$

La funzione di trasferimento del filtro ladder si ottiene moltiplicando tra di loro le funzioni di trasferimento dei 4 filtri passa-basso che lo compongono considerando anche la retroazione negativa:

$$\begin{aligned} Y(s) &= H_1^4(s) \cdot (X(s) - kY(s)) \\ (1 + kH_1^4(s))Y(s) &= H_1^4(s) \cdot (X(s)) \\ H(s) &= \frac{Y(s)}{X(s)} = \frac{H_1^4(s)}{1 + kH_1^4(s)} = \frac{\frac{1}{(1+s)^4}}{1 + k\frac{1}{(1+s)^4}} = \frac{1}{k + (1+s)^4} \end{aligned}$$

Quando $k = 0$ il filtro ladder equivale a 4 filtri passa-basso ad un polo collegati in serie. I poli del filtro sono:

$$\begin{aligned} k + (1 + s)^4 &= 0 \\ (1 + s)^4 &= -k \\ 1 + s &= (-k)^{\frac{1}{4}} \\ s &= -1 + (-k)^{\frac{1}{4}} \end{aligned}$$

Nel campo C dei numeri complessi elevare un numero ad $1/4$ significa ottenere 4 valori diversi. Infatti, il risultato è formato da 4 radici complesse:

$$\begin{aligned} s_i &= -1 + (-1)^{\frac{1}{4}} \cdot (k)^{\frac{1}{4}} & (k \geq 0), 0 \leq i < 4 \\ s_i &= -1 + (-1)^{\frac{1}{2} \cdot \frac{1}{2}} \cdot (k)^{\frac{1}{4}} \\ s_i &= -1 + (j)^{\frac{1}{2}} \cdot (k)^{\frac{1}{4}} \cdot (k)^{\frac{1}{4}} \\ s_i &= -1 + e^{j \frac{\pi+2\pi i}{2} \cdot \frac{1}{2}} \cdot (k)^{\frac{1}{4}} \\ s_i &= -1 + e^{j \frac{\pi+2\pi i}{4}} \cdot (k)^{\frac{1}{4}} \\ s_i &= -1 + \left(\cos\left(\frac{\pi + 2\pi i}{4}\right) + j \sin\left(\frac{\pi + 2\pi i}{4}\right) \right) \cdot (k)^{\frac{1}{4}} \end{aligned}$$

Le 4 soluzioni complesse rappresentano i 4 poli della funzione:

$$\begin{aligned} s_0 &= -1 + \left(\cos\left(\frac{\pi}{4}\right) + j \sin\left(\frac{\pi}{4}\right) \right) \cdot (k)^{\frac{1}{4}} = -1 + \left(\frac{1}{\sqrt{2}} + \frac{j}{\sqrt{2}} \right) \cdot (k)^{\frac{1}{4}} = \\ &= -1 + \frac{(1 + j)}{\sqrt{2}} \cdot (k)^{\frac{1}{4}} \\ s_1 &= -1 + \left(\cos\left(\frac{\pi + 2\pi}{4}\right) + j \sin\left(\frac{\pi + 2\pi}{4}\right) \right) \cdot (k)^{\frac{1}{4}} = -1 + \left(\cos\left(\frac{3\pi}{4}\right) + j \sin\left(\frac{3\pi}{4}\right) \right) \cdot (k)^{\frac{1}{4}} = \\ &= -1 + \frac{(-1 + j)}{\sqrt{2}} \cdot (k)^{\frac{1}{4}} \\ s_2 &= -1 + \left(\cos\left(\frac{\pi + 4\pi}{4}\right) + j \sin\left(\frac{\pi + 4\pi}{4}\right) \right) \cdot (k)^{\frac{1}{4}} = -1 + \left(\cos\left(\frac{5\pi}{4}\right) + j \sin\left(\frac{5\pi}{4}\right) \right) \cdot (k)^{\frac{1}{4}} = \\ &= -1 + \frac{(-1 - j)}{\sqrt{2}} \cdot (k)^{\frac{1}{4}} \\ s_3 &= -1 + \left(\cos\left(\frac{\pi + 6\pi}{4}\right) + j \sin\left(\frac{\pi + 6\pi}{4}\right) \right) \cdot (k)^{\frac{1}{4}} = -1 + \left(\cos\left(\frac{7\pi}{4}\right) + j \sin\left(\frac{7\pi}{4}\right) \right) \cdot (k)^{\frac{1}{4}} = \\ &= -1 + \frac{(+1 - j)}{\sqrt{2}} \cdot (k)^{\frac{1}{4}} \end{aligned}$$

Per $k = 0$ tutti i poli si trovano ad $s = -1$ e, man mano che il coefficiente k aumenta, essi si muovono lungo 4 linee rette diverse, ognuna delle quali forma un

angolo di 45° con l'asse delle ascisse come mostrato in figura 36.

Inoltre, l'incremento di k da 0 a 4 causa lo spostamento di 2 poli verso l'asse immaginario creando un picco di risonanza in corrispondenza della frequenza di taglio del filtro come mostrato in figura 37.

Per $k = 4$, quei due poli si trovano sull'asse immaginario dato che la loro coordinata reale è 0⁴ e il filtro diventa instabile.

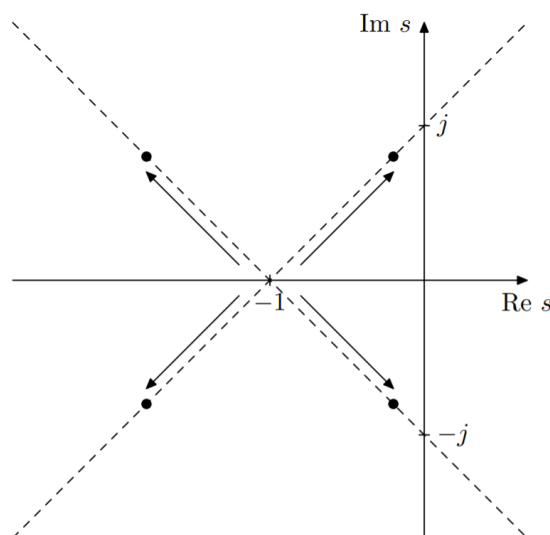


Figura 36: Poli del filtro ladder passa-basso a 4 poli (immagine adattata da [8]).

6.2.5 Filtro ladder non lineare del Minimoog

Le versioni dei filtri discussi fino ad ora sono tutti lineari, ovvero, il circuito del filtro era composto solo da operazioni lineari: addizioni, moltiplicazioni e integrali. Dal momento che i filtri nei sintetizzatori analogici hanno componenti elettronici che presentano delle non linearità, per effettuare un'emulazione corretta si sono aggiunti all'interno del diagramma a blocchi degli elementi non lineari.

I filtri di questo tipo hanno un comportamento più complesso permettendo di creare suoni più ricchi e di incrementare la risonanza a tal punto da far entrare il filtro in uno stato di *auto-oscillazione*.

$${}^4Re\left(-1 + \frac{1+j}{\sqrt{2}}4^{\frac{1}{4}}\right) = 0 \text{ e } Re\left(-1 + \frac{1-j}{\sqrt{2}}4^{\frac{1}{4}}\right) = 0$$

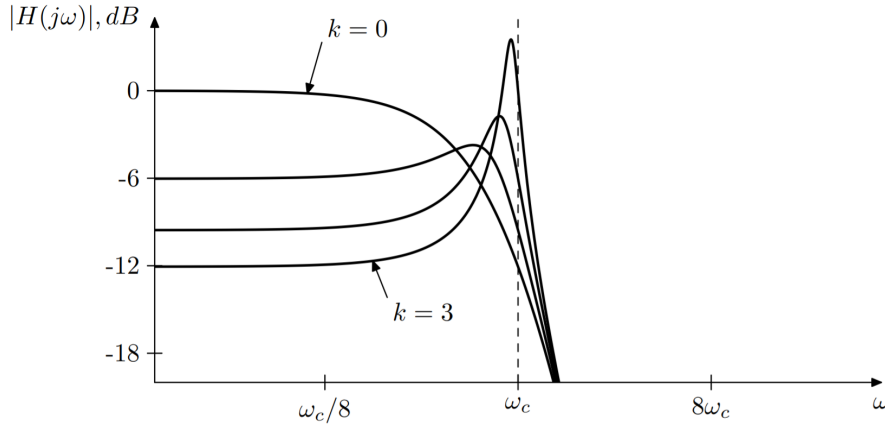


Figura 37: Risposta in ampiezza del filtro ladder passa-basso a 4 poli per diversi valori di k (immagine adattata da [8]).

Waveshaping

Un *waveshaper* è un sistema che applica una funzione non lineare ad un segnale presente al suo ingresso e fornisce il risultato in uscita:

$$y(t) = f(x(t))$$

Normalmente il waveshaper è un sistema tempo-invariante (ovvero i suoi parametri non variano nel tempo) e non ha memoria, questo implica che la sua uscita dipende solo dall'ingresso corrente e non dallo stato passato del sistema.

Saturatori

Sono la categoria di waveshaper più utilizzata. Essi si dividono in: saturatori limitati e saturatori non limitati. Uno dei saturatori limitati più utilizzati è la tangente iperbolica mostrata in figura 38. Il valore dell'output, qualsiasi sia l'input, non eccede mai i valori ± 1 saturando il segnale.

Uno dei saturatori non limitati più utilizzati è il seno iperbolico inverso mostrato in figura 39.

I saturatori nella struttura del filtro

Nel filtro ladder lineare la risonanza è implementata usando un anello di retroazione. Quando il guadagno (il blocco moltiplicatore) della retroazione supera una certa soglia (per esempio $k = 4$ per il filtro ladder del Minimoog) il filtro diventa instabile e l'uscita incrementa idealmente fino ad infinito. Aggiungere un saturatore nello schema a blocchi permette di evitare l'incremento infinito del segnale

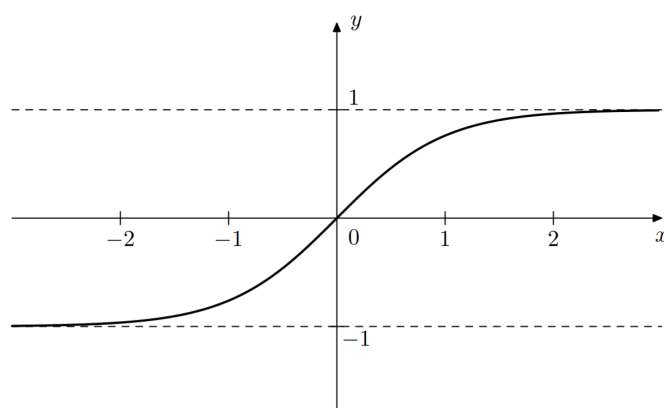


Figura 38: Tangente iperbolica $y = \tanh(x)$ (immagine adattata da [8]).

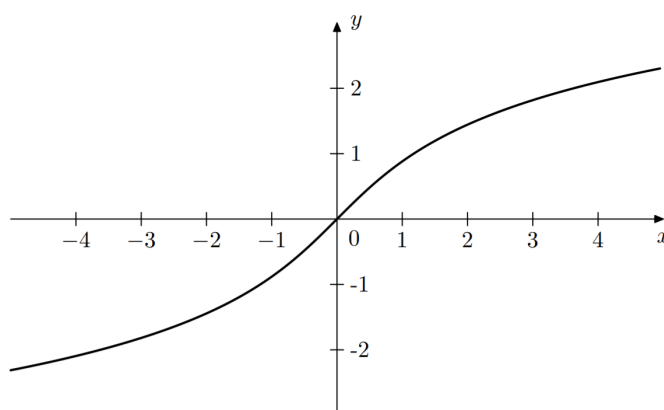


Figura 39: Seno iperbolico inverso $y = \sinh^{-1}x$ (immagine adattata da [8]).

mantenendo il filtro stabile. Una delle posizioni classiche in cui viene inserito è esattamente dopo il blocco del sommatore come mostra la figura 40. Dal momento che si sta usando una tangente iperbolica qualunque sia il valore in ingresso l'uscita sarà limitata e, dal momento che i 4 filtri sono stabili in senso BIBO⁵, anche la loro uscita sarà limitata.

Filtri passa-basso ad 1 polo con OTA

L'implementazione del filtro ladder in questo progetto è stata fatta emulando il comportamento non lineare degli amplificatori operazionali⁶. In questo caso il saturatore è posizionato prima dell'integratore come mostrato in figura 41.

⁵*Bounded Input Bounded Output* è una proprietà dei sistemi la quale enuncia che: a fronte di un segnale in ingresso limitato in ampiezza anche l'uscita prodotta sarà limitata in ampiezza.

⁶*OTA: Operational Transconductance Amplifier* in italiano *Amplificatore Operazionale*

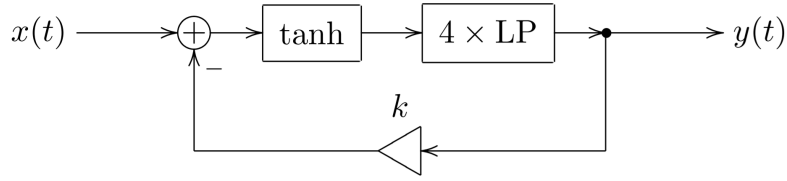


Figura 40: Filtro ladder con un saturatore prima dei filtri (immagine adattata da [8]).

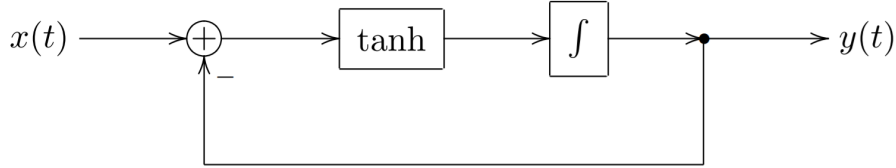


Figura 41: Filtro passa-basso ad un polo non lineare che emula gli OTA (immagine adattata da [8]).

Da questa struttura è possibile derivare la struttura del filtro completo mostrato in figura 42. La rappresentazione matematica di questa struttura e la sua risoluzione saranno viste in seguito. Prima è necessario introdurre le versioni tempo discrete dell'integratore e dei filtri visti in questa sezione.

6.3 Filtri tempo-discreti

I filtri digitali sono dei sistemi che effettuano operazioni matematiche su segnali digitali per modificare il loro contenuto armonico. La differenza rispetto i filtri tempo-continui, i quali elaborano difatti un segnale continuo nel tempo, è che essi elaborano un segnale tempo-discreto che viene prima campionato e quantizzato. Una volta che il segnale è digitalizzato, esso è reso indipendente dal suo periodo di campionamento T_s effettuando una *normalizzazione* e considerando ogni campione ad una distanza unitaria l'uno dall'altro. Per questo motivo, nel corso del capitolo si assume che la frequenza di campionamento $f_s = 1$ e che rispettivamente il periodo di campionamento $T = 1$. La frequenza di Nyquist sarà quindi $1/2$ o, in termini di frequenza angolare, sarà uguale a π .

6.3.1 Integrazione discreta naive

Dal momento che si lavora con architetture digitali è necessario progettare un modello tempo-discreto dell'integratore analogico. Come descritto precedentemente,

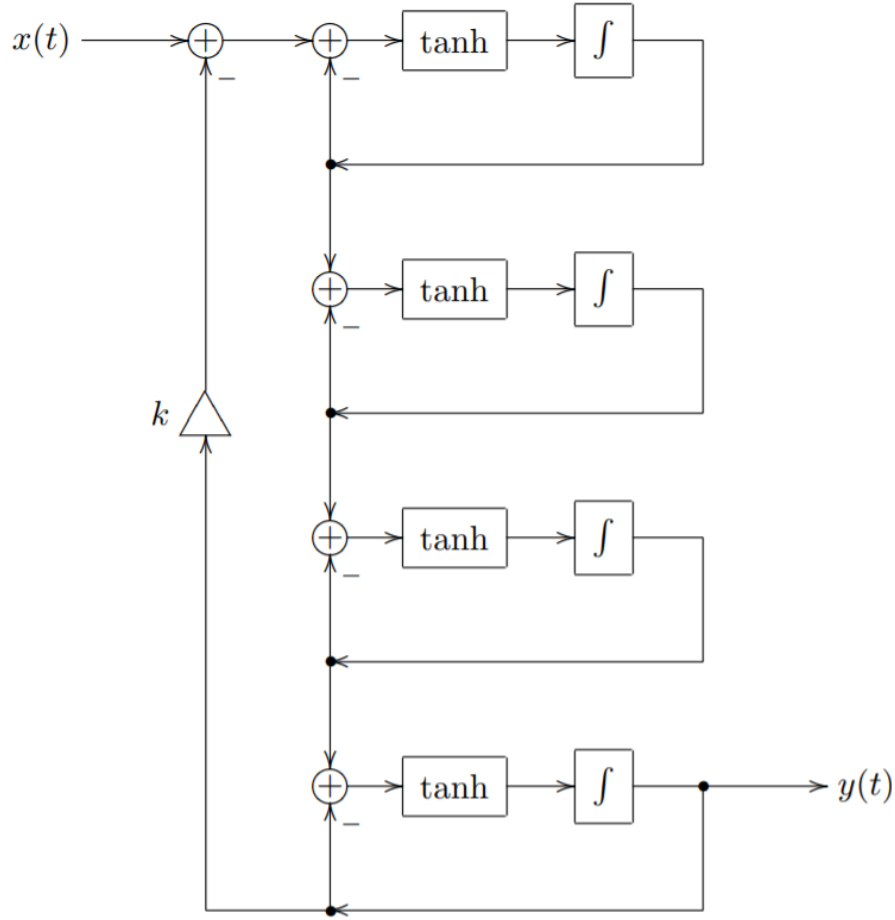


Figura 42: Filtro ladder del Minimoog con emulazione degli OTA (immagine adattata da [8]).

l'equazione di un integratore tempo-continuo è la seguente:

$$y(t) = y(t_0) + \int_{t_0}^t x(\tau) d\tau$$

Nel tempo discreto, l'integrale può essere approssimato da una sommatoria di campioni in input.

Assumendo che $T = 1$, un integratore tempo-discreto *naive* può essere definito come:

$$y[n] = y[n_0 - 1] + \sum_{k=n_0}^n x(k)$$

Se $T \neq 1$ allora i valori in input accumulati dalla sommatoria vanno moltiplicati per il valore di T .

A livello di pseudo-codice si ottiene:

```
1 out = out + in*T
```

Dove la variabile *out* indica lo stato dell'integratore che accumula i campioni, la variabile *in* indica i valori di ogni campione in ingresso all'integratore, mentre la variabile *T* indica il valore del periodo di campionamento (distanza tra un campione e il successivo nel tempo discreto normalizzato).

6.3.2 Filtro passa-basso naïve

Il circuito in figura 31 può essere implementato digitalmente utilizzando l'integratore *naïve* spiegato precedentemente. Assumendo sempre $T = 1$, a livello di codice l'uscita può essere ottenuta nel seguente modo:

```
1 y = y + (x-y)*omega_c
```

Questa implementazione funziona bene fin tanto che $\omega_c \ll 1$ (ω_c deve essere normalizzato) e quindi finchè la frequenza di taglio è molto minore della frequenza di campionamento. Questo perchè a valori elevati di ω_c il filtro diventa instabile.

6.4 Diagramma a blocchi

I diagrammi a blocchi di sistemi tempo-discreti sono simili a quelli tempo-continui, solo che al posto degli integratori si hanno unità che ritardano il segnale di un campione e che servono per mantenere lo stato del filtro. La struttura di un integratore discreto *naïve* è mostrato nella figura 43.

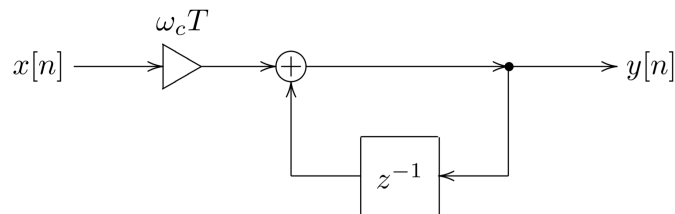


Figura 43: Integratore discreto naïve (immagine adattata da [8]).

Esso è utilizzato per costruire il diagramma a blocchi del filtro passa-basso ad un polo mostrato in figura 44. In figura 45 è mostrato lo stesso filtro ma combinando le due unità di ritardo in una sola. Il blocco z^{-1} nella retroazione del filtro indica che il valore di $y[n]$ viene ritardato di un campione e poi sottratto al valore corrente in ingresso $x[n]$. Questo ritardo non dovrebbe esistere dal momento che nei rispettivi circuiti analogici non è presente nella retroazione. Nei prossimi capitoli verrà discussa una soluzione senza unità di delay che corrisponde all'implementazione di un'equazione *zero-delay feedback*.

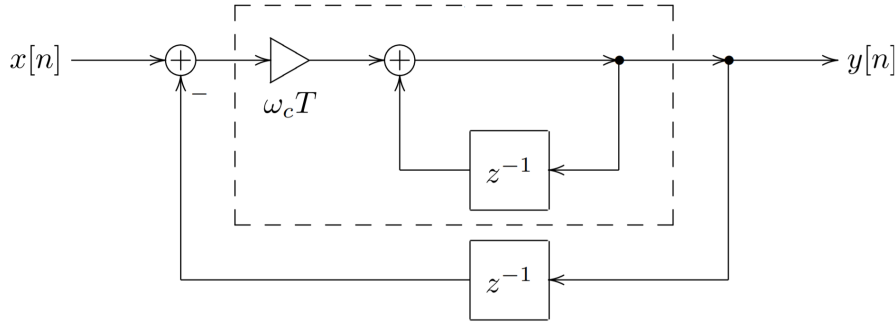


Figura 44: Filtro passa-basso ad 1 polo (immagine adattata da [8]).

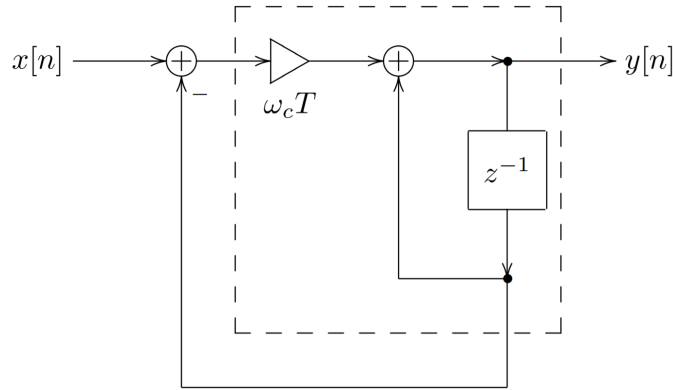


Figura 45: Filtro passa-basso ad 1 polo con una sola unità di delay (immagine adattata da [8]).

6.5 Funzione di trasferimento

Si consideri la figura 45. Supponendo che il periodo sia unitario ($T = 1$), l'equazione alle differenze che descrive l'uscita è:

$$y[n] = y[n-1] + \omega_c(x[n] - y[n-1])$$

La funzione di trasferimento del filtro è ottenuta calcolando la trasformata Z dell'equazione alle differenze:

$$Y(z) = Y(z)z^{-1} + \omega_c(X(z) - Y(z)z^{-1})$$

$$(1 - z^{-1} + \omega_c z^{-1})Y(z) = \omega_c(X(z))$$

$$H(z) = \frac{Y(z)}{X(z)} = \frac{\omega_c}{1 - (1 - \omega_c)z^{-1}}$$

La risposta in frequenza del filtro è ottenuta sostituendo z con $e^{j\omega}$.

La risposta in ampiezza è $|H(e^{j\omega})|$ ed è mostrata in figura 46. La risposta in fase è $\arg H(e^{j\omega})$ ed è mostrata in figura 47.

Come si può notare dai grafici, la risposta in ampiezza ed in fase del filtro discreto devia sempre di più da quella del filtro analogico man mano che la frequenza aumenta. Se la risposta in ampiezza potrebbe essere migliorata correggendo la frequenza di taglio del filtro, la risposta in fase assume sempre valore 0 alla frequenza di Nyquist (quando invece dovrebbe essere circa $-\pi/2$). Se si dovessero sommare due segnali in uscita da due filtri con delle deviazioni nella risposta in ampiezza e in fase, si potrebbero ottenere risultati non desiderati. Questo è il problema principale dei filtri naive e il motivo per cui sono state sviluppate tecniche migliori e più robuste.

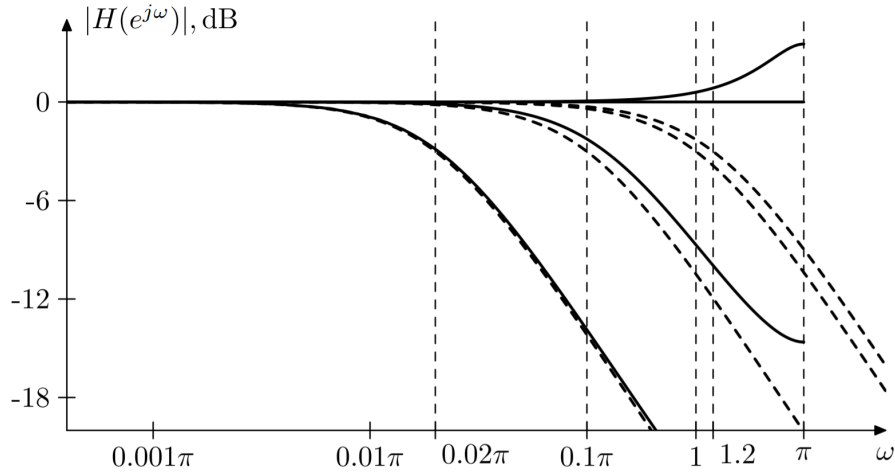


Figura 46: Risposta in ampiezza di un filtro passa-basso naive con diversi valori di cutoff. Le linee continue rappresentano la risposta del filtro discreto, mentre quelle tratteggiate la relativa risposta del filtro continuo (immagine adattata da [8]).

6.6 Integrazione trapezoidale

Assumendo sempre $T = 1$, la formula per l'integrazione trapezoidale è la seguente:

```
1 out = out + (in + prev_in)/2;
2 prev_in = in;
```

dove la variabile *in* rappresenta l'input corrente, la variabile *out* rappresenta l'output corrente e la variabile *prev_in* rappresenta l'input precedente. Il diagramma a blocchi nella figura 48 mostra l'integratore trapezoidale nella sua *I forma diretta*.

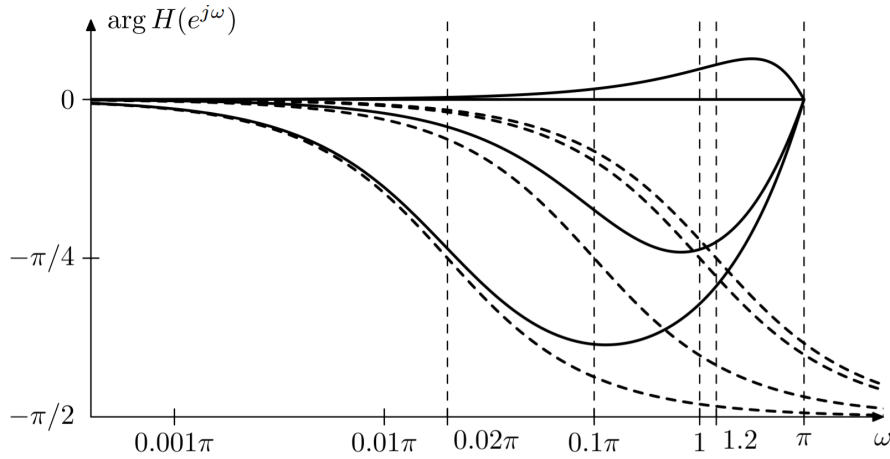


Figura 47: Risposta in fase di un filtro passa-basso naive con diversi valori di cutoff. Le linee continue rappresentano la risposta del filtro discreto, mentre quelle tratteggiate la relativa risposta del filtro continuo (immagine adattata da [8]).

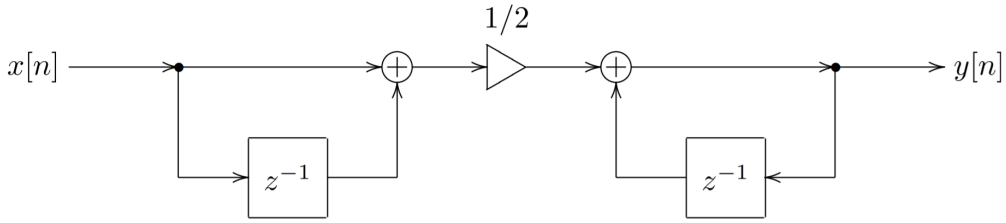


Figura 48: Integratore trapezoidale nella I forma diretta ($T = 1$) (immagine adattata da [8]).

L'equazione per calcolare l'uscita dell'integratore trapezoidale è la seguente:

$$y[n] = y[n_0 - 1] + \sum_{k=n_0}^n \frac{x[k-1] + x[k]}{2} \quad (1)$$

Si supponga che $y[n_0 - 1] = 0$ e $x[n_0 - 1] = 0$, si può riscrivere l'equazione come:

$$\begin{aligned} y[n] &= \sum_{k=n_0}^n \frac{x[k-1] + x[k]}{2} = \frac{1}{2} \left(\sum_{k=n_0}^n x[k-1] + \sum_{k=n_0}^n x[k] \right) = \\ &= \frac{1}{2} \left(\sum_{k=n_0+1}^n x[k-1] + \sum_{k=n_0}^n x[k] \right) = \frac{1}{2} \left(\sum_{k=n_0}^{n-1} x[k] + \sum_{k=n_0}^n x[k] \right) = \end{aligned}$$

$$y[n] = \frac{u[n-1] + u[n]}{2} \quad (2)$$

dove

$$u[n] = \sum_{k=n_0}^n x[k]$$

$x[k]$ è l'input degli integratori.

$u[n]$ è l'uscita dell'integratore

$y[n]$ è la media aritmetica tra l'uscita precedente e corrente dell'integratore.

Il risultato della seconda equazione rappresenta l'integratore nella sua *II forma diretta* anche chiamata *forma canonica* mostrata in figura 49.

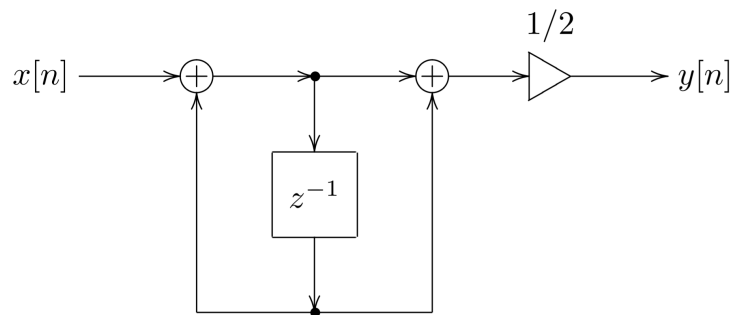


Figura 49: Integratore trapezoidale nella II forma diretta ($T = 1$) (immagine adattata da [8]).

Riscrivendo l'equazione 1 dell'integratore trapezoidale:

$$\begin{aligned}
 y[n] &= y[n_0 - 1] + \frac{1}{2} \left(\sum_{k=n_0}^n x[k-1] + \sum_{k=n_0}^n x[k] \right) = \\
 &= y[n_0 - 1] + \frac{1}{2} \left(x[n_0 - 1] + \sum_{k=n_0+1}^n x[k-1] + x[n] + \sum_{k=n_0}^{n-1} x[k] \right) = \\
 &= y[n_0 - 1] + \frac{x[n_0 - 1]}{2} + \frac{x[n]}{2} + \frac{1}{2} \left(\sum_{k=n_0}^{n-1} x[k] + \sum_{k=n_0}^{n-1} x[k] \right) = \\
 &= y[n_0 - 1] + \frac{x[n_0 - 1]}{2} + \frac{x[n]}{2} + \frac{1}{2} \left(2 \sum_{k=n_0}^{n-1} x[k] \right) = \\
 &= y[n_0 - 1] + \frac{x[n_0 - 1]}{2} + \frac{x[n]}{2} + \sum_{k=n_0}^{n-1} x[k]
 \end{aligned}$$

e riscrivendo l'uscita al campione precedente dell'integratore come:

$$u[n-1] = y[n] - \frac{x[n]}{2} = y[n_0 - 1] + \frac{x[n_0 - 1]}{2} + \left(\sum_{k=n_0}^{n-1} x[k] \right)$$

Si può notare che:

$$y[n] = u[n-1] + \frac{x[n]}{2} \quad (3)$$

Sostituendo $y[n]$ con l'equazione 2 si ottiene:

$$\frac{u[n-1] + u[n]}{2} = u[n-1] + \frac{x[n]}{2}$$

$$u[n-1] + u[n] = 2u[n-1] + x[n]$$

$$u[n] = u[n-1] + x[n]$$

Partendo dall'equazione 3 si può scrivere $u[n-1]$ come:

$$u[n-1] = y[n] - \frac{x[n]}{2}$$

e sostituire il risultato all'interno dell'equazione precedente per ricavare $u[n]$ ⁷:

$$u[n] = u[n-1] + x[n] = y[n] - \frac{x[n]}{2} + x[n] = y[n] + \frac{x[n]}{2}$$

Di seguito i 2 risultati principali:

$$y[n] = u[n-1] + \frac{x[n]}{2}$$

$$u[n] = y[n] + \frac{x[n]}{2}$$

Esprimendo graficamente quanto ottenuto, si ottiene la *II forma diretta trasposta* anche chiamata *forma canonica trasposta* dell'integratore trapezoidale.

Il gain di $1/2$ che precede l'integratore può essere combinato al cutoff gain in un singolo elemento come mostrato nella figura 50.

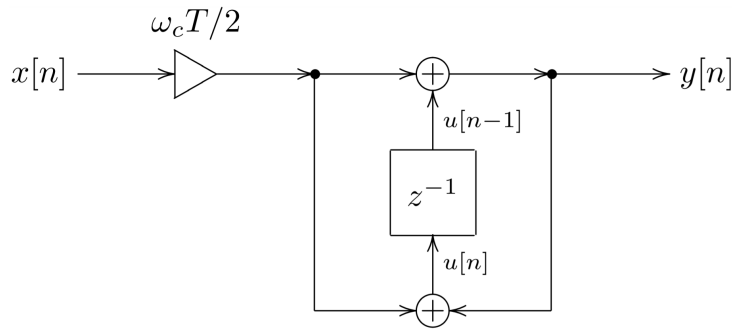


Figura 50: Integratore trapezoidale nella II forma diretta trasposta con il *cutoff gain* integrato (immagine adattata da [8]).

Le funzioni di trasferimento dei 3 integratori trapezoidali sono identiche.

Calcolando la trasformata Z delle equazioni di $y[n]$ e di $u[n]$ sopra descritte:

$$Y(z) = U(z)z^{-1} + \frac{X(z)}{2}$$

$$U(z) = Y(z) + \frac{X(z)}{2}$$

⁷Si noti anche che dalla figura 50, supponendo $T = 1$ e $\omega_c = 1$ il valore di $u[n]$ può essere ottenuto da $u[n] = y[n] + \frac{x[n]}{2}$

e sostituendo la seconda equazione al posto della prima:

$$Y(z) = \left(Y(z) + \frac{X(z)}{2} \right) z^{-1} + \frac{X(z)}{2}$$

$$Y(z)z = Y(z) + \frac{X(z)}{2} + \frac{X(z)}{2}z$$

$$Y(z)(z - 1) = \frac{X(z)}{2}(z + 1)$$

La funzione di trasferimento dell'integratore trapezoidale è:

$$H(z) = \frac{Y(z)}{X(z)} = \frac{1}{2} \cdot \frac{(z + 1)}{(z - 1)}$$

Se si considera l'elemento di gain T e il cutoff gain ω_c si ottiene:

$$H(z) = \frac{\omega_c T}{2} \cdot \frac{(z + 1)}{(z - 1)}$$

6.7 Trasformata Bilineare

Si consideri il diagramma a blocchi tempo-continuo in figura 31.

Sostituire tutti gli integratori tempo-continui con degli integratori trapezoidali tempo-discreti, corrisponde a sostituire tutti i termini s^{-1} nella funzione di trasferimento nel seguente modo:

$$s^{-1} = \frac{T}{2} \cdot \frac{z + 1}{z - 1}$$

La sostituzione può essere scritta come:

$$s = \frac{2}{T} \cdot \frac{z - 1}{z + 1}$$

Essa è chiamata *trasformata bilineare* (BLT). Per questo motivo gli integratori trapezoidali si chiamano anche *integratori BLT*.

Si consideri $H_a(s)$ la funzione di trasferimento del sistema tempo-continuo e $H_d(z)$ la rispettiva funzione di trasferimento tempo-discreta. Il loro legame è rappresentato dalla seguente equazione:

$$H_d(z) = H_a\left(\frac{2}{T} \cdot \frac{z - 1}{z + 1}\right)$$

La risposta in frequenza tempo discreta equivale a:

$$\begin{aligned}
 H_d(e^{j\omega T}) &= H_a\left(\frac{2}{T} \cdot \frac{e^{j\omega T} - 1}{e^{j\omega T} + 1}\right) = H_a\left(\frac{2}{T} \cdot \frac{e^{j\omega T/2}(e^{j\omega T/2} - e^{-j\omega T/2})}{e^{j\omega T/2}(e^{j\omega T/2} + e^{-j\omega T/2})}\right) = \\
 &= H_a\left(\frac{2}{T} \cdot \frac{e^{j\omega T/2} - e^{-j\omega T/2}}{e^{j\omega T/2} + e^{-j\omega T/2}}\right) = H_a\left(\frac{2}{T} \cdot j \frac{\frac{e^{j\omega T/2} - e^{-j\omega T/2}}{2j}}{\frac{e^{j\omega T/2} + e^{-j\omega T/2}}{2}}\right) = \\
 &= H_a\left(\frac{2}{T} \cdot j \frac{\sin(j\omega T/2)}{\cos(j\omega T/2)}\right) = H_a\left(\frac{2}{T} j \tan \frac{\omega T}{2}\right)
 \end{aligned}$$

Si noti che $H_a(s)$ è valutata sull'asse immaginario. Questo vuol dire che la trasformata bilineare mappa l'asse immaginario del piano s nella circonferenza unitaria del piano z . La risposta in frequenza digitale ad ω equivale alla risposta in frequenza analogica valutata a $\frac{2}{T} \tan(\frac{\omega T}{2})$. Quindi, la risposta in frequenza analogica compresa nel range $0 \leq \omega < +\infty$ è mappato nel range di frequenza digitale $0 \leq \omega T_s < +\pi$ ($0 \leq \omega < +\pi f_s$). Indicando la frequenza analogica come ω_a e la frequenza digitale come ω_d , la relazione tra le due frequenze può essere espressa nel seguente modo:

$$\omega_a = \frac{2}{T} \tan \frac{\omega_d T}{2}$$

$$\frac{\omega_a T}{2} = \tan \frac{\omega_d T}{2}$$

Quello che fa la trasformata bilineare è mappare il range di frequenze da 0 a $+\infty$ al range da 0 alla frequenza di Nyquist senza cambiare la risposta in frequenza.

Si può ricavare la *trasformata bilineare inversa* partendo da:

$$s = \frac{2}{T} \cdot \frac{z-1}{z+1}$$

$$(z+1) \frac{sT}{2} = z-1$$

$$\frac{sT}{2} + z \frac{sT}{2} + 1 = z$$

$$z \left(1 - \frac{sT}{2} \right) = 1 + \frac{sT}{2}$$

$$z = \frac{1 + \frac{sT}{2}}{1 - \frac{sT}{2}}$$

Da questo risultato si può notare che: se $s = 0$ allora $z = 1$, se $s = \infty$ allora $z = -1$. Si può anche notare che se $Re(s) < 0$ (si sta quindi considerando il semipiano negativo nel dominio delle s), allora: $\left| 1 + \frac{sT}{2} \right| < \left| 1 - \frac{sT}{2} \right|$ e quindi $|z| < 1$. Questo dimostra che il semipiano sinistro è mappato all'interno della circonferenza unitaria nel piano delle z . Infatti, la trasformata bilineare mappa il semipiano negativo nel dominio delle s nella regione interna alla circonferenza unitaria nel dominio delle z , l'asse immaginario è mappato sulla circonferenza unitaria (solo una volta), mentre il semipiano positivo (destro) è mappato nella regione esterna alla circonferenza. In conclusione, se si sostituisce l'integratore *naive* con un integratore trapezoidale nella struttura di un qualsiasi filtro, questa sostituzione è chiamata *topology-preserving transform (TPT)*. Con questo termine si indica che il filtro digitale non solo va ad implementare la funzione di trasferimento del corrispettivo analogico ma emula esplicitamente il comportamento dell'integrazione analogica preservando la topologia (ovvero i collegamenti circuitali) della struttura analogica originale.

6.8 Cutoff Prewarping

Si supponga di utilizzare un filtro passa-basso come quello mostrato in figura 54 in cui si vuole una frequenza di taglio pari a ω_c . Se si dovesse mettere il valore del parametro ω_c nel rispettivo elemento di gain $\omega_c T/2$, la risposta in frequenza e il suo valore alla frequenza di taglio sarebbero diverse da quelle attese.

La figura 51 mostra che il valore della risposta in ampiezza alla frequenza di taglio non è di -3dB per i filtri digitali (come invece dovrebbe essere) e che man mano il valore di frequenza di taglio aumenta, aumenta anche l'errore. Questa differenza tra la risposta analogica e digitale è dovuta alla deformazione dell'asse delle frequenze.

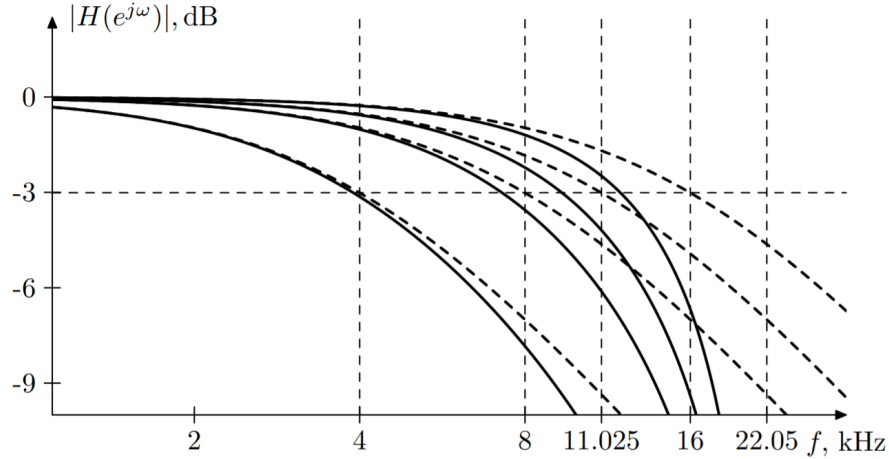


Figura 51: Risposta in ampiezza per varie frequenze di taglio di un filtro passa-basso TPT ad un polo senza prewarping. Le linee continue rappresentano la risposta del filtro digitale mentre quelle tratteggiate rappresentano la risposta del filtro analogico per le stesse frequenze di taglio $F_s = 44100 Hz$ (immagine adattata da [8]).

Per far fronte a questo problema si deve fare un *cutoff prewarping*, ovvero una correzione della frequenza di taglio in modo che il punto di *cutoff* del filtro sia posizionato esattamente alla frequenza specifica ω_c .

Questo implica che il valore di ω_c nel gain del filtro non sarà quello reale ma sarà un valore diverso rappresentabile come:

$$\tilde{\omega}_c = f(\omega_c) = \frac{2}{T} \tan \frac{\omega_c T}{2}$$

Sapendo che il gain prima del blocco dell'integratore in figura 54 è pari a $\omega_c T/2$, sostituendo ω_c con $\tilde{\omega}_c$ si può ottenere:

$$\tilde{\omega}_c \cdot \frac{T}{2} = \frac{2}{T} \tan \frac{\omega_c T}{2} \cdot \frac{T}{2} = \tan \frac{\omega_c T}{2}$$

Applicando questa trasformazione nel blocco del gain del filtro, l'errore della risposta in ampiezza viene ridistribuito. Se in figura 51, in assenza di prewarping, l'errore è zero per $\omega = 0$ e cresce al crescere di ω , con il prewarping l'errore è zero per $\omega = \omega_c$ e aumenta allontanandosi da questo punto come mostra la figura 52.

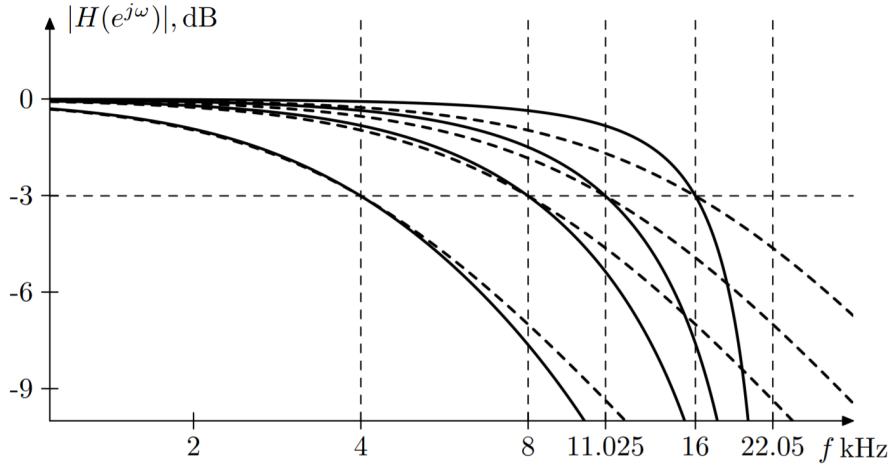


Figura 52: Risposta in ampiezza per varie frequenze di taglio di un filtro passa-basso TPT ad un polo con prewarping. Le linee continue rappresentano la risposta del filtro digitale mentre quelle tratteggiate rappresentano la risposta del filtro analogico per le stesse frequenze di taglio $F_s = 44100 Hz$ (immagine adattata da [8]).

6.9 Zero-Delay Feedback

Sostituire gli integratori naive nella struttura di un filtro discreto con gli integratori trapezoidali introduce il problema che il ramo del feedback non contiene alcun elemento di ritardo. Si prenda in considerazione una semplice struttura di un filtro passa-basso mostrato in figura 53. All'inizio della computazione di un nuovo campione i segnali A e B sono conosciuti. $A = x[n]$ è il campione correntemente in input mentre B è ottenuto dalla memoria dell'elemento z^{-1} (essa indica lo stato del filtro). Può essere trovato il valore di $C = A - B$ e $D = (\omega_c T)C$ e infine $E = D + B$. Il valore di E è poi salvato nella memoria interna del filtro e inviato in uscita come nuovo $y[n]$.

Questo approccio risolutivo non funziona per la struttura in figura 54 dato che c'è una retroazione senza l'unità di ritardo e quindi non si può trovare un punto di inizio per la computazione.

Una possibile soluzione è quella di introdurre un blocco z^{-1} (una unità di delay) all'interno della retroazione. In questo modo non ci sono più feedback senza delay ma la risposta in frequenza del filtro cambia, così come la funzione di trasferimento. Quindi, l'unica possibile soluzione è quella di risolvere l'equazione *zero-delay feedback* del filtro in figura 54. Con questo termine si intende evitare di introdurre un ritardo di un campione nella retroazione.

Si consideri l'integratore trapezoidale in *II forma canonica trasposta* in figura 50.

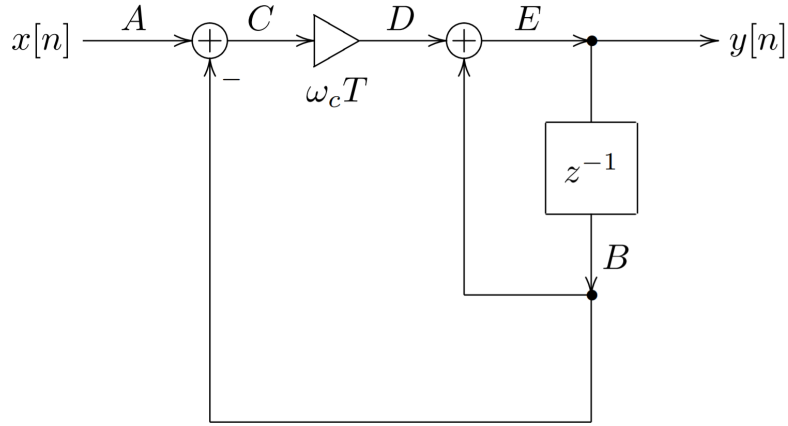


Figura 53: Filtro passa-basso naive ad 1 polo (immagine adattata da [8]).

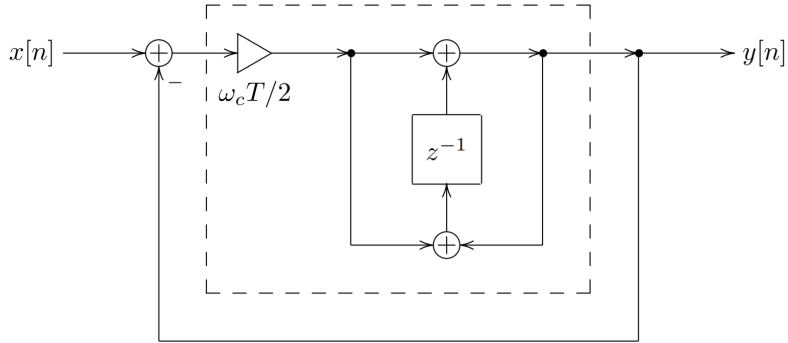


Figura 54: Filtro TPT passa-basso ad 1 polo (immagine adattata da [8]).

Dato che all'interno della struttura non ci sono retroazioni senza unità di ritardo, si può ottenere la seguente espressione per $y[n]$:

$$y[n] = \frac{\omega_c T}{2} x[n] + u[n-1]$$

Nel momento in cui $x[n]$ arriva all'input dell'integratore, tutti i valori del secondo membro dell'equazione sono conosciuti. Introducendo le seguenti notazioni:

$$g = \frac{\omega_c T}{2}$$

$$s[n] = u[n-1]$$

L'uscita dell'integratore può essere riscritta come:

$$y[n] = gx[n] + s[n]$$

$$y = gx + s$$

Questo risultato è fondamentale dato che ad ogni istante n , l'uscita dell'integratore y è una funzione lineare del suo input x e dai parametri g ed s che sono noti. La funzione $f(x) = g(x) + s$ è chiamata *risposta istantanea* dell'integratore all'istante n . Il coefficiente g è chiamato *guadagno della risposta istantanea* e il termine s è chiamato *offset della risposta istantanea*.

Si consideri la struttura del filtro in figura 55.

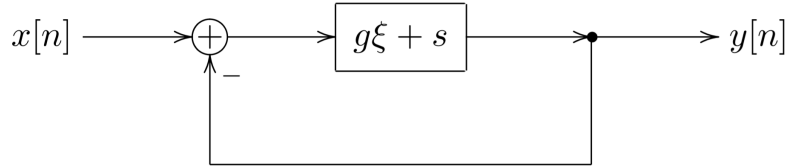


Figura 55: Filtro TPT passa-basso con l'integratore nella sua risposta istantanea (immagine adattata da [8]).

Il termine ε sostituisce il termine x nell'espressione $gx + s$ per indicare l'input dell'integratore mentre $x[n]$ indica il segnale di input all'intero filtro.

L'equazione zero-delay feedback da risolvere è la seguente:

$$\begin{aligned} y[n] &= g(x[n] - y[n]) + s \\ y &= g(x - y) + s \end{aligned}$$

Essa è un'equazione risolvibile in pochi passaggi:

$$\begin{aligned} y &= gx - gy + s \\ y + gy &= gx + s \\ (1 + g)y &= gx + s \\ y &= \frac{gx + s}{1 + g} \end{aligned}$$

L'equazione ottenuta può essere usata per ottenere la risposta istantanea dell'intero filtro in figura 54:

$$y = \frac{g}{1 + g}x + \frac{s}{1 + g}$$

Introducendo le seguenti notazioni:

$$\begin{aligned} G &= \frac{g}{1 + g} \\ S &= \frac{s}{1 + g} \end{aligned}$$

L'equazione diventa:

$$y = Gx + S$$

Il risultato ottenuto sarà utilizzato per risolvere l'equazione zero-delay feedback di filtri con un maggior numero di poli e quindi di ordine superiore al primo.

6.10 Implementazione del filtro

Nelle sezioni precedenti è stata fatta un'introduzione generale dei filtri TPT e dei vari problemi da affrontare durante la progettazione del filtro. In questa sezione sono invece descritte 3 implementazioni, dalla più semplice alla più complessa:

- Filtro passa-basso ad un polo lineare
- Filtro ladder passa-basso a 4 poli lineare
- Filtro ladder passa-basso a 4 poli non lineare

6.10.1 Filtro passa-basso ad un polo lineare

Consideriamo il filtro nella figura 56. Esso è equivalente a quello in figura 54 con l'aggiunta di variabili di supporto.

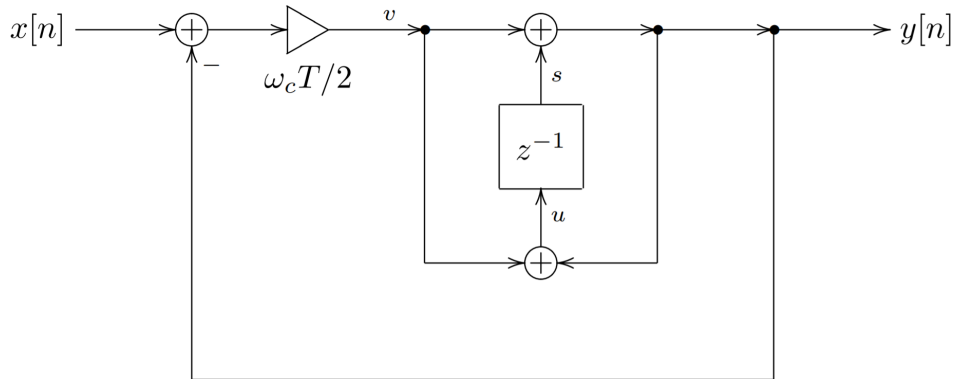


Figura 56: Filtro TPT passa-basso (immagine adattata da [8]).

Ricordando che $g = \omega_c T / 2$, il valore di v può essere ottenuto nel seguente modo:

$$\begin{aligned} v &= g(x - y) = g(x - Gx - S) = g\left(x - \frac{g}{1+g}x - \frac{s}{1+g}\right) = \\ &= g\left(\frac{1+g}{1+g}x - \frac{g}{1+g}x - \frac{s}{1+g}\right) = g\left(\frac{x}{1+g} - \frac{s}{1+g}\right) = \\ &= g\frac{x - s}{1+g} \end{aligned}$$

L'uscita del filtro y è data da:

$$y = v + s$$

L'input del blocco z^{-1} è ottenibile come:

$$u = y + v$$

Le precedenti espressioni possono essere implementate nel seguente modo:

```

1 // Il codice elabora un campione per volta
2 // G = g/(1+g)
3 // La variabile 's' rappresenta lo stato dell'integratore (z^-1)
4
5 v = (x - s) * G;
6 y = v + s;
7 s = y + v;

```

La figura 57 mostra un diagramma a blocchi che rappresenta il codice. In questo caso non si hanno più retroazioni senza unità di ritardo risolvendo così il problema della zero-delay feedback.

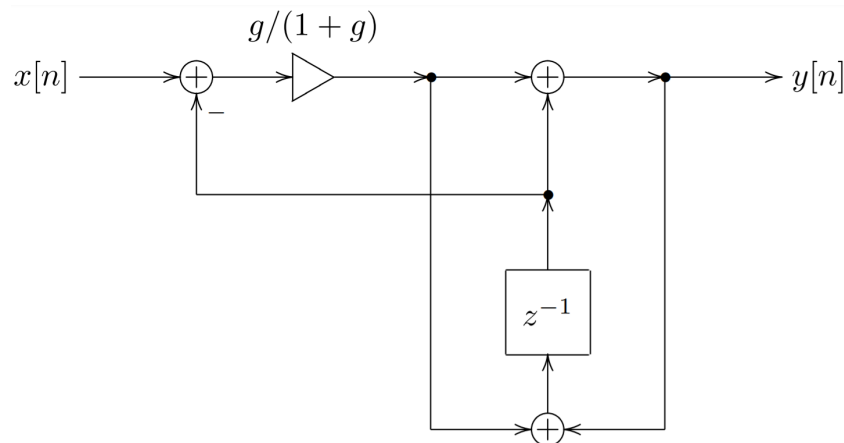


Figura 57: Filtro TPT passa-basso ad 1 polo senza retroazioni *delayless* (immagine adattata da [8]).

6.10.2 Filtro passa-basso a 4 poli lineare

Si consideri la risposta istantanea di un filtro passa-basso del primo ordine mostrato in figura 55. Il ladder del Minimoog può essere ottenuto connettendo in serie 4 di questi filtri. Le loro risposte istantanee sono indicate come $f_n(\varepsilon) = g\varepsilon + s_n$.

Sebbene il coefficiente g sia uguale per tutti 4 i filtri, la variabile s_n dipende dal loro stato interno e può essere diverso per ognuno. Combinando 2 filtri in serie si ottiene:

$$f_2(f_1(\varepsilon)) = g(g\varepsilon + s_1) + s_2 = g^2\varepsilon + gs_1 + s_2$$

Aggiungendo il terzo:

$$f_3(f_2(f_1(\varepsilon))) = g(g^2\varepsilon + gs_1 + s_2) + s_3 = g^3\varepsilon + g^2s_1 + gs_2 + s_3$$

Aggiungendo il quarto:

$$f_4(f_3(f_2(f_1(\varepsilon)))) = g(g^3\varepsilon + g^2s_1 + gs_2 + s_3) + s_4 = g^4\varepsilon + g^3s_1 + g^2s_2 + gs_3 + s_4 = G\varepsilon + S$$

dove

$$G = g^4S = g^3s_1 + g^2s_2 + gs_3 + s_4$$

La struttura del filtro ladder nella sua forma istantanea $G\varepsilon + S$ è mostrata nella figura 58.

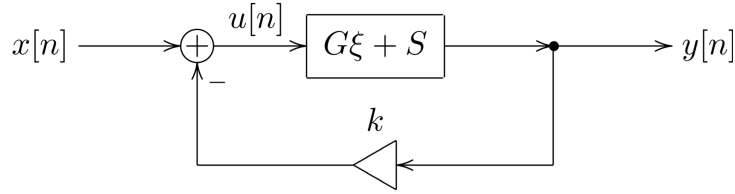


Figura 58: Struttura della risposta istantanea di un filtro ladder TPT passa-basso a 4 poli (immagine adattata da [8]).

La risoluzione del circuito può essere fatta usando la variabile u la quale indica il campione in input all'integratore:

$$\begin{aligned} u &= x - ky = x - k(Gu + S) \\ (1 + kG)u &= x - kS \\ u &= \frac{x - kS}{1 + kG} \end{aligned}$$

Il valore di u può essere usato per computare le uscite di ogni filtro passa-basso y_n , aggiornare le loro variabili di stato s_n ed ottenere y_4 come l'uscita del quarto filtro. Si ricorda che $0 < g < 1$ e quindi anche $0 < G < 1$.

Il seguente codice è un'implementazione del filtro ladder a 4 poli lineare:

```

1 float processSample(float x)
2 {
3     // Pre-calculus
4     input = x;
5     S = (g * g * g) * s1 + (g * g) * s2 + (g) * s3 + s4;
6     u = (x - k * S) / (1 + k * G);
7     u = tanh(u);
8     // Forward path
9     // Filter 1
10    v1 = (u - s1) * G1p;
11    y1 = v1 + s1;
12    s1 = y1 + v1;
13    // Filter 2
14    v2 = (y1 - s2) * G1p;
15    y2 = v2 + s2;
16    s2 = y2 + v2;
17    // Filter 3
18    v3 = (y2 - s3) * G1p;
19    y3 = v3 + s3;
20    s3 = y3 + v3;
21    // Filter 4
22    v4 = (y3 - s4) * G1p;
23    y4 = v4 + s4;
24    s4 = y4 + v4;
25    return y4;
26 }

```

Nota: quando la risonanza del filtro aumenta, le frequenze più basse diminuiscono attenuando molto il segnale. Questo problema può essere risolto in 2 modi (nel codice non sono implementate):

- Amplificando il segnale in input al primo filtro di un fattore $(1+k)$.
In questo modo, se la risonanza aumenta, anche l'ampiezza delle componenti del segnale aumenta andando a controbilanciare l'attenuazione dovuta all'aumento della risonanza.
- Introducendo un filtro passa-alto nella retroazione per eliminare le basse frequenze in modo che solo le frequenze più alte del segnale siano modificate dal feedback.

6.10.3 Equazione zero-delay feedback non lineare

L'introduzione di un blocco non lineare all'interno del modello del filtro complica il calcolo dell'uscita.

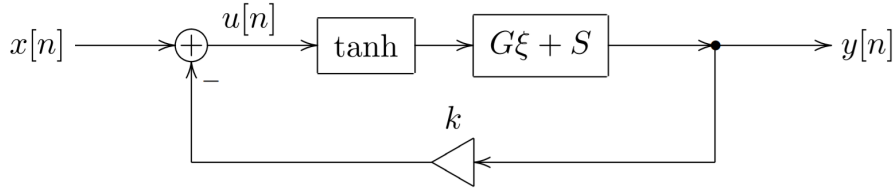


Figura 59: Filtro ladder TPT non lineare nella sua risposta istantanea (immagine adattata da [8]).

Si consideri la figura 59; l'equazione *zero-delay feedback* è la seguente:

$$u = x - k(G \tanh(u) + S)$$

Essa è un'equazione trascendente non risolvibile tramite un metodo analitico ma attraverso un metodo numerico che permette di trovare soluzioni approssimate. Uno degli algoritmi più popolari nell'ambito del DSP è il metodo Newton-Raphson spiegato nelle prossime sezioni. L'equazione sopra può essere riscritta come:

$$(x - kS) - u = kG \tanh(u)$$

si può notare che se $k \geq 0$ allora il membro di destra è una funzione crescente di u mentre il membro di sinistra è una funzione decrescente di u .

Questo vuol dire che le 2 funzioni si intersecano in un solo punto dando una sola soluzione. Una volta trovata la soluzione per u , si calcola il valore in uscita del waveshaper e poi si aggiorna il valore dello stato di ogni filtro passa-basso per poi aggiornare l'uscita y .

6.10.4 Filtro passa-basso a 4 poli non lineare

Come visto precedentemente, le risposte istantanee dei filtri lineari sono funzioni lineari dei loro input, quindi, le risposte istantanee di filtri non lineari sono funzioni non lineari dei loro input:

$$y = F(x, S)$$

dove x indica il segnale in input e S indica lo stato del filtro. Questa equazione descrive l'uscita della figura 41. Se si collegano in serie 4 filtri passa-basso non lineari ad un polo identici e si aggiunge una retroazione, si ottiene la struttura di un filtro ladder a 4 poli non lineare. La figura 60 mostra il filtro nella sua risposta istantanea. Si indichi con u il segnale in ingresso al primo filtro.

L'equazione zero-delay feedback dell'intero ladder diventa:

$$u = x - k \cdot F(F(F(F(u, S_1), S_2), S_3), S_4) \quad (4)$$

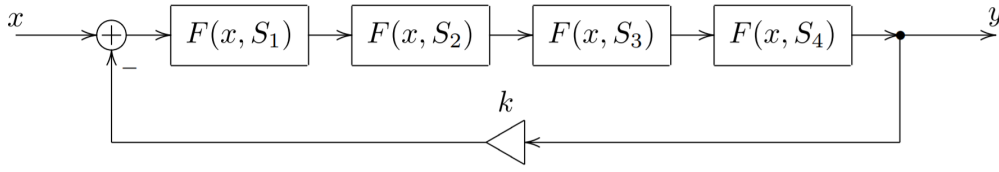


Figura 60: Filtro ladder non lineare nella sua risposta istantanea (immagine adattata da [8]).

dove S_n indica lo stato dei rispettivi filtri all'interno del ladder.

Se si usa un metodo iterativo per risolvere l'equazione $F(x, S)$ il costo è elevato siccome bisogna trovare una soluzione numerica all'equazione zero-delay feedback del filtro ad 1 polo. Nell'equazione 4 dove le funzioni sono 'innestate' tra di loro la complessità del calcolo aumenta di molto. Per questo motivo, una soluzione valida è quella di riscrivere l'equazione 4 come un sistema di equazioni, ognuna delle quali descrive la risposta istantanea di ogni filtro ad un polo all'interno della struttura come mostra la figura 61. Indicando i segnali in input ad ogni filtro ad un polo come x_n e i segnali di output come y_n , le equazioni zero-delay feedback possono essere scritte come:

$$y_n = g \tanh(x_n - y_n) + s_n$$

Dato che $x_{n+1} = y_n$ si può riscrivere sistema in questo modo:

$$y_n = g \tanh(y_{n-1} - y_n) + s_n \quad n = 1, \dots, 4$$

Sapendo che il segnale in input al primo filtro è $y_0 = x - ky_4$, si possono esplicitare le uscite dei 4 filtri passa-basso che compongono il ladder:

$$\begin{aligned} y_1 &= g \tanh(y_0 - y_1) + s_1 = g \tanh(x - ky_4 - y_1) + s_1 \\ y_2 &= g \tanh(y_1 - y_2) + s_2 \\ y_3 &= g \tanh(y_2 - y_3) + s_3 \\ y_4 &= g \tanh(y_3 - y_4) + s_4 \end{aligned}$$

Il sistema di equazioni ottenuto può essere scritto in maniera più concisa nel seguente modo:

$$\mathbf{y} = (y_1 \ y_2 \ y_3 \ y_4)^T$$

Le funzioni che descrivono l'uscita del filtro possono anch'esse essere scritte in forma vettoriale:

$$\Phi(\mathbf{y}) = \begin{pmatrix} g \tanh(x - ky_4 - y_1) + s_1 \\ g \tanh(y_1 - y_2) + s_2 \\ g \tanh(y_2 - y_3) + s_3 \\ g \tanh(y_3 - y_4) + s_4 \end{pmatrix}$$

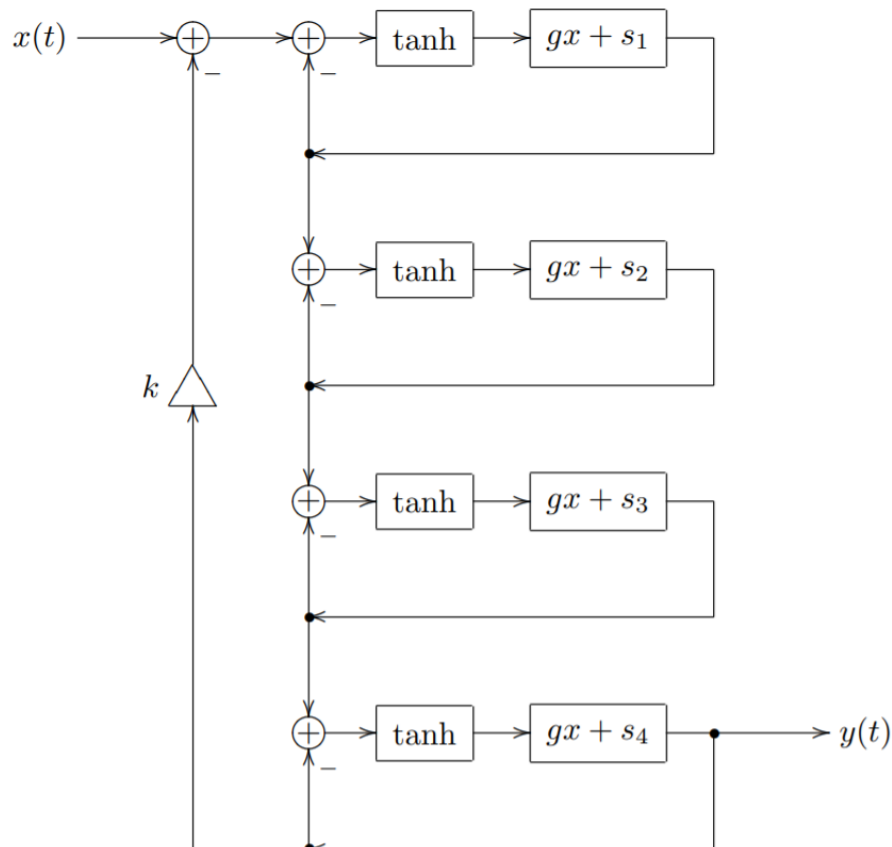


Figura 61: Filtro ladder del Minimoog nella sua risposta istantanea con emulazione degli OTA (immagine adattata da [8]).

Infine, il sistema di equazioni può essere scritto in maniera concisa nel seguente modo:

$$\mathbf{y} = \Phi(\mathbf{y})$$

dove Φ indica il vettore delle funzioni da applicare al vettore dei valori $\mathbf{y}$. Il sistema da risolvere è formato da 4 equazioni non lineari a 4 incognite.

Algoritmo di Newton-Raphson

Per la risoluzione del sistema è utilizzato l'algoritmo di Newton-Raphson⁸ multi-dimensionale in modo da risolvere simultaneamente tutte le equazioni zero-delay feedback non lineari nel sistema.

⁸Anche chiamato metodo delle tangenti

Esso permette di computare iterativamente il vettore delle uscite successive $\mathbf{y}_{n+1}$ partendo da quelle precedenti $\mathbf{y}_n$:

$$\mathbf{y}_{n+1} = \mathbf{y}_n - \left(\frac{\partial(\Phi(\mathbf{y}) - \mathbf{y})}{\partial \mathbf{y}}(\mathbf{y}_n) \right)^{-1} \cdot (\Phi(\mathbf{y}_n) - \mathbf{y}_n) \quad (5)$$

Si spiegano di seguito i passaggi dell'algoritmo implementato nella classe *Matrix*. L'obiettivo è risolvere $\Phi(\mathbf{y}) - \mathbf{y} = 0$ ovvero il seguente sistema di 4 equazioni trascendenti:

$$\begin{aligned} f_1 &= g \cdot \tanh(x - ky_4 - y_1) + s_1 - y_1 = 0 \\ f_2 &= g \cdot \tanh(y_1 - y_2) + s_2 - y_2 = 0 \\ f_3 &= g \cdot \tanh(y_2 - y_3) + s_3 - y_3 = 0 \\ f_4 &= g \cdot \tanh(y_3 - y_4) + s_4 - y_4 = 0 \end{aligned}$$

Il metodo che si occupa della risoluzione si chiama *newtonRaphson()*.

Dal momento che l'algoritmo non trova il risultato esatto ma una sua approssimazione, per prima cosa bisogna stabilire una soglia che indichi l'errore accettabile nel calcolo delle uscite dei filtri. In questo caso è stato scelto un errore dell'ordine dei micro (10^{-6}). Successivamente si computa il vettore dei *residui* composto dal risultato delle 4 funzioni citate sopra:

$$\Phi(\mathbf{y}_n) - \mathbf{y}_n \rightarrow (f_1 \quad f_2 \quad f_3 \quad f_4)$$

Successivamente si calcola la norma del vettore nel seguente modo:

$$norm = \sqrt{f_1^2 + f_2^2 + f_3^2 + f_4^2}$$

e si verifica che $norm \leq threshold$. Il calcolo della norma è fondamentale dato che essa indica di quanto l'algoritmo è vicino alla soluzione.

In particolare, se $norm \leq threshold$ l'algoritmo termina perchè significa che i valori di $y_1 \dots y_4$ utilizzati nel calcolo di $f_1 \dots f_4$ sono uguali a quelli corretti a meno di un *errore* $\leq threshold$, in caso negativo l'algoritmo continua.

Successivamente (se l'algoritmo non termina subito) bisogna computare la matrice delle derivate parziali chiamata anche *Jacobian Matrix*. Nell'equazione 5 essa è rappresentata nel seguente modo:

$$\frac{\partial(\Phi(\mathbf{y}) - \mathbf{y})}{\partial \mathbf{y}}(\mathbf{y}_n)$$

Essa, in questo caso, è una matrice 4x4 dove per ogni riga è calcolata la derivata parziale di una determinata funzione f_n rispetto una determinata incognita y_n .

In particolare, nella prima riga, la prima colonna rappresenta il risultato della derivata parziale di f_1 rispetto ad y_1 , la seconda colonna rappresenta il risultato della derivata parziale di f_1 rispetto y_2 , la terza colonna rappresenta il risultato della derivata parziale di f_1 rispetto y_3 , la quarta colonna rappresenta il risultato della derivata parziale di f_1 rispetto y_4 . Lo stesso procedimento viene fatto per le altre 3 righe prendendo in considerazione le altre 3 funzioni.

Il tutto può essere espresso matematicamente nel seguente modo:

$$\begin{pmatrix} \frac{\partial f_1}{\partial y_1} & \frac{\partial f_1}{\partial y_2} & \frac{\partial f_1}{\partial y_3} & \frac{\partial f_1}{\partial y_4} \\ \frac{\partial f_2}{\partial y_1} & \frac{\partial f_2}{\partial y_2} & \frac{\partial f_2}{\partial y_3} & \frac{\partial f_2}{\partial y_4} \\ \frac{\partial f_3}{\partial y_1} & \frac{\partial f_3}{\partial y_2} & \frac{\partial f_3}{\partial y_3} & \frac{\partial f_3}{\partial y_4} \\ \frac{\partial f_4}{\partial y_1} & \frac{\partial f_4}{\partial y_2} & \frac{\partial f_4}{\partial y_3} & \frac{\partial f_4}{\partial y_4} \end{pmatrix}$$

Di seguito è rappresentata la *Jacobian Matrix* partendo dalle funzioni f_1 f_2 f_3 f_4 precedentemente definite:

$$\begin{pmatrix} -g + g \tanh^2(y_1 + ky_4 - x) - 1 & 0 & \dots & \dots \\ g - g \tanh^2(y_1 - y_2) & -g + g \tanh^2(y_1 - y_2) - 1 & \dots & \dots \\ 0 & g - g \tanh^2(y_2 - y_3) & \dots & \dots \\ 0 & 0 & 0 & \dots \\ \dots & 0 & -gk + g \tanh^2(y_1 + ky_4 - x) & \dots \\ \dots & 0 & 0 & \dots \\ \dots & -g + g \tanh^2(y_2 - y_3) - 1 & 0 & \dots \\ \dots & g - g \tanh^2(y_3 - y_4) & -g + g \tanh^2(y_3 - y_4) - 1 & \dots \end{pmatrix}$$

Si noti che la matrice è stata divisa in 2 per questioni di spazio. La prima parte corrisponde alla colonna 1 e 2 mentre la seconda parte corrisponde alla colonna 3 e 4. Queste operazioni sono effettuate dal metodo *calculateJacobianMatrix()*.

Successivamente viene calcolata la *Jacobian Matrix* inversa tramite il metodo *inverse()* e poi viene aggiornato il vettore degli output $\mathbf{y}_{n+1}$ eseguendo una moltiplicazione tra la *Jacobian Matrix* inversa⁹ e il vettore dei residui¹⁰ negato attraverso il metodo *updateOutput()*. Una volta aggiornati i risultati si calcola nuovamente la norma e si verifica che sia minore o uguale della soglia. In caso positivo l'algoritmo termina, in caso negativo viene computato un altro ciclo con il vettore contenente i valori degli output $\mathbf{y}$ aggiornati allo step precedente. La figura 62 mostra il diagramma di flusso dell'algoritmo.

⁹ $\left(\frac{\partial(\Phi(\mathbf{y}) - \mathbf{y})}{\partial \mathbf{y}}(\mathbf{y}_n) \right)^{-1}$

¹⁰ $-(\Phi(\mathbf{y}_n) - \mathbf{y}_n)$: il segno '-' corrisponde a quello dopo y_n e prima della *Jacobian Matrix* inversa nella 5

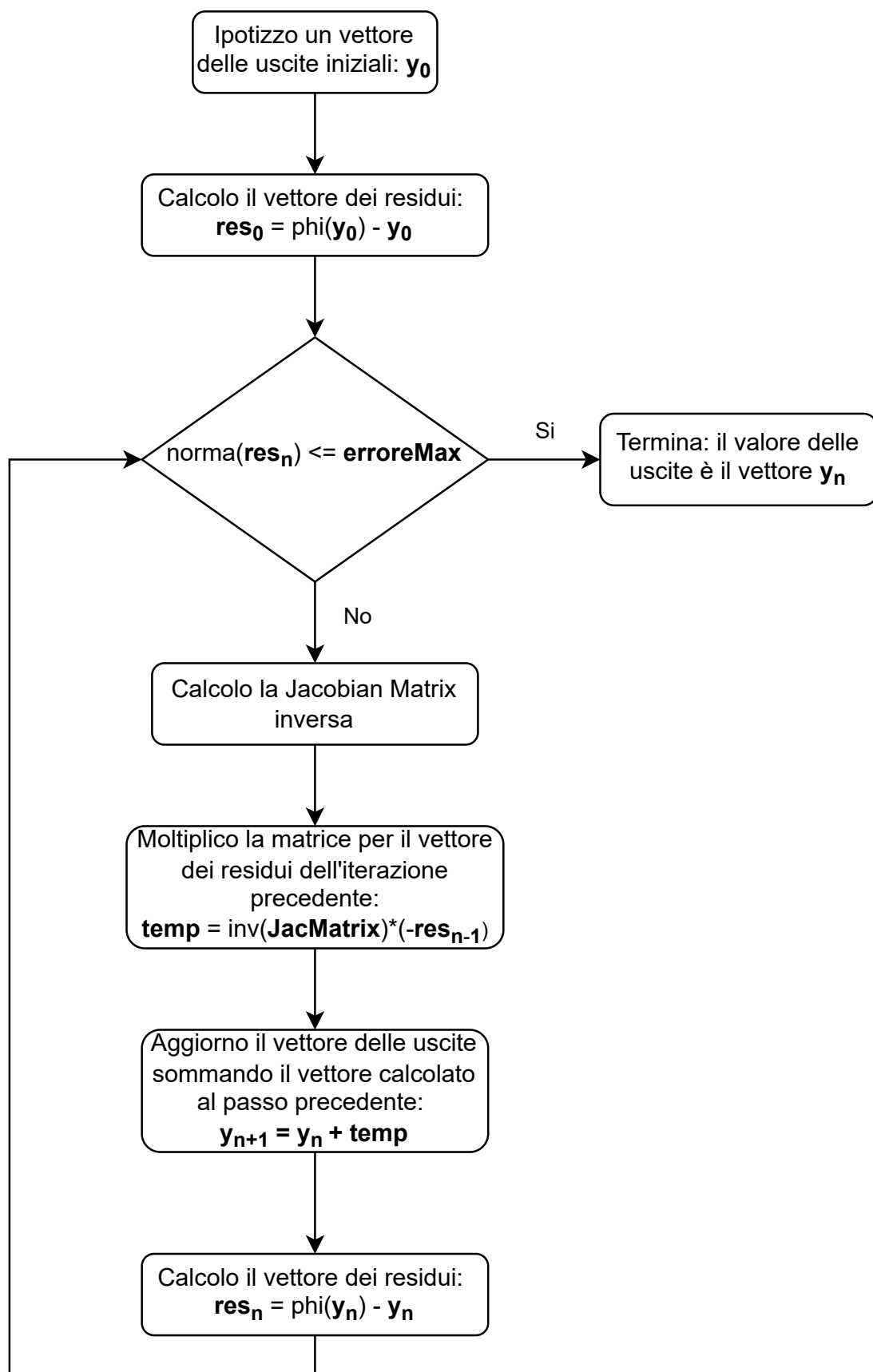


Figura 62: Diagramma di flusso dell'algoritmo di Newton-Raphson multidimensionale.

Capitolo 7

Testing

7.1 Oscillatori

Per quanto riguarda gli oscillatori la fase di testing comprende:

- La verifica della loro accordatura per accertarsi che le onde generate siano intonate correttamente.
- La verifica che le prime 10 armoniche dell'onda abbiano una potenza simile a quella teorica calcolata.
- La comparazione tra la tecnica di sintesi BLIT (limitata in banda) e la tecnica di sintesi naive (non limitata in banda) in termini spettrali.

Le onde considerate per il test sono la dente di sega e la quadra dato che le altre sono calcolate a partire da esse o usando la medesima tecnica. Infatti, il processo di generazione dell'onda dente di sega inverso è lo stesso della dente di sega e il processo di generazione delle due onde rettangolari è analogo a quello dell'onda quadra. L'onda triangolare è ricavata integrando la quadra, mentre la triangolare/dente di sega è ottenuta facendo una somma pesata di queste 2 onde. Le frequenze prese in esame sono comprese tra il Do2 e il Do8 ognuna a distanza di un'ottava; di seguito la tabella:

Do2	Do3	Do4	Do5	Do6	Do7	Do8
65.41Hz	130.81Hz	261.63 Hz	523.25 Hz	1046.50 Hz	2093 Hz	4186.01 Hz

7.1.1 Onda dente di sega

Per verificare che la potenza delle prime 10 armoniche sia corretta, si introduce la serie di Fourier dell'onda dente di sega:

$$Saw(t) = \sum_{k=1}^{\infty} \frac{\sin(2\pi k f t)}{k}$$

Sviluppando la serie fino alla decima armonica si può notare che un'onda dente di sega contiene armonici pari e dispari che hanno una frequenza multipla della fondamentale di un fattore k . Inoltre, l'ampiezza delle componenti sinusoidali che compongono il segnale sono scalate di un valore pari a $\alpha = 1/k$.

Nell'appendice A sono riportate ulteriori informazioni riguardo la potenza ideale di ciascuna armonica che compone l'onda e quella effettivamente misurata. Dai dati si può notare che l'errore è nella maggior parte dei casi trascurabile eccetto per le componenti con una frequenza molto vicina a quella di Nyquist le quali hanno una potenza molto diversa da quella ideale a causa della attenuazione dovuta alla finestra di Hamming e all'effetto filtrante del leaky integrator. Per quanto riguarda l'intonazione, la fondamentale di ogni onda ha una frequenza che equivale a quelle in tabella 7.1 o che si discosta di un fattore dell'ordine dei centesimi per il Do2 e il Do3.

7.1.2 Onda quadra

Per verificare che la potenza delle prime 10 armoniche sia corretta, si introduce la serie di Fourier dell'onda quadra:

$$Square(t) = \sum_{k=1}^{\infty} \frac{\sin((2k-1)2\pi f t)}{(2k-1)}$$

Sviluppando la serie fino alla decima armonica si può notare che un'onda quadra contiene solamente armonici dispari che quindi avranno una frequenza multipla della fondamentale di un fattore dispari $(2k-1)$. Inoltre l'ampiezza delle componenti sinusoidali che compongono il segnale sono scalate di un valore pari a: $\alpha = 1/(2k-1)$.

Così come per l'onda dente di sega, anche per la quadra sono riportate ulteriori informazioni e dati nell'appendice A. Le osservazioni che si possono fare sono analoghe a quelle fatte nel precedente paragrafo.

7.1.3 Grafici dell'onda dente di sega

Nei seguenti grafici sono mostrati:

- In grigio lo spettro dell'onda dente di sega generata tramite la tecnica naive.
- In blu lo spettro dell'onda dente di sega generata tramite la tecnica BLIT.
- In nero una linea spezzata che collega i valori ideali degli armonici calcolati nella tabella A.0.1.

Questo confronto è fatto per dimostrare come la tecnica naive presenti una quantità di aliasing nettamente maggiore rispetto alla tecnica BLIT soprattutto per frequenze elevate. A titolo di verifica, di seguito è fatta un'analisi qualitativa di 2 dei grafici mostrati. Si consideri lo spettro del segnale in figura 65 dove la fondamentale è di 261.63 Hz. Si può notare che l'aliasing per l'onda naive è compreso tra i -45 e i -70 dBFS mentre quello dell'onda generata tramite BLIT è compreso tra i -70 e i -100 dBFS. Nel secondo caso ho 30dB di aliasing in meno rispetto il primo, questo implica che il suono sarà più pulito e conterrà meno rumore.

Si consideri invece la figura 69 dove la fondamentale ha una frequenza di 4186 Hz. Si può notare che l'aliasing per l'onda naive è compreso tra i -15 e i -50 dBFS mentre quello dell'onda generata tramite BLIT è compreso tra i -60 e i -120 dBFS. Nel secondo caso ho circa 50dB di aliasing in meno rispetto il primo. Inoltre, in questo esempio il rumore generato dalla tecnica naive è più percepibile a livello uditivo rispetto il precedente (-45 dB) dato che la sua potenza è maggiore (-15 dB), mentre l'aliasing generato dalla tecnica BLIT è impercettibile.

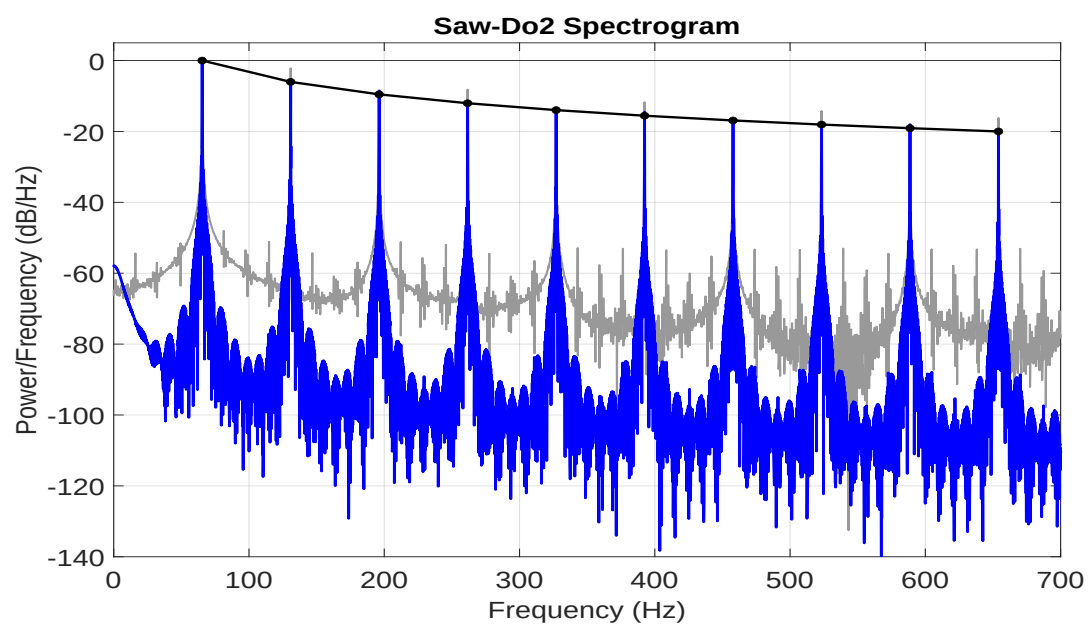


Figura 63: Spettro di un'onda dente di sega. Frequenza: 65.41Hz (Do2).

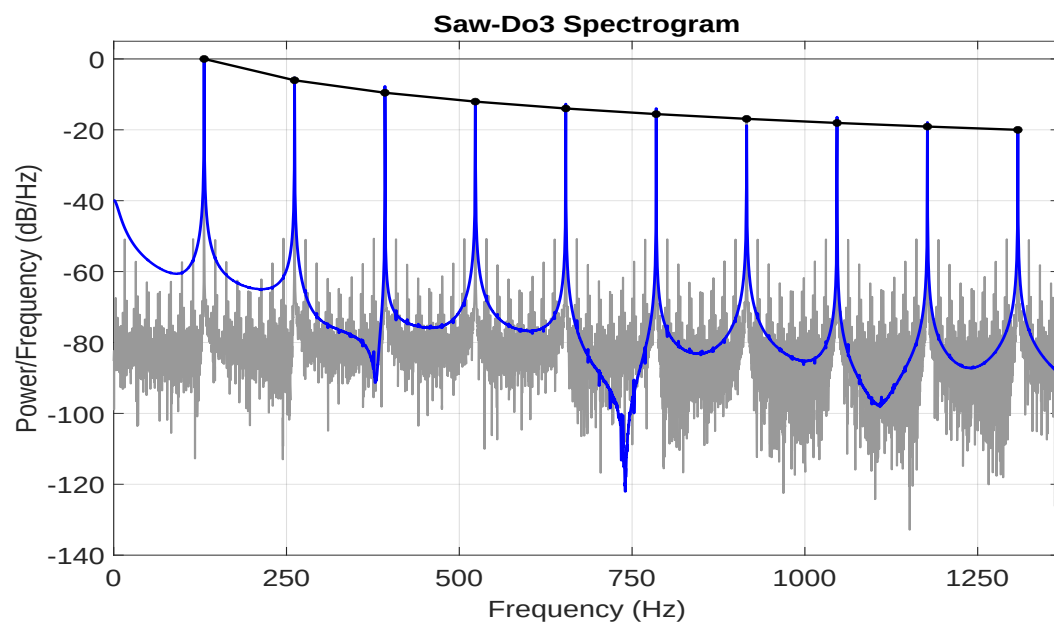


Figura 64: Spettro di un'onda dente di sega. Frequenza: 130.81Hz (Do3).

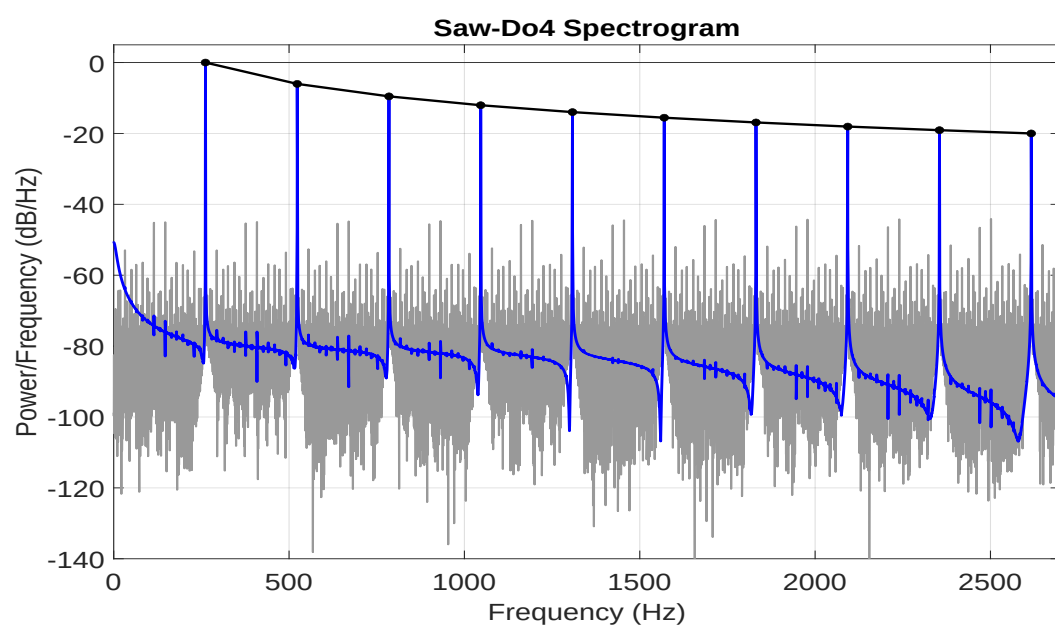


Figura 65: Spettro di un'onda dente di sega. Frequenza: 261.63Hz (Do4).

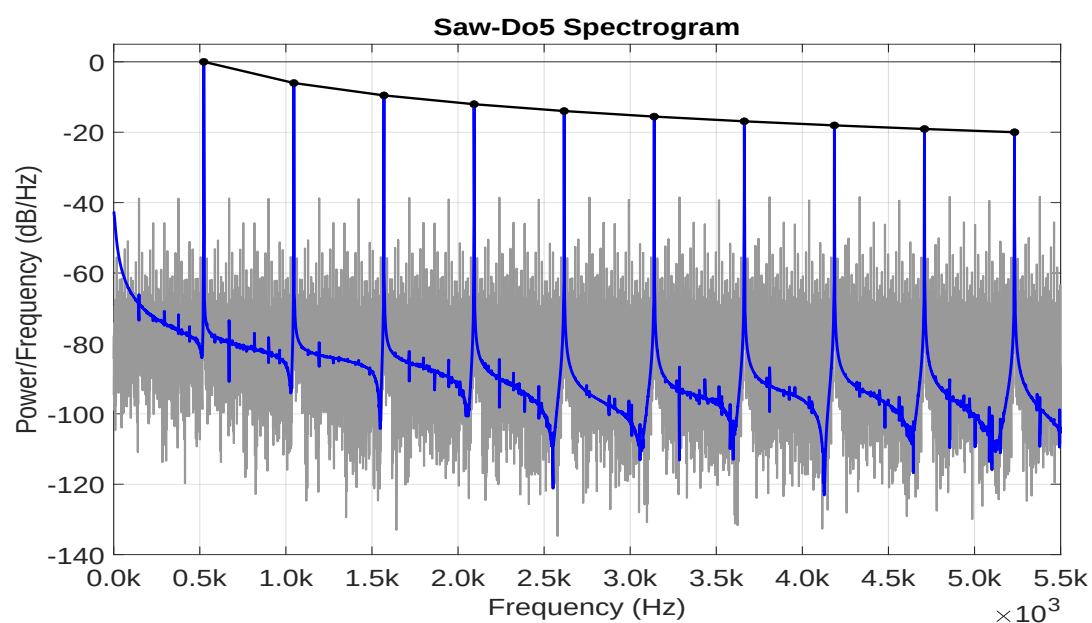


Figura 66: Spettro di un'onda dente di sega. Frequenza: 523.25Hz (Do5).

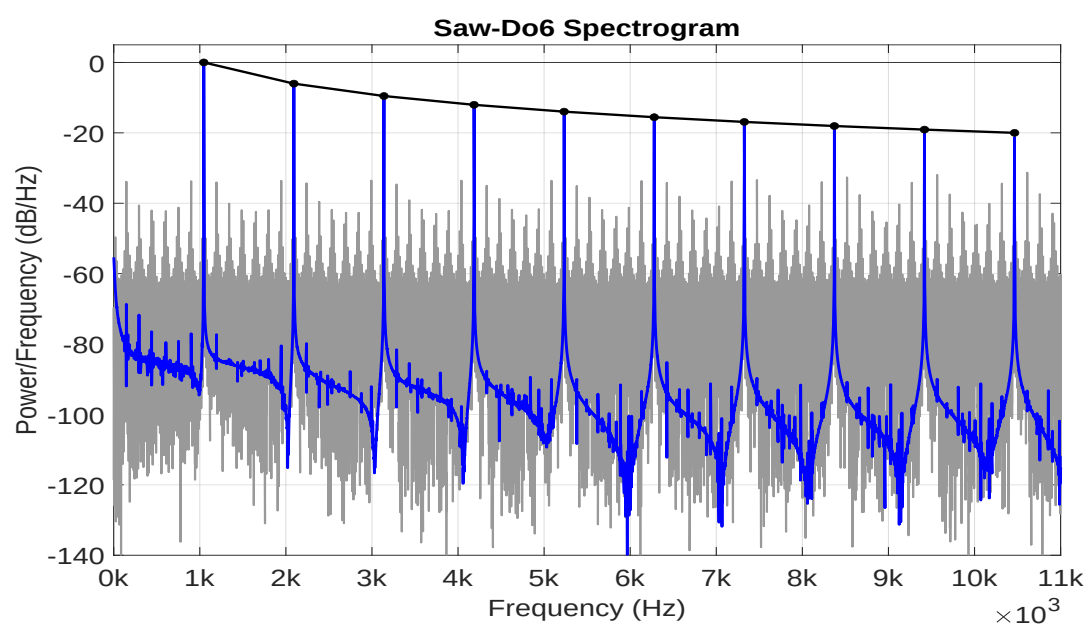


Figura 67: Spettro di un'onda dente di sega. Frequenza: 1046.50Hz (Do6).

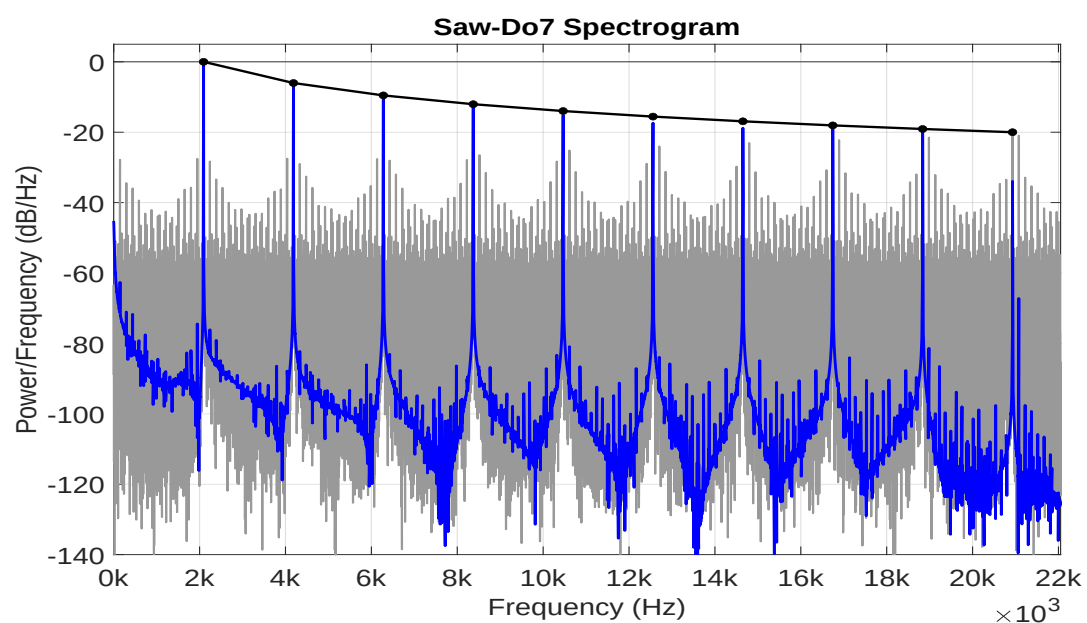


Figura 68: Spettro di un'onda dente di sega. Frequenza: 2093Hz (Do7).

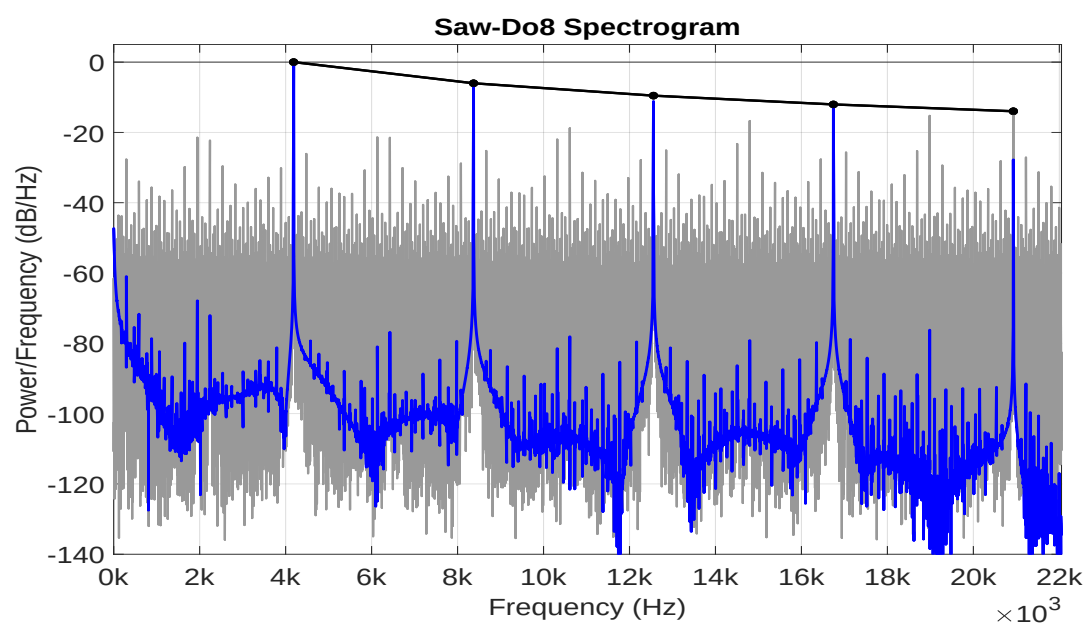


Figura 69: Spettro di un'onda dente di sega. Frequenza: 4186.01Hz (Do8).

7.1.4 Grafici dell'onda quadra

La stessa procedura di testing è stata eseguita prendendo in esame l'onda quadra. Si consideri lo spettro in figura 70 dove la fondamentale è di 65.41 Hz.

Si può notare che l'aliasing per l'onda naive è compreso tra i -55 e i -80 dBFS mentre quello dell'onda generata tramite BLIT è compreso tra i -60 e i -110 dBFS. Nel secondo caso ho circa 30dB di aliasing in meno rispetto il primo, questo implica (come già detto) che il suono sarà più pulito e conterrà meno rumore.

Si consideri invece la figura 75 dove la fondamentale ha una frequenza di 2093 Hz. In questo caso si può notare che l'aliasing per l'onda naive è compreso tra i -25 e i -60 dBFS mentre quello dell'onda generata tramite BLIT è compreso tra i -60 e i -120 dBFS. Nel secondo caso ho circa 50dB di aliasing in meno rispetto il primo e quindi avrò meno rumore.

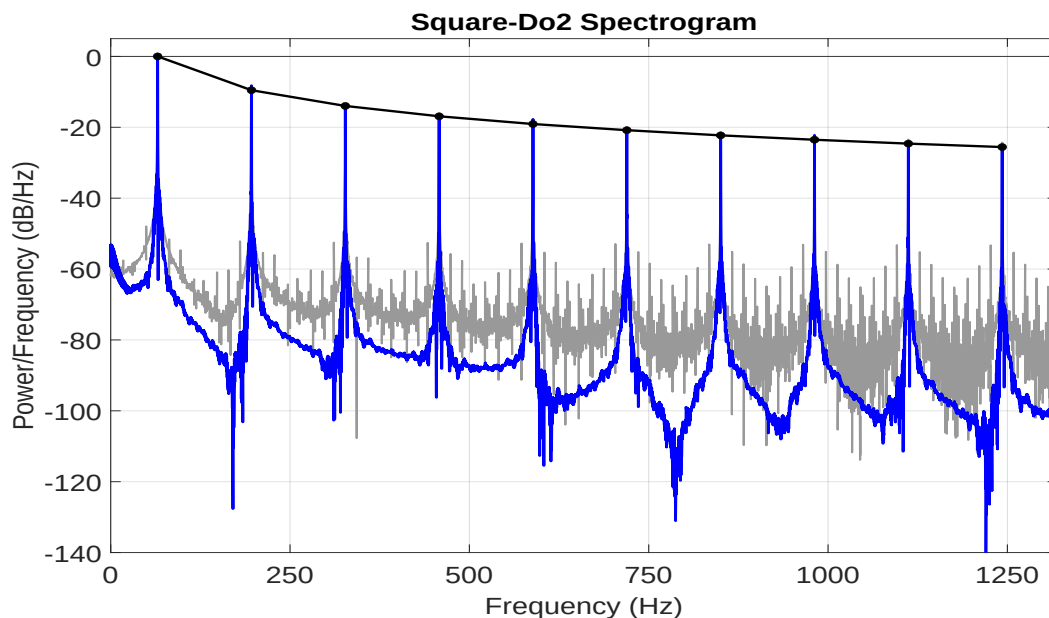


Figura 70: Spettro di un'onda quadra. Frequenza: 65.41Hz (Do2).

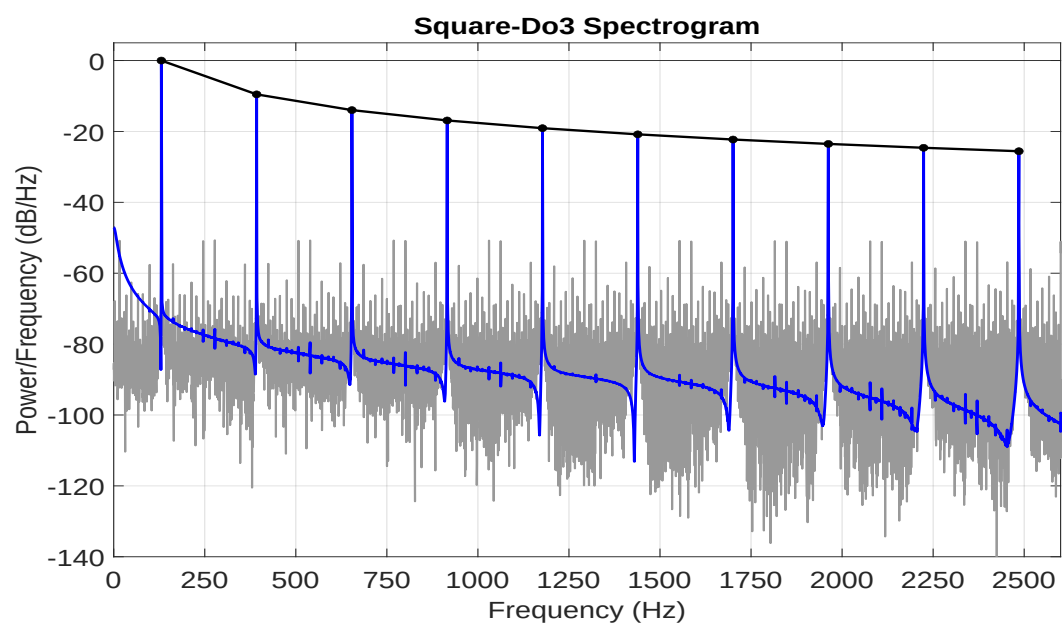


Figura 71: Spettro di un'onda quadra. Frequenza: 130.81Hz (Do3).

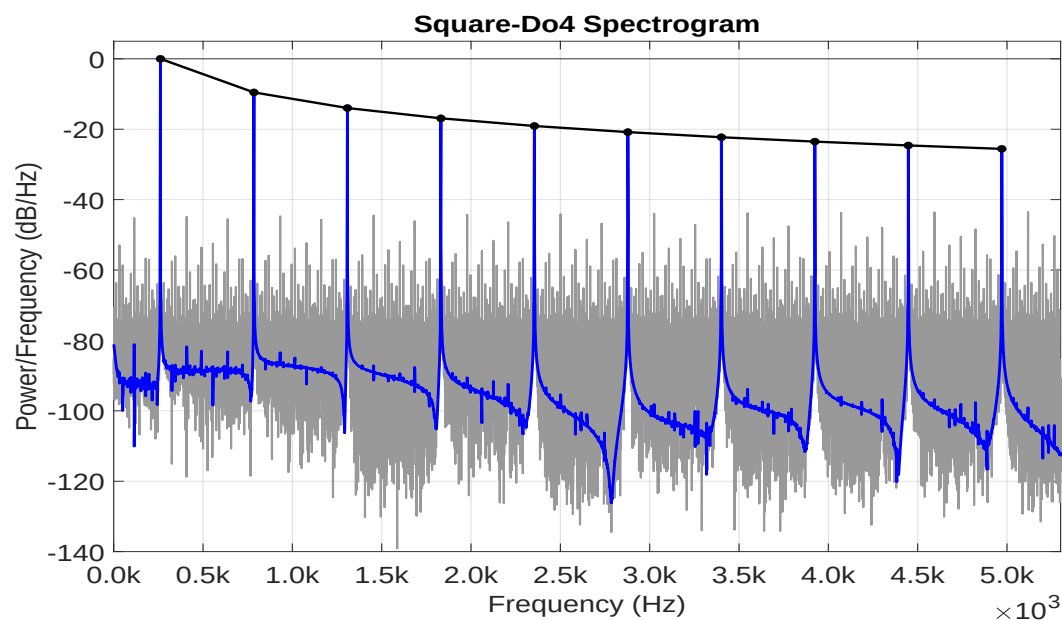


Figura 72: Spettro di un'onda quadra. Frequenza: 261.63Hz (Do4).

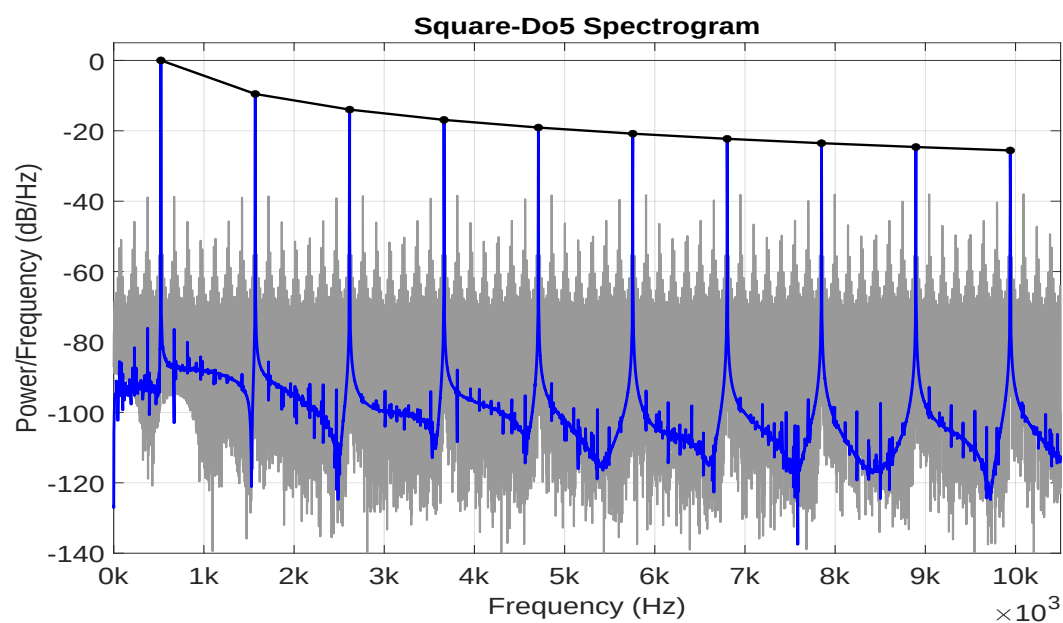


Figura 73: Spettro di un'onda quadra. Frequenza: 523.25Hz (Do5).

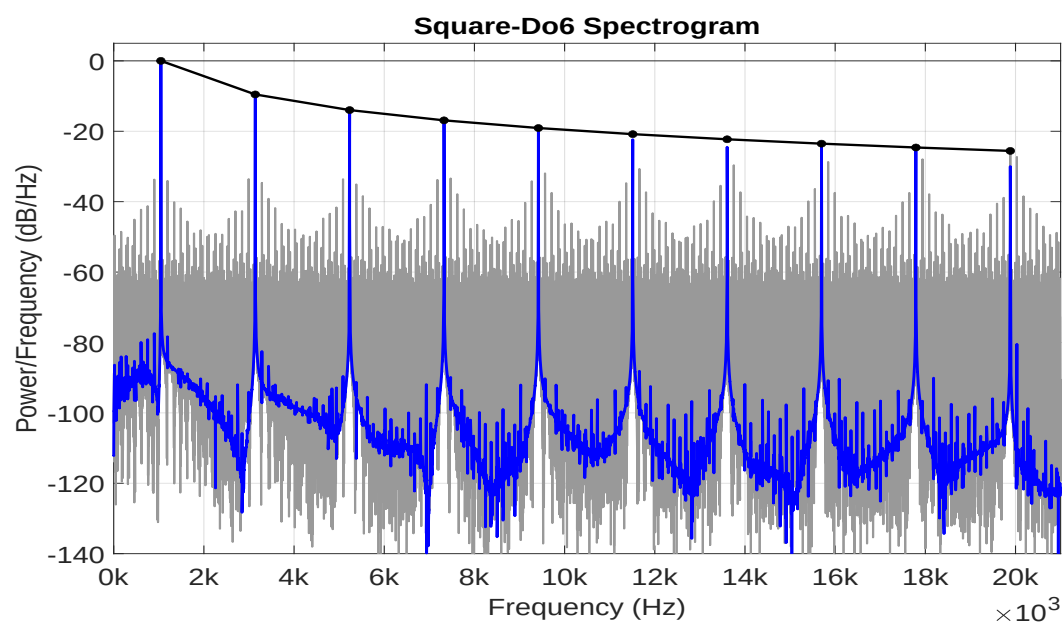


Figura 74: Spettro di un'onda quadra. Frequenza: 1046.50Hz (Do6).

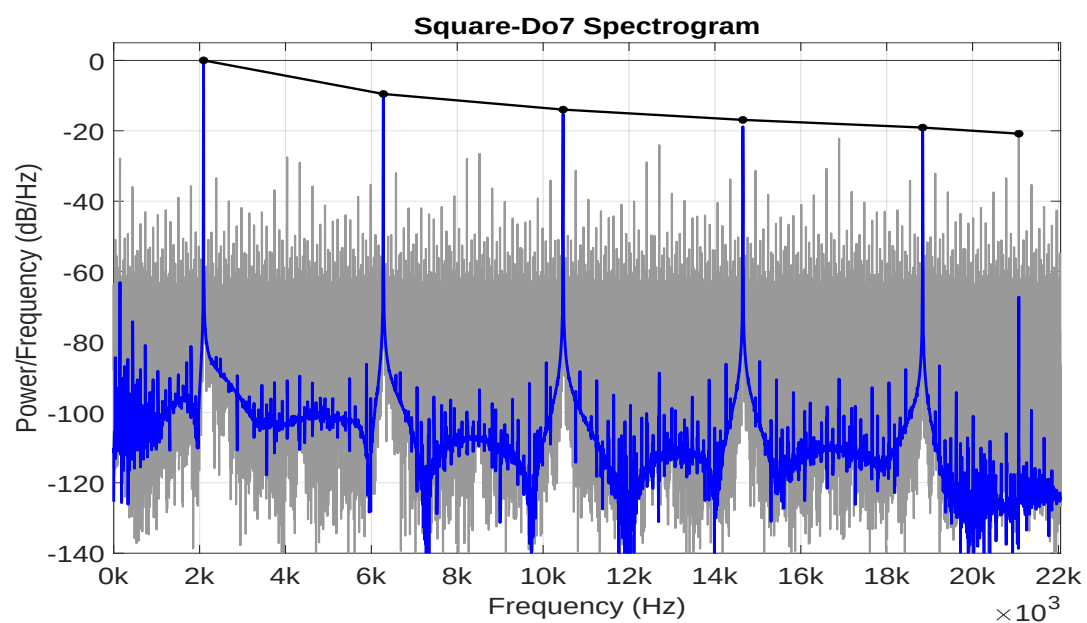


Figura 75: Spettro di un'onda quadra. Frequenza: 2093Hz (Do7).

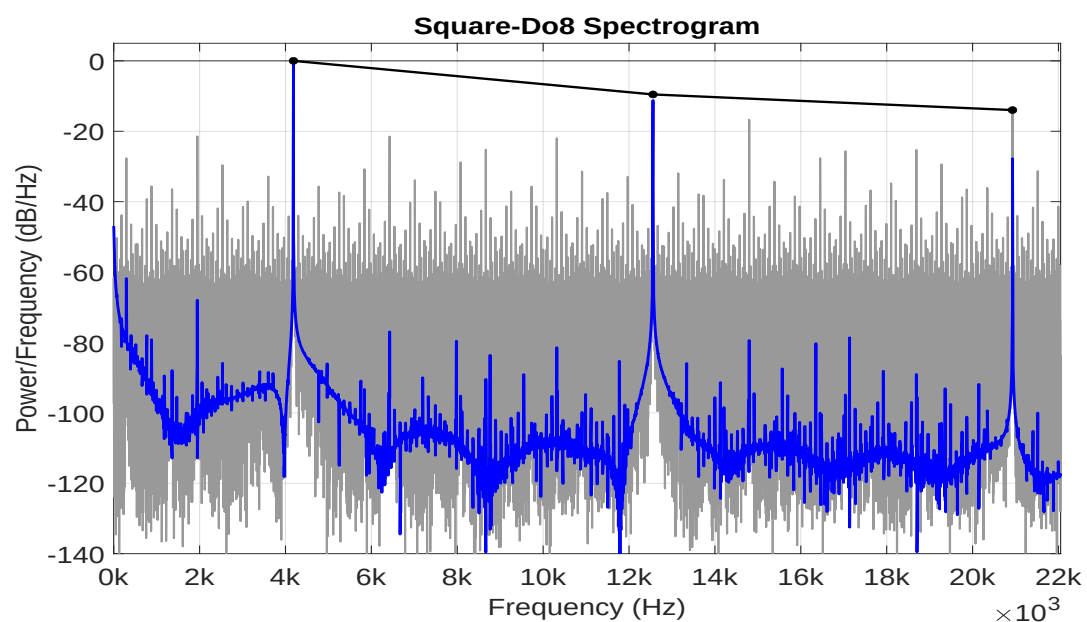


Figura 76: Spettro di un'onda quadra. Frequenza: 4186.01Hz (Do8)

7.2 Filtro

Il filtro è stato testato con la tecnica dello swept-sine. Essa è in grado di misurare simultaneamente la risposta all'impulso (e in frequenza) del filtro e la sua distorsione armonica dando in input al sistema una sinusoide con una frequenza crescente dai 10 Hz ai 22.000 Hz. I seguenti grafici mostrano la risposta in frequenza del filtro per diversi valori di cutoff e di risonanza. In blu è indicata la risposta in frequenza della prima armonica (ovvero la sinusoide a frequenza variabile data in input), negli altri colori è indicata la risposta in frequenza delle altre componenti generate dalla distorsione del filtro. Più precisamente: in rosso è indicata la seconda armonica, in arancione la terza, in viola la quarta e in verde la quinta.

I valori delle frequenze di taglio scelti sono: 500Hz, 1kHz, 5kHz e 10kHz, in modo da poter visualizzare la risposta del filtro da frequenze più basse a frequenze più alte. Come valori di risonanza sono stati scelti 1 e 15 ¹ in modo da poter verificare il comportamento del filtro in uno stato non risonante e in uno stato risonante. Nelle figure 77, 79, 81 e 83 è mostrato il comportamento del filtro in uno stato non risonante per le frequenze di taglio scelte. Nel grafico 79 si vede molto bene il roll-off del filtro di circa 80dB/dec (24dB/oct) tipico del ladder del Minimoog. Nelle figure 78, 80, 82 e 84 è mostrato il comportamento del filtro in uno stato risonante. Purtroppo si può notare che i picchi non avvengono esattamente ai valori delle frequenze di taglio (come dovrebbe essere) ma in un loro intorno sempre più crescente al crescere del cutoff. In particolare: per un cutoff di 500Hz il picco è a 625 Hz, per un cutoff di 1000Hz il picco è a 1125Hz, per un cutoff di 5000Hz il picco è a 4510Hz, per un cutoff di 10000Hz il picco è a 7390Hz.

Questo errore è dovuto alla deformazione della frequenza di taglio spiegata nella sezione 6.8. Per poterlo ridurre possono essere fatti accorgimenti più sofisticati in modo da definire una funzione che mappa la frequenza ω_c nel dominio tempo-continuo nella frequenza $\tilde{\omega}_c$ nel dominio tempo-discreto in modo più preciso ².

¹Il filtro entra in uno stato di auto-oscillazione già per valori di risonanza circa uguali a 3. Come valore è stato scelto 15 dato che in questo modo i picchi di risonanza nel grafico sono meglio visibili.

²Ulteriori informazioni possono essere trovate nel capitolo 3.8 di [8]

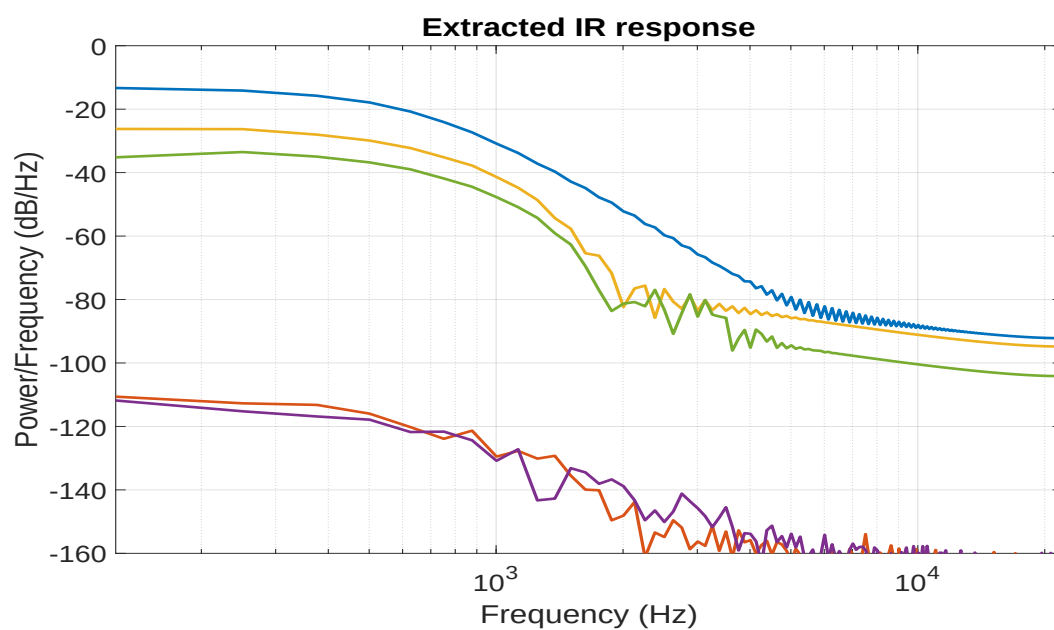


Figura 77: Risposta in frequenza del filtro con cutoff = 500Hz e risonanza = 1

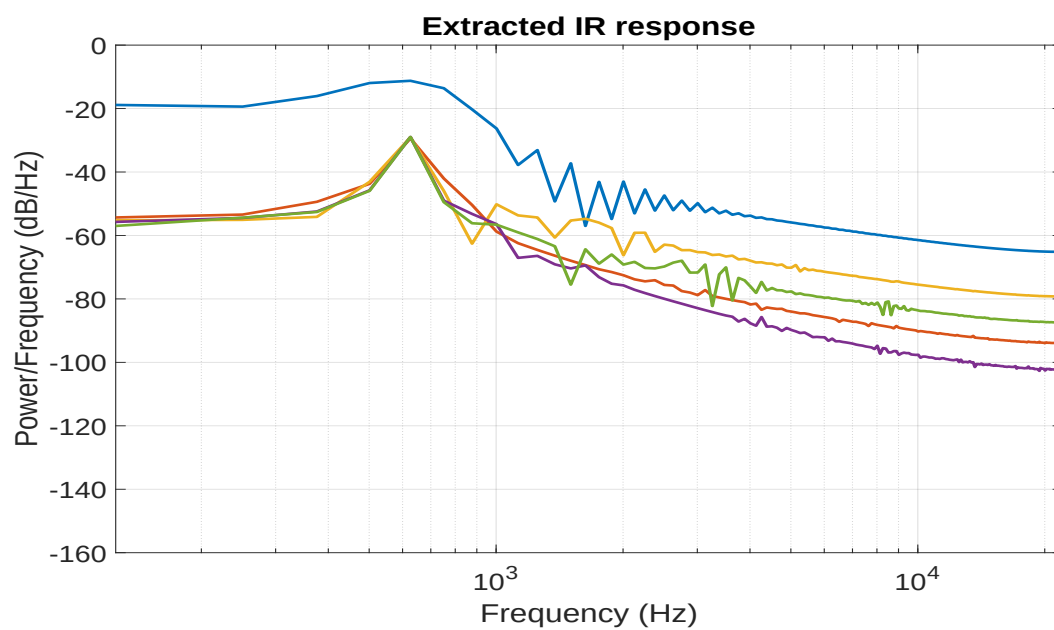


Figura 78: Risposta in frequenza del filtro con cutoff = 500Hz e risonanza = 15

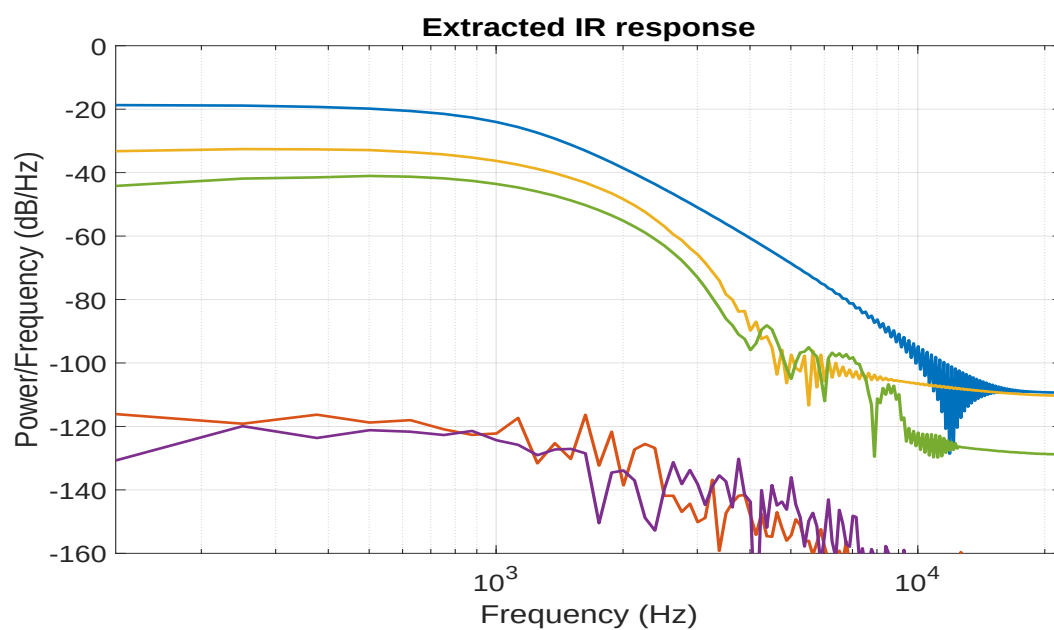


Figura 79: Risposta in frequenza del filtro con cutoff = 1kHz e risonanza = 1

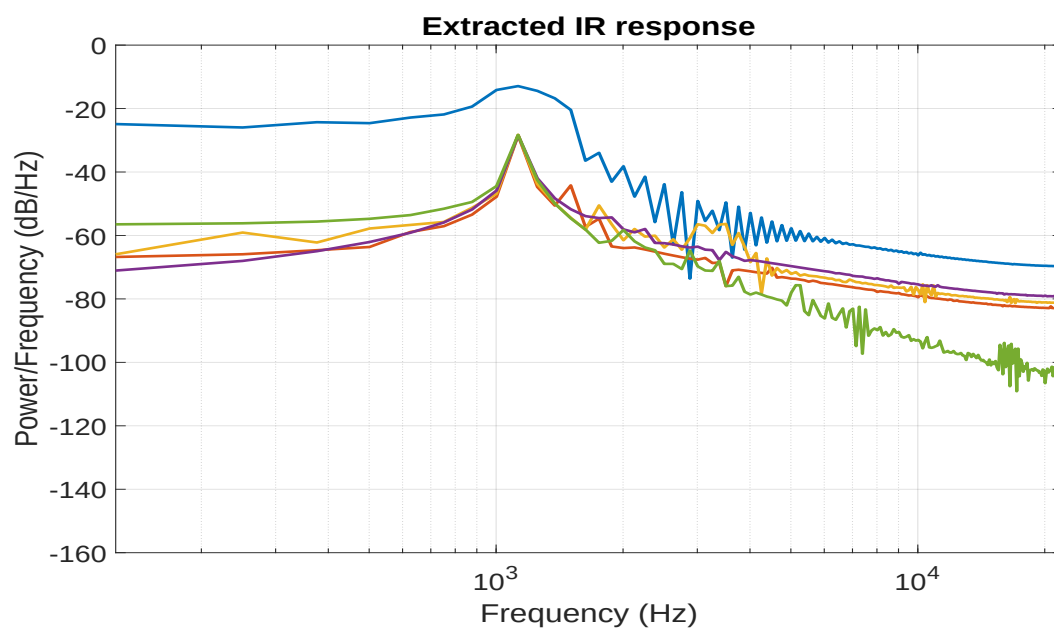


Figura 80: Risposta in frequenza del filtro con cutoff = 1kHz e risonanza = 15

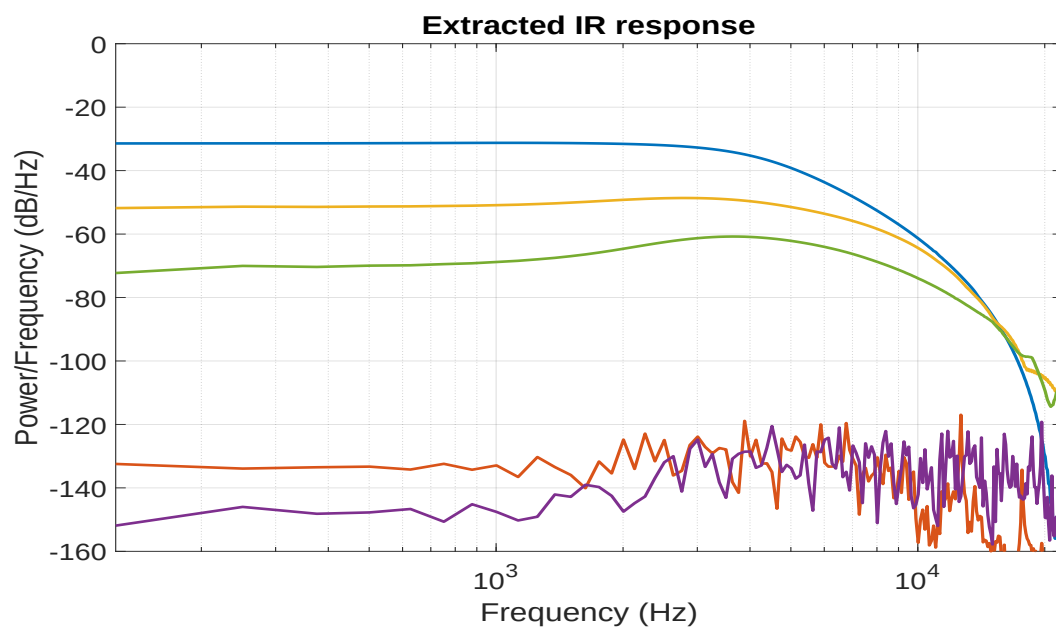


Figura 81: Risposta in frequenza del filtro con cutoff = 5kHz e risonanza = 1

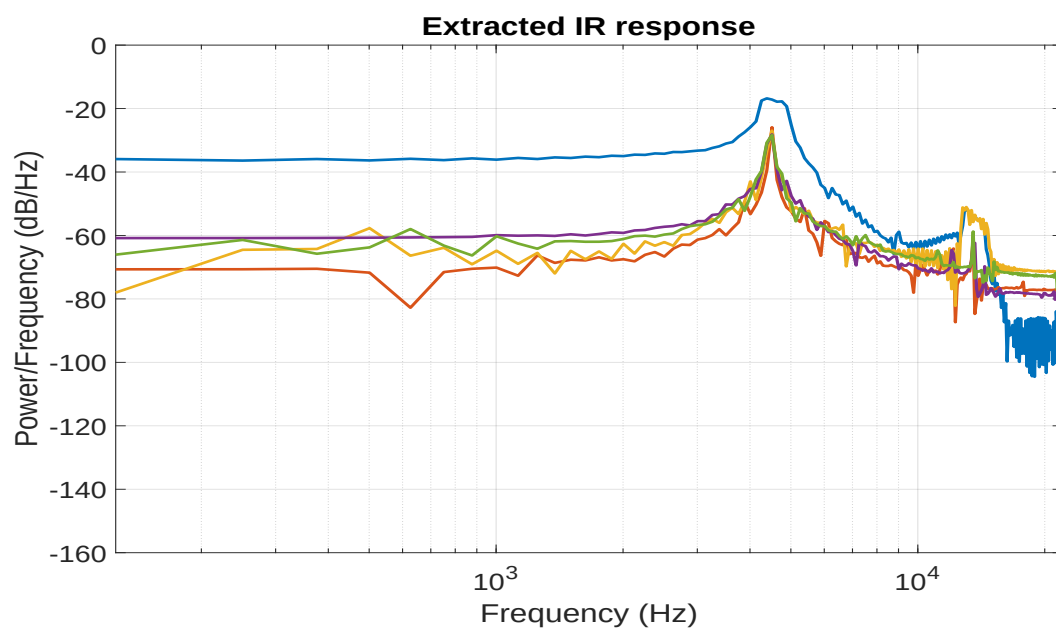


Figura 82: Risposta in frequenza del filtro con cutoff = 5kHz e risonanza = 15

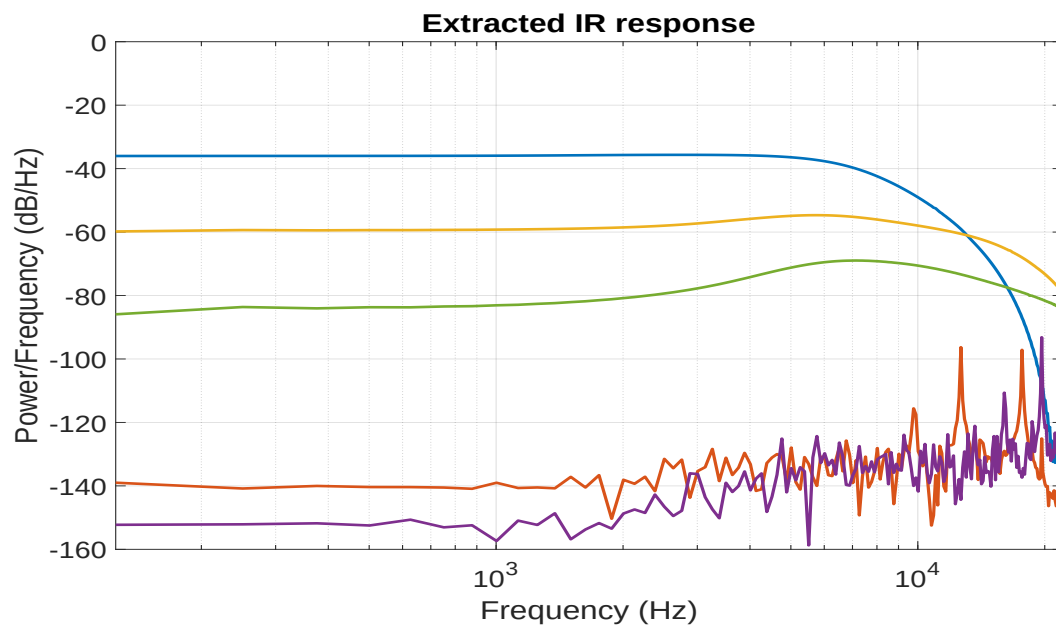


Figura 83: Risposta in frequenza del filtro con cutoff = 10kHz e risonanza = 1

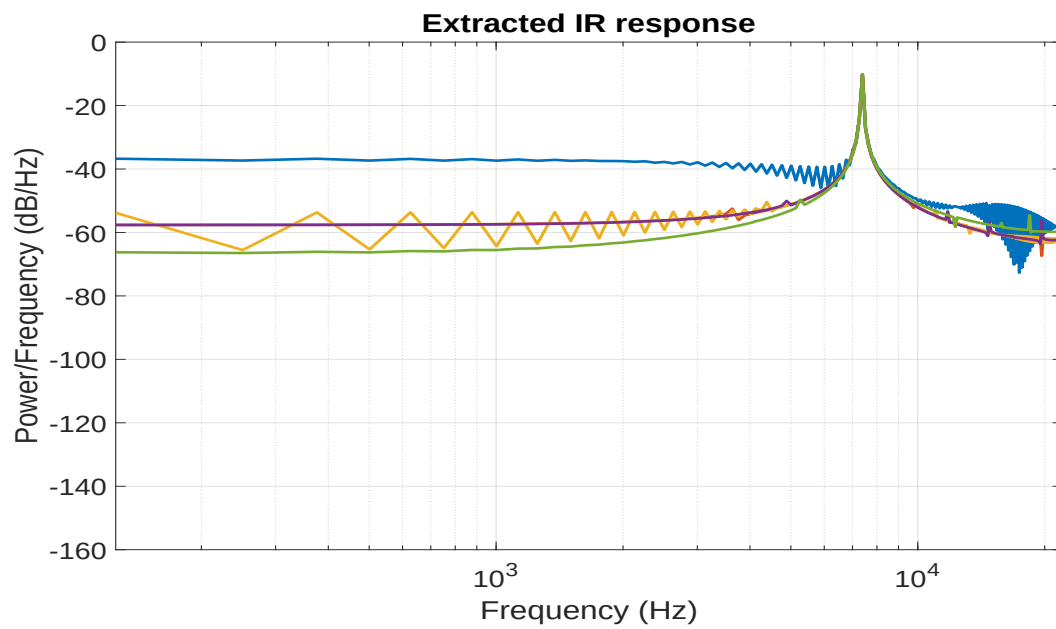


Figura 84: Risposta in frequenza del filtro con cutoff = 10kHz e risonanza = 15

Capitolo 8

Conclusioni

8.1 Conclusioni

Gli obiettivi posti all'inizio del progetto sono stati tutti raggiunti. Ogni funzionalità presente nel pannello del Minimoog è stata testata con successo usando la DAW Ableton Live e il testing degli oscillatori e del filtro è stato fatto usando anche MATLAB. Come si può notare nella sezione precedente le onde generate dagli oscillatori rispecchiano correttamente quelle della versione analogica e l'aliasing risulta impercettibile anche a frequenze elevate. Il filtro ha una risposta in frequenza che rispecchia quella del ladder del Minimoog. Inoltre, la modulazione della frequenza di taglio a frequenze audio funziona e, incrementando la risonanza, il filtro entra in uno stato di auto-oscillazione mantenendo un comportamento stabile. Ci sono alcuni miglioramenti che possono essere fatti rispetto questi 2 specifici algoritmi che verranno discussi nella prossima sezione.

8.2 Sviluppi futuri

In questo progetto non sono state implementate tutte le funzionalità del Minimoog: alcune sono state omesse perchè meno rilevanti, altre, invece, per difficoltà di implementazione. La seguente lista comprende le parti non implementate:

- After touch: è un segnale generato quando, dopo aver premuto un tasto, la pressione su di esso aumenta ulteriormente. Grazie ad esso l'intensità della nota suonata può aumentare anche dopo che il tasto è stato premuto.
- Oscillatore a 440Hz: è utilizzato come accordatore per i 3 oscillatori audio presenti nel sintetizzatore. Esso è utile dato che gli oscillatori dei sintetizzatori analogici con il passare del tempo possono perdere l'intonazione. Dal momento che il plugin è digitale questo problema non si pone.

- Rumore Rosso: nel Minimoog è possibile modulare la frequenza di taglio del filtro e gli oscillatori con il rumore rosa e il rumore rosso. Nella versione digitale la modulazione è invece fatta dal rumore bianco e il rumore rosa.
- Retroazione del segnale di uscita: nella versione analogica il segnale di output (se non è collegata nessuna fonte audio esterna al sintetizzatore) viene riportato in input e sommato a quello degli oscillatori permettendo di modificare il timbro del suono e di distorcerlo. Nella versione digitale non c'è questa funzionalità, ma è solo possibile ricevere un segnale audio esterno.
- Auto-modulazione dell'oscillatore 3: nella versione analogica questo oscillatore può modulare in frequenza sè stesso, mentre nella versione digitale ciò non è possibile per difficoltà di implementazione.
- Segnali di controllo in input: nella versione analogica ci sono dei jack ai quali può essere fornito in input un segnale controllato in tensione per modificare: la frequenza degli oscillatori, il livello del volume, la frequenza di taglio del filtro e l'involuppo ADSR. Inoltre è possibile fornire in input un segnale esterno da usare come fonte di modulazione per oscillatori e filtro.
- Segnali di controllo in output: come nel caso precedente, esistono dei jack di uscita che permettono di inviare dei segnali di controllo diversi. Essi possono essere generati da: la frequenza della nota premuta sulla tastiera, la pressione di un tasto, il livello di pressione applicato sul tasto e la after-pressure. Sia questi segnali che quelli al punto precedente non sono stati implementati.

La seguente lista comprende invece i miglioramenti che possono essere apportati alle funzionalità presenti:

- L'onda triangolare quando viene modulata assume valori oltre il limite consentito e il segnale viene clippato generando dei suoni sgradevoli. Il problema è stato parzialmente risolto usando un leaky integrator variabile nel quale la posizione del polo del filtro varia al variare della frequenza di modulazione. Nonostante questa modifica abbia portato a dei miglioramenti in alcuni casi il segnale clippa ancora.
- Quando è suonata velocemente una melodia con la tastiera l'onda triangolare clippa e sono generati gli stessi suoni del caso precedente. Una possibile soluzione a questo problema (e probabilmente per migliorare il precedente) potrebbe essere quella di usare un leaky integrator ad-hoc come quello mostrato nell'articolo [6].
- La risposta in frequenza del filtro può essere migliorata e resa il più possibile simile all'originale definendo una funzione di mapping più precisa tra

la frequenza analogica ω_c e la corrispettiva frequenza digitale $\tilde{\omega}_c$. Ulteriori informazioni sono presenti al capitolo 3.8 del libro [8].

- L'interfaccia del Minimoog potrebbe essere migliorata non solo esteticamente ma anche resa più fruibile permettendo il ridimensionamento dell'intera finestra o di un singolo blocco.
- Alcune parti del progetto possono essere ottimizzate per evitare l'uso eccessivo della CPU. Per esempio quando lo switch di una fonte audio è spento, invece che generare il suono e moltiplicarlo per zero dall'ADSR, può essere usata una condizione per non generare del tutto il suono riducendo lo sforzo computazionale.

Appendice A

Informazioni aggiuntive sul testing degli oscillatori

A.0.1 Onda dente di sega

La seguente tabella riporta il valore α con cui è moltiplicata ciascuna armonica e la differenza in potenza tra la fondamentale e le successive armoniche in decibel.

Dente di sega	
α	Potenza (dB)
1/2	-6.02
1/3	-9.54
1/4	-12.04
1/5	-13.98
1/6	-15.56
1/7	-16.9
1/8	-18.06
1/9	-19.08
1/10	-20
1/11	-20.82

Nelle seguenti tabelle è riportata nella prima colonna la frequenza di ogni armonica del segnale e nella seconda l'escursione tra la potenza ideale dell'armonica e la potenza effettivamente misurata.

Dente di sega - Do2 (fig: 63)		Dente di sega - Do3 (fig: 64)	
Frequenza (Hz)	Potenza (dBFS)	Frequenza (Hz)	Potenza (dBFS)
65.39	0	130.79	0
130.83	-0.01	261.64	-0.74
196.22	-1.05	392.43	-1.84
261.61	-0.07	523.22	+0.93
327.05	0	654.08	-1.29
392.44	-1.05	784.87	-1.61
457.83	-0.1	915.72	+2.11
523.27	+0.03	1046.51	-1.67
588.66	-1.97	1177.3	+1.65
654.05	-0.95	1308.15	-1.06
Errore medio	0.523	Errore medio	1.29

Dente di sega - Do4 (fig: 65)		Dente di sega - Do5 (fig: 66)	
Frequenza (Hz)	Potenza (dBFS)	Frequenza (Hz)	Potenza (dBFS)
261.63	0	523.25	0
523.25	0	1046.5	0.1
784.88	-0.01	1569.75	-0.03
1046.5	-0.01	2093	-0.05
1308.13	-0.02	2616.25	-0.07
1569.75	-0.03	3139.5	-0.1
1831.38	-0.04	3662.75	-0.14
2093	-0.05	4186	-0.19
2354.63	-0.06	4709.25	-0.24
2616.25	-0.07	5232.5	-0.29
Errore medio	0.029	Errore medio	0.121

Dente di sega - Do6 (fig: 67)		Dente di sega - Do7 (fig: 68)	
Frequenza (Hz)	Potenza (dBFS)	Frequenza (Hz)	Potenza (dBFS)
1046.5	0	2093	0
2093	-0.04	4186	-0.14
3139.5	-0.09	6279	-0.38
4186	-0.18	8372	-0.73
5232.5	-0.28	10465	-1.20
6279	-0.42	12558	-1.81
7325.5	-0.58	14651.06	-1.91
8372	-0.77	16744.06	-0.26
9418.5	-0.99	18837.06	-0.42
10465	-1.23	20930.06	-13.92
Errore medio	0.458	Errore medio	2.077

Dente di sega - Do8 (fig: 69)	
Frequenza (Hz)	Potenza (dBFS)
4186	0
8372	-0.59
12558	-1.67
16744.06	-0.11
20930.06	-13.7
Errore medio	3.214

Dai dati si può notare che le frequenze delle fondamentali hanno un valore uguale a quello mostrato in tabella 7.1 o che si discosta di un fattore dell'ordine dei centesimi per il Do2 e il Do3. L'escursione tra la potenza delle armoniche misurata e quella ideale è nella maggior parte dei casi minore del decibel salvo in alcuni in cui raggiunge valori nell'intorno di 2 dB. Le 2 anomalie principali che si possono notare dalle tabelle sono le escursioni misurate per l'ultima armonica (20930.06Hz) della nota Do7 e Do8. I valori sono così elevati dato che i sinc del treno di impulsi sono finestrati usando una finestra di Hamming con una frequenza di taglio pari a quella di Nyquist. Dato che le 2 armoniche hanno una frequenza vicina a Nyquist, la loro potenza sarà maggiormente attenuata dalla finestra. Un altro motivo potrebbe essere l'effetto di smoothing del leaky integrator applicato al treno di impulsi il quale attenuata le alte frequenze.

A.0.2 Onda quadra

Le seguenti tabelle sono analoghe a quelle viste nel paragrafo precedente per l'onda dente di sega.

Quadra	
α	Potenza (dB)
1/3	-9.54
1/5	-13.98
1/7	-16.9
1/9	-19.08
1/11	-20.82
1/13	-22.28
1/15	-23.52
1/17	-24.61
1/19	-25.58
1/21	-26.44

Quadra - Do2 (fig: 70)		Quadra - Do3 (fig: 71)	
Frequenza (Hz)	Potenza (dBFS)	Frequenza (Hz)	Potenza (dBFS)
65.39	0	130.81	0
196.22	+1.33	392.44	+0.01
327.05	-0.03	654.06	+0.01
457.83	+0.16	915.69	0
588.66	+1.37	1177.31	-0.01
719.49	-0.05	1438.94	-0.02
850.27	+0.14	1700.56	-0.02
981.10	-1.3	1962.19	-0.03
1111.93	-0.20	2223.81	-0.04
1242.71	+0.32	2485.44	-0.05
Errore medio	0.49	Errore medio	0.019

Quadra - Do4 (fig: 72)		Quadra - Do5 (fig: 73)	
Frequenza (Hz)	Potenza (dBFS)	Frequenza (Hz)	Potenza (dBFS)
261.63	0	523.25	0
784.88	-0.01	1569.75	-0.03
1308.13	-0.02	2616.25	-0.07
1831.38	-0.04	3662.75	-0.14
2354.63	-0.06	4709.25	-0.24
2877.88	-0.09	5755.75	-0.36
3401.13	-0.12	6802.25	-0.50
3924.38	-0.17	7848.75	-0.68
4447.63	-0.21	8895.25	-0.88
4970.88	-0.26	9941.75	-0.25
Errore medio	0.098	Errore medio	0.315

Quadra - Do6 (fig: 74)		Quadra - Do7 (fig: 75)	
Frequenza (Hz)	Potenza (dBFS)	Frequenza (Hz)	Potenza (dBFS)
1046.5	0	2093	0
3139.5	-0.09	6279	-0.38
5232.5	-0.28	10465	-1.2
7325.5	-0.58	14651.06	-1.91
9418.5	-0.99	18837.06	-0.42
11511.5	-1.52	21076.94	-46.37
13604.5	-2.21	-	-
15697.56	-1.12	-	-
17790.56	+0.34	-	-
19883.56	-4.41	-	-
Errore medio	1.154	Errore medio	8.38

Quadra - Do8 (fig: 76)	
Frequenza (Hz)	Potenza (dBFS)
4186	0
12558	-1.67
20930.06	-13.78
Errore medio	5.15

Anche in questo caso i valori di frequenza delle fondamentali sono uguali o si discostano dal valore atteso di un fattore dell'ordine dei centesimi per il Do2 e il Do3. L'escursione tra la potenza delle armoniche misurata e quella ideale è nella maggior parte dei casi minore del decibel salvo in alcuni in cui raggiunge valori nell'intorno di 1.5 dB. Anche in questo caso le armoniche con una frequenza vicina a quella di Nyquist hanno una potenza molto minore rispetto quella teorica calcolata per gli stessi motivi spiegati per l'onda dente di sega.

Bibliografia

- [1] Timothy Scott Stilson. *Efficiently-variable non oversampled algorithms in virtual analog music synthesis - A root locus perspective*. PhD thesis, Giugno 2006.
- [2] Tim Stilson and Julius Smith. Alias-free digital synthesis of classic analog waveforms, 1996.
- [3] Vesa Välimäki and Antti Huovilainen. Antialiasing oscillators in subtractive synthesis. *IEEE Signal Processing Magazine*, 24:116–125, 2007.
- [4] Juhan Nam, Vesa Välimäki, Jonathan Abel, and Julius Smith. Efficient antialiasing oscillator algorithms using low-order fractional delay filters. *Audio, Speech, and Language Processing, IEEE Transactions on*, 18:773 – 785, 06 2010.
- [5] Jussi Pekonen, Juhan Nam, Julius Smith, and Vesa Välimäki. Optimized polynomial spline basis function design for quasi-bandlimited classical waveform synthesis. *IEEE Signal Process. Lett.*, 19:159–162, 03 2012.
- [6] Sebastian Kraft and Udo Zölzer. Lp-blit : Bandlimited impulse train synthesis of lowpass-filtered waveforms. 2017.
- [7] Stephan Tassart. Band-limited impulse train generation using sampled infinite impulse responses of analog filters. *Audio, Speech, and Language Processing, IEEE Transactions on*, 21:488–497, 03 2013.
- [8] Vadim Zavalishin. *The art of VA filter design*. 2.1.0 edition, Ottobre 2018.
- [9] Timothy S. Stilson and Julius Orion Smith. Analyzing the moog vcf with considerations for digital implementation. Ottobre 2008.
- [10] Antti Huovilainen. Non-linear digital implementation of the moog ladder filter. 2004.

- [11] Antti Huovilainen and Vesa Välimäki. New approaches to digital subtractive synthesis. 09 2005.
- [12] Stefano D'Angelo and Vesa Välimäki. An improved virtual analog model of the moog ladder filter. pages 729–733, 10 2013.
- [13] MoogMusic. *Minimoog model D user's manual*.

Progetto sviluppato presso il Laboratorio di Informatica Musicale
<https://www.lim.di.unimi.it>